

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Analiza imaginilor cu fund de ochi pentru ajutor în
diagnosticul retinopatiei diabetice**

Lucrare de dizertație

Prezentată ca cerință parțială pentru obținerea
titlului de *Master*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației*

Conducător științific
Conf. dr. ing. Laura Maria FLOREA

Absolvent
Stancovici Marian

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Master* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Stancovici Marian

.....

Cuprins

Lista figurilor	iv
Lista tabelelor	v
Lista acronimelor	vii
1. Introducere	1
2. Fundamente teoretice	4
2.1. Diabetul zaharat și complicațiile sale oculare	4
2.2. Anatomia ochiului și retina	5
2.3. Retinopatia diabetică – definiție, stadii de evoluție și simptome	7
2.4. Imagistica de fund de ochi (”fundus photography”)	9
2.5. Diagnosticul clinic – procesul actual și limitările sale	10
2.6. Învățarea automată și învățarea profundă în analiza imaginilor medicale	11
2.7. Rețele neuronale convoluționale	13
2.8. Transfer learning și modele pre-antrenate	16
3. Stadiul actual al cercetării	18
3.1. Analiza seturilor de date disponibile	18
3.2. Metoda bazată pe arhitectura Inception-V3 [1]	20
3.3. “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]	21
3.4. “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]	22
3.5. Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]	23
3.6. Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]	24
3.7. Arhitectura “Dual Branch” [6]	26
3.8. Abordarea “Explainable AI” [7]	26
3.9. Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]	27
3.10. Concluzii	28
4. Setul de date și preprocesarea imaginilor	29
4.1. Sursele de date utilizate	29
4.2. Distribuția claselor	30
4.3. Construirea seturilor augmentate și preprocesate	32
4.4. Preprocesarea Ben Graham	32
4.5. Încărcarea datelor în experimente	33
4.6. Augmentarea datelor	33
4.7. Împărțirea datelor și controlul evaluării	34
5. Metodologia propusă	35
5.1. Fluxul general al sistemului	35
5.2. Modelele analizate	35

5.3.	Arhitectura ResNet50	36
5.4.	Arhitectura EfficientNet-B3	36
5.5.	Arhitectura InceptionV3	37
5.6.	Arhitectura Inception-ResNet-V2	37
5.7.	Funcții de pierdere utilizate	38
5.8.	Optimizatorul și hiperparametrii de antrenare	38
5.9.	Strategia de evaluare	39
5.10.	Implementarea procesului de antrenare	40
5.11.	Optimizări recente ale protocolului experimental	40
6.	Rezultate experimentale	42
6.1.	Configurația experimentală	42
6.2.	Evoluția funcției de pierdere	43
6.3.	Evoluția acurateții pe setul de validare	44
6.4.	Analiza scorului AUC	45
6.5.	Analiza curbelor ROC multclasă	46
6.6.	Analiza matricelor de confuzie	48
6.7.	Comparație între modelele testate	49
6.8.	Interpretarea erorilor de clasificare	50
7.	Aplicația de inferență	52
7.1.	Scopul aplicației dezvoltate	52
7.2.	Arhitectura aplicației	52
7.3.	Componenta backend	52
7.4.	Componenta frontend	53
7.5.	Fluxul de utilizare	54
7.6.	Integrarea modelului antrenat	54
7.7.	Limitări practice ale aplicației	55
8.	Concluzii și direcții viitoare	56
8.1.	Concluzii generale	56
8.2.	Rezultatele obținute față de obiectivele inițiale	56
8.3.	Limitările lucrării	56
8.4.	Direcții viitoare	57
	Bibliografie	59
	Anexa A. Cod	62

Lista figurilor

2.1.	Structura anatomica a fundului de ochi	6
2.2.	Reprezentare grafică a diferențelor vizuale între o imagine de fund de ochi fără semne patologice și una cu retinopatie diabetică proliferativă	8
2.3.	Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională	13
2.4.	Schema generală a unei rețele neuronale convoluționale	14
2.5.	Schema unei arhitecturi ResNet50	14
2.6.	Schema unei arhitecturi Inception-v3	15
2.7.	Schema unei arhitecturi EfficientNet-B3	15
2.8.	Schema unei arhitecturi Inception-ResNet-v2	16
3.1.	Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]	19
3.2.	Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2] . . .	21
3.3.	Arhitectura U-net modificată [4]	24
3.4.	Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]	25
4.1.	Exemple de imagini din setul <i>Diabetic_Balanced_Data</i> pentru fiecare clasă de severitate	29
4.2.	Exemple de imagini din setul <i>Diabetic_Balanced_Aug_Ben_Graham_matched</i> pentru fiecare clasă de severitate	30
4.3.	Distribuția imaginilor pe clase și subseturi în <i>Diabetic_Balanced_Data</i>	31
5.1.	Structura blocurilor MBConv în EfficientNet-B3	37
5.2.	Exemplu de hărți Grad-CAM pentru un model ResNet50 antrenat cu Focal Loss și Adam	40
6.1.	Evoluția antrenării pentru Inception-ResNet-V2 cu Focal Loss în experimentele inițiale	44
6.2.	Evoluția funcției de pierdere pentru ResNet50 pe Balanced Aug + APTOS . . .	44
6.3.	Evoluția acurateții pentru ResNet50 antrenat pe Balanced + APTOS	45
6.4.	Evoluția AUC pentru ResNet50 pe Balanced Aug + APTOS	46
6.5.	Curbe ROC pentru InceptionV3 în experimentele inițiale	47
6.6.	Curbe ROC pentru ResNet50 antrenat pe Balanced + APTOS și testat pe APTOS	47
6.7.	Matrice de confuzie normalizată pentru ResNet50 pe Balanced, $\gamma=1.6$. . .	48
6.8.	Matrice de confuzie normalizată pentru ResNet50 antrenat pe Balanced + APTOS și testat pe APTOS	49
6.9.	Precision, recall și F1 pe clase pentru ResNet50 testat pe Balanced după antrenare Balanced + APTOS	50
6.10.	Exemplu Grad-CAM pentru ResNet50 evaluat pe Balanced după antrenare Balanced + APTOS	50
7.1.	Zona de recomandări afișată în aplicația de inferență după obținerea predicției .	53
7.2.	Interfața aplicației de inferență dezvoltate pentru clasificarea retinopatiei diabetice	54
7.3.	Meniul dropdown pentru selecția modelului în aplicația de inferență	54

Lista tabelelor

2.1.	Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice	7
3.1.	Tabel comparativ între seturile de date disponibile	20
4.1.	Distribuția imaginilor în setul de date <i>Diabetic_Balanced_Data</i>	30
4.2.	Distribuția imaginilor în setul <i>Diabetic_Balanced_Aug_Ben_Graham_matched</i> . . .	31
4.3.	Distribuția imaginilor în setul <i>APTOS Ben Graham matched</i>	31
6.1.	Rezultatele inițiale pe setul <i>Diabetic_Balanced_Data</i>	42
6.2.	Analiza parametrului γ pentru ResNet50 cu Focal Loss pe Balanced	43
6.3.	Rezultate cross-dataset și evaluare separată pe domenii	43

Lista acronimelor

SOTA = State of the Art

DR = Diabetic Retinopathy

Deep DRiD = Deep Diabetic Retinopathy Image Dataset

UoA-DR = University of Auckland Diabetic Retinopathy

IDRiD = Indian Diabetic Retinopathy Image Dataset

DDR = Dataset of Diabetic Retinopathy

DRIVE = Digital Retinal Images for Vessel Extraction

QKW = Quadratic Weighted Kappa

Capitolul 1

Introducere

Contextul și motivația lucrării

Retinopatia diabetică reprezintă o provocare majoră de sănătate, fiind principala cauză a orbirii prevenibile în rândul populației active. Creșterea continuă a numărului de pacienți cu diabet generează presiune asupra sistemelor medicale, depășind adesea capacitatea de diagnosticare a specialiștilor. Din cauză că această afecțiune poate evolua asimptomatic în fazele inițiale, identificarea pacienților la risc este critică înainte ca leziunile retiniene să devină ireversibile, subliniind necesitatea unor soluții de monitorizare scalabile și eficiente.

Complexitatea diagnosticării este dată de necesitatea unei clasificări precise, conform standardelor internaționale care împart boala în cinci clase de severitate [3], [6]. Procesul impune medicului să distingă între stadiile incipiente, marcate doar de microanevrisme discrete [9], și formele avansate, caracterizate de hemoragii și neovascularizație. Această granularitate a diagnosticului este dificilă de menținut constant în programele de "screening" în masă, unde oboseala și subiectivitatea specialiștilor pot duce la erori de clasificare cu impact asupra deciziilor terapeutice.

Pentru a depăși aceste limitări, domeniul imagisticii medicale a adoptat soluții bazate pe inteligență artificială. Trecerea de la metodele clasice de procesare a imaginii [10] la algoritmi de învățare profundă ("Deep Learning") a marcat un punct de inflexiune, validat clinic prin studii extensive [1]. Rețelele neurale convoluționale au eliminat necesitatea definirii manuale a regulilor vizuale [11], având capacitatea de a învăța trăsături complexe direct din datele brute [7] și de a identifica corelații subtile între texturi și patologia ochiului.

Evoluția recentă a acestor tehnologii a permis tranziția de la modele experimentale simple la arhitecturi robuste [5], capabile să opereze în condiții clinice reale. Între timp cercetarea actuală s-a distanțat de clasificarea binară elementară, concentrându-se pe sisteme care pot gestiona diversitatea imaginilor provenite din multiple surse prin tehnici de adaptare la domeniu [8]. Algoritmi moderni demonstrează o flexibilitate crescută la artefacte și zgomot vizual, oferind o consistență a diagnosticului care reușesc să atingă nivelul expertizei umane [7], [2].

Obiectivele lucrării

Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

Aceasta urmărește atingerea următoarelor obiective:

- Analizarea metodelor deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi.

- Căutarea seturilor de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice.
- Analizarea seturilor de date găsite.
- Propunerea unei metode bazate pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice.
- Analizarea performanței pentru metoda propusă în diverse cazuri de antrenare/testare

Structura lucrării

Lucrarea este organizată în opt capitole principale, urmărind trecerea de la contextul medical și stadiul actual al cercetării la metodologia propusă, rezultatele experimentale și aplicația de inferență dezvoltată.

- **Capitolul 1 – Introducere:** Se prezintă contextul medical al retinopatiei diabetice, motivația temei, obiectivele lucrării și modul de organizare a documentului.
- **Capitolul 2 – Fundamente teoretice:** Se introduc noțiunile medicale și tehnice necesare: diabetul zaharat, retina, stadiile retinopatiei diabetice, imaginile de fund de ochi, rețelele neuronale convoluționale și principiile învățării profunde.
- **Capitolul 3 – Stadiul actual al cercetării:** Se analizează metodele existente pentru detecția și clasificarea automată a retinopatiei diabetice, cu accent pe arhitecturi CNN, ”transfer learning” și tehnici de generalizare între seturi de date.
- **Capitolul 4 – Setul de date și preprocesarea imaginilor:** Se descriu datele utilizate, distribuția claselor, problemele de dezechilibru, etapele de preprocesare, augmentarea imaginilor și împărțirea în seturi de antrenare, validare și testare.
- **Capitolul 5 – Metodologia propusă:** Se prezintă fluxul general al sistemului, modelele analizate, arhitecturile ResNet50, EfficientNet-B3, InceptionV3 și Inception-ResNet-V2, funcțiile de pierdere utilizate, optimizatorul, hiperparametrii și strategia de evaluare.
- **Capitolul 6 – Rezultate experimentale:** Include analiza curbelor de antrenare, a acurateții, a scorului AUC, a curbelor ROC multclasă și a matricelor de confuzie, precum și comparația între configurațiile testate.
- **Capitolul 7 – Aplicația de inferență:** Descrie aplicația web dezvoltată pentru testarea modelului, arhitectura frontend-backend, fluxul de utilizare și integrarea modelului antrenat.
- **Capitolul 8 – Concluzii și direcții viitoare:** Se sintetizează rezultatele obținute, contribuțiile lucrării, limitările identificate și posibilele direcții de dezvoltare ulterioară.

Capitolul 2

Fundamente teoretice

2.1 Diabetul zaharat și complicațiile sale oculare

Diabetul zaharat este o afecțiune metabolică cronică definită prin menținerea unor valori crescute ale glicemiei pe perioade lungi. În mod fiziologic, nivelul glucozei din sânge este reglat în principal prin acțiunea insulinei, hormon produs de pancreas, care facilitează absorbția glucozei de către celule și utilizarea acesteia ca sursă de energie. În cazul diabetului, acest mecanism este perturbat fie prin absența sau producerea insuficientă de insulină, fie prin rezistența țesuturilor la acțiunea acesteia. Din punct de vedere clinic, cele mai cunoscute forme sunt diabetul zaharat de tip 1, asociat cu distrugerea celulelor beta pancreatice și deficit insulinar sever, și diabetul zaharat de tip 2, asociat mai ales cu rezistența la insulină și cu factori metabolici precum obezitatea, sedentarismul și predispoziția genetică [12].

Importanța diabetului pentru această lucrare nu se limitează la caracterul său metabolic, ci derivă în special din complicațiile pe care le produce. Hiperglicemia cronică afectează vasele de sânge de calibru mic și mare, nervii periferici, rinichii, cordul și structurile oculare. Complicațiile microvasculare sunt de interes major în oftalmologie, deoarece retina este un țesut puternic vascularizat și sensibil la variațiile aportului de oxigen și nutrienți. Vasele retiniene au un diametru redus, iar peretele vascular poate fi deteriorat gradual de modificările biochimice asociate diabetului. În timp, apar alterări ale permeabilității vasculare, microanevrisme, hemoragii, exudate, zone ischemice și, în stadiile avansate, vase de neoformație [12], [13].

Retina este una dintre puținele regiuni ale organismului în care microcirculația poate fi observată direct prin imagistică neinvazivă. Această particularitate transformă examinarea fundului de ochi într-un instrument esențial pentru monitorizarea efectelor diabetului asupra sistemului vascular. Din perspectivă medicală, modificările retiniene reflectă atât afectarea oculară locală, cât și severitatea afectării microvasculare. Din perspectivă computațională, imaginile retiniene oferă un suport vizual bogat, în care semnele patologice pot fi analizate prin metode automate de procesare a imaginilor și prin modele de învățare profundă [1], [14].

Complicațiile oculare ale diabetului includ retinopatia diabetică, edemul macular diabetic, cataracta și glaucomul. Dintre acestea, retinopatia diabetică este cea mai importantă în contextul "screening-ului" automat, deoarece evoluează de obicei lent, poate fi asimptomatică în fazele inițiale și poate produce pierdere ireversibilă de vedere dacă nu este identificată și tratată la timp [13]. Edemul macular diabetic este o complicație strâns legată de retinopatie și apare atunci când permeabilitatea vasculară crescută determină acumularea de lichid în zona maculei, regiunea responsabilă pentru vederea centrală fină [13]. Chiar și atunci când retinopatia nu este în stadiu proliferativ, afectarea maculară poate reduce semnificativ acuitatea vizuală.

În contextul acestei lucrări, cele mai relevante complicații oculare pot fi sintetizate astfel:

- **Retinopatia diabetică** – afectează vasele retiniene și reprezintă principala țintă a sistemului automat de clasificare.

- **Edemul macular diabetic** – implică acumularea de lichid în zona maculei și poate reduce vederea centrală.
- **Cataracta și glaucomul** – nu sunt obiectivul direct al clasificării, dar pot influența calitatea imaginii și interpretarea clinică.

Din punctul de vedere al sistemelor automate, retinopatia diabetică este o problemă dificilă deoarece semnele bolii au dimensiuni, forme și contraste diferite, iar severitatea trebuie estimată pe o scară ordinală, nu doar printr-o decizie binară sănătos/bolnav. Această particularitate justifică abordarea multiclasă detaliată în secțiunea următoare, unde sunt prezentate stadiile bolii și semnele vizuale asociate fiecărui nivel de severitate [1], [3], [15].

2.2 Anatomia ochiului și retina

Ochiul uman este un sistem optic complex, specializat în captarea luminii și transformarea acesteia în semnale nervoase interpretate de creier. Din punct de vedere funcțional, componentele principale implicate în formarea imaginii sunt corneea, cristalinul, umoarea apoasă, corpul vitros și retina. Corneea și cristalinul focalizează lumina, iar retina funcționează ca un strat fotosensibil situat în partea posterioară a globului ocular. În contextul analizei automate a imaginilor, retina este structura de interes, deoarece aici pot fi observate vasele sanguine, discul optic, macula și leziunile asociate retinopatiei diabetice [13], [9].

Retina este formată din mai multe straturi celulare, incluzând fotoreceptori, celule bipolare, celule ganglionare și celule de suport. Fotoreceptorii transformă stimulii luminoși în semnale electrice, iar informația este transmisă prin nervul optic către cortexul vizual. Regiunea centrală a retinei, numită maculă, este responsabilă pentru vederea detaliată, necesară cititului, recunoașterii fețelor și percepției fine a culorilor. În centrul maculei se află foveea, zona cu cea mai mare densitate de conuri și cu acuitatea vizuală maximă. Din acest motiv, afectarea maculei prin edem sau exudate poate avea consecințe majore asupra calității vieții pacientului [16], [13].

În imaginile de fund de ochi, câteva repere anatomice sunt esențiale pentru interpretare. Discul optic apare de obicei ca o regiune circulară sau ovalară, mai luminoasă, din care pornesc vasele retiniene principale. Vasele se ramifică progresiv și formează o rețea vasculară care acoperă suprafața retiniană vizibilă. Macula se observă ca o zonă mai întunecată, situată lateral față de discul optic, iar foveea reprezintă centrul acesteia. Aceste detalii se pot observa în figura de mai jos. Pentru algoritmi de analiză, aceste structuri au rol dublu: pe de o parte oferă informație anatomică utilă, iar pe de altă parte pot produce confuzii. De exemplu, discul optic este o regiune luminoasă care poate fi confundată cu exudatele dacă modelul nu învață contextul anatomic corect.

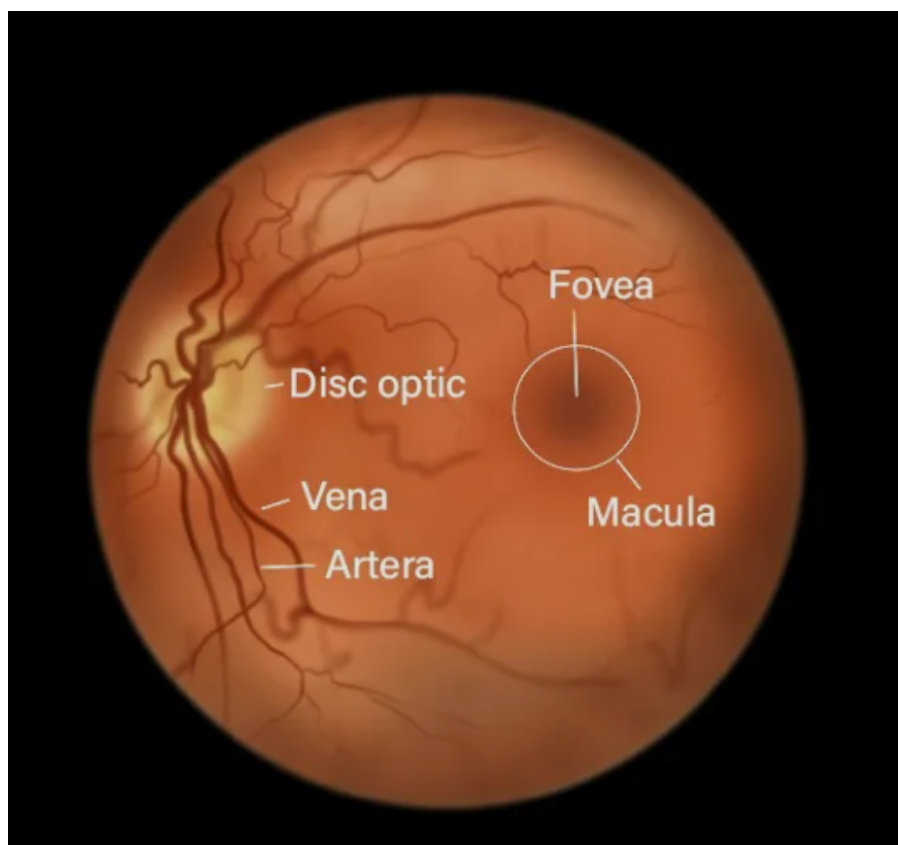


Figura 2.1: Structura anatomica a fundului de ochi

Principalele repere anatomice urmărite într-o imagine de fund de ochi sunt:

- **Discul optic** – regiune luminoasă din care pornesc vasele retiniene principale.
- **Macula** – zonă responsabilă pentru vederea centrală, importantă în evaluarea edemului macular.
- **Fovea** – centrul maculei, asociat cu acuitatea vizuală maximă.
- **Rețeaua vasculară** – structură relevantă pentru identificarea hemoragiilor, microanevrismelor și modificărilor ischemice.

Vasele retiniene au o importanță deosebită în detectarea modificărilor patologice. În diabet, afectarea microvasculară determină fragilizarea pereților capilari și modificări ale fluxului sanguin. Microanevrismele apar ca mici dilatații ale capilarelor, fiind printre cele mai timpurii semne vizibile ale retinopatiei diabetice. Hemoragiile sunt rezultatul ruperii sau scurgerii vaselor deteriorate, iar exudatele apar prin depunerea de lipide și proteine în urma creșterii permeabilității vasculare [16]. În stadiile avansate, ischemia retiniană poate stimula formarea de vase noi, fragile și anormale, fenomen numit neovascularizație [13].

Din perspectiva prelucrării imaginilor, retina prezintă o combinație de structuri globale și detalii locale fine. Structurile globale includ forma câmpului vizual, poziția discului optic și distribuția vaselor principale, iar detaliile locale includ microanevrismele, punctele hemoragice, exudatele mici sau modificările subtile de textură [17], [10].

Un alt aspect relevant este diversitatea aspectului retinei: pigmentarea fundului de ochi, diametrul vaselor și poziția discului optic pot varia între pacienți și între achiziții. Modelele

automate trebuie astfel să distingă între variații anatomice normale și semne patologice reale, iar influența artefactelor de imagine este discutată separat în secțiunea dedicată fotografiei fundus.

2.3 Retinopatia diabetică – definiție, stadii de evoluție și simptome

Retinopatia diabetică este o complicație microvasculară a diabetului zaharat, caracterizată prin afectarea progresivă a vaselor de sânge retiniene. Mecanismele patologice includ lezarea endoteliului vascular, pierderea pericitelor, creșterea permeabilității capilare, ocluzii vasculare și ischemie retiniană. În timp, aceste procese duc la apariția unor leziuni vizibile în imaginile fundului de ochi. În fazele incipiente, pacientul poate să nu observe nicio modificare a vederii, ceea ce face ca "screening-ul" periodic să fie esențial. În fazele avansate, boala poate produce vedere încețoșată, pete în câmpul vizual, scăderea acuității vizuale și, în cazuri severe, pierderea vederii [13].

Clasificarea clinică a retinopatiei diabetice este de obicei organizată pe niveluri de severitate. În lucrările de învățare automată, inclusiv în seturile de date publice folosite frecvent, se utilizează adesea o scară cu cinci clase: [1], [3], [15]

- Clasa 0 – fără retinopatie diabetică
- Clasa 1 – retinopatie diabetică ușoară
- Clasa 2 – retinopatie diabetică moderată
- Clasa 3 – retinopatie diabetică severă
- Clasa 4 – retinopatie diabetică proliferativă

Această formulare este utilă pentru antrenarea modelelor de clasificare, deoarece transformă evaluarea clinică într-o problemă de predicție multiclasă.

Tabela 2.1: Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice

Clasa	Grad de severitate	Semne vizuale urmărite în imagine
0	Fără retinopatie diabetică	Nu sunt observate leziuni specifice; imaginea poate fi considerată normală din punct de vedere al retinopatiei diabetice.
1	Retinopatie ușoară	Microanevrisme izolate, de obicei greu vizibile și ușor de confundat cu zgomot sau artefacte de achiziție.
2	Retinopatie moderată	Număr mai mare de leziuni, cu posibile hemoragii, exudate dure și modificări vasculare vizibile.
3	Retinopatie severă	Afectare vasculară extinsă, numeroase hemoragii și risc crescut de progresie către forma proliferativă.
4	Retinopatie proliferativă	Neovascularizație și risc major de complicații severe, precum hemoragie vitroasă sau dezlipire de retină.

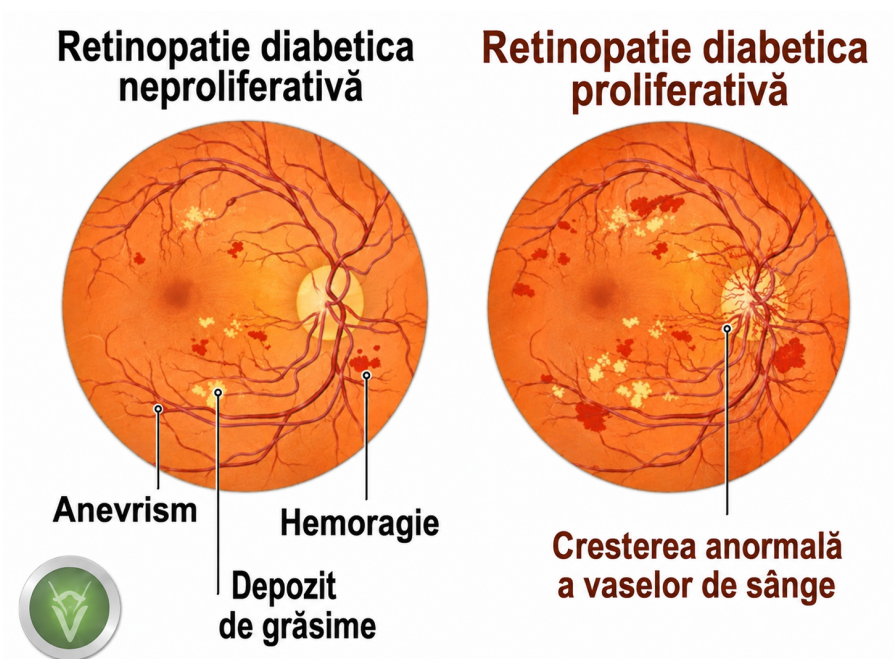


Figura 2.2: Reprezentare grafică a diferențelor vizuale între o imagine de fund de ochi fără semne patologice și una cu retinopatie diabetică proliferativă

Clasele 0 și 1 sunt de obicei cele mai subtile din punct de vedere vizual. Absența semnelor evidente într-o fotografie nu garantează absența completă a riscului, deoarece detectabilitatea leziunilor depinde de calitatea imaginii și de câmpul vizual acoperit. În același timp, retinopatia ușoară poate fi reprezentată doar de câteva microanevrisme, motiv pentru care confuziile între clasele 0, 1 și 2 sunt frecvente în modelele automate [1], [15].

Clasele 3 și 4 sunt asociate cu afectare vasculară mai evidentă și cu risc clinic mai ridicat. Imaginea devine, în general, mai evidentă din punct de vedere patologic, iar modelele automate tind să obțină performanțe mai bune pentru aceste stadii decât pentru cele incipiente. Totuși, identificarea corectă a claselor avansate este critică, deoarece subestimarea severității poate întârzia intervenția medicală. Într-un sistem de screening, o eroare de tip fals negativ pentru o formă proliferativă este mult mai gravă decât o confuzie între două clase adiacente [14].

În afara clasificării pe stadii, este importantă și natura leziunilor observabile. Cele mai importante semne patologice sunt:

- **Microanevrismele** – printre primele semne ale bolii, vizibile ca puncte roșii foarte mici.
- **Hemoragiile** – pot avea forme variate, de la punctiforme la pete mai extinse.
- **Exudatele dure** – regiuni galben-albicioase, bine delimitate, asociate cu scurgeri lipidice.
- **Exudatele moi** – pete vătămate care indică zone de ischemie în stratul fibrelor nervoase.
- **Neovascularizația** – semn al trecerii către forma proliferativă.

Metodele automate pot aborda aceste semne fie prin clasificare globală a imaginii, fie prin segmentarea leziunilor, fie prin combinații multi-task, în care modelul învață simultan mai multe sarcini legate de severitate și localizare [17], [10], [16], [13], [3], [5].

2.4 Imagistica de fund de ochi ("fundus photography")

Fotografia de fund de ochi reprezintă una dintre cele mai utilizate metode de examinare imagistică a retinei. Aceasta permite captarea unei imagini color a suprafeței retiniene, incluzând discul optic, vasele sanguine, macula și eventualele leziuni patologice. Spre deosebire de metode mai complexe, precum angiografia cu fluoresceină sau tomografia în coerență optică, fotografia fundus este relativ rapidă, neinvazivă și potrivită pentru programe de "screening" la scară largă [1]. Din acest motiv, majoritatea seturilor de date utilizate în cercetarea privind detecția automată a retinopatiei diabetice se bazează pe astfel de imagini [7], [15].

În mod obișnuit, o imagine fundus este obținută cu o cameră specializată care iluminează retina și captează lumina reflectată prin pupilă. Achiziția poate fi realizată cu sau fără dilatarea pupilei, iar câmpul vizual poate varia în funcție de echipament. Imaginile pot avea rezoluții diferite, niveluri diferite de contrast și grade variabile de claritate. În plus, pot apărea artefacte precum reflexii luminoase, umbre, zone întunecate, margini circulare, praf pe lentilă sau defocalizare. Toate aceste elemente influențează atât evaluarea umană, cât și performanța algoritmilor de clasificare [15], [8].

Pentru analiza automată, imaginea fundus prezintă mai multe provocări. Prima este variația de iluminare. Unele imagini sunt supraexpuse, iar leziunile luminoase pot fi greu de distins de fundal. Altele sunt subexpuse, iar leziunile roșii sau vasele fine pot deveni slab vizibile. A doua provocare este variația cromatică: tonul general al retinei diferă între pacienți și între camere. A treia provocare este rezoluția foarte mare a imaginilor originale. Leziunile mici pot ocupa doar câțiva pixeli, iar redimensionarea agresivă poate elimina informație relevantă. În același timp, procesarea directă a imaginilor la rezoluție completă poate fi costisitoare computațional [1], [6].

Cele mai frecvente dificultăți întâlnite în imaginile de fund de ochi sunt:

- variații de iluminare și contrast între imagini;
- diferențe cromatice între pacienți și între camere de achiziție;
- rezoluții diferite și dimensiuni mari ale imaginilor originale;
- artefacte precum reflexii, blur, margini întunecate sau câmp vizual incomplet;
- dimensiunea foarte mică a unor leziuni, în special a microanevrismelor.

Preprocesarea imaginilor are rolul de a reduce aceste variații și de a oferi modelului date mai consistente. Etapele frecvente includ decuparea marginilor negre, redimensionarea la o dimensiune standard, normalizarea valorilor pixelilor, corectarea iluminării și aplicarea unor transformări de augmentare. În unele abordări, se utilizează metode pentru evidențierea vaselor sau pentru îmbunătățirea contrastului local. În altele, modelul primește imaginea aproape brută, iar rețeaua neuronală învață singură trăsăturile relevante. Alegerea depinde de dimensiunea setului de date, de arhitectura folosită și de obiectivul experimentului [10], [9], [7].

Într-un flux tipic de lucru, preprocesarea poate include:

- decuparea zonelor negre din jurul câmpului circular al imaginii;
- redimensionarea imaginilor la dimensiunea cerută de model;

- normalizarea intensităților pixelilor;
- corectarea parțială a iluminării neuniforme;
- augmentări precum rotații, flip-uri, modificări de luminozitate sau contrast.

Din punct de vedere practic, preprocesarea trebuie să păstreze leziunile fine, nu doar să uniformizeze aspectul general al imaginii. O redimensionare prea agresivă poate elimina microanevrismele, iar o corecție de contrast nepotrivită poate accentua artefactele. De aceea, etapele de preprocesare sunt alese în funcție de modelul folosit și de rezoluția minimă la care semnele patologice rămân vizibile.

Aceste observații sunt importante pentru proiectarea fluxului experimental: imaginile trebuie aduse la o formă comparabilă fără a elimina informația patologică fină. Diferența dintre abordările bazate pe trăsături proiectate manual și cele bazate pe învățare profundă este discutată în secțiunea următoare [17], [10], [11].

Un element important în folosirea imaginilor fundus pentru clasificare este raportul dintre informația globală și cea locală. O imagine poate conține semne patologice mici, dar decizia finală depinde și de distribuția acestora în câmpul retinian. Un model care analizează doar fragmente locale poate detecta leziuni, dar poate pierde contextul global. Un model care analizează imaginea întreagă poate învăța distribuția generală, dar poate rata detalii fine după redimensionare. Din acest motiv, literatura recentă explorează arhitecturi care combină informații la mai multe scări, ramuri multiple sau mecanisme de atenție [6], [5].

2.5 Diagnosticul clinic – procesul actual și limitările sale

Diagnosticul clinic al retinopatiei diabetice se bazează pe evaluarea imaginilor retiniene de către medici oftalmologi sau specialiști instruiți în screening. În cadrul unui flux standard, pacientului i se realizează una sau mai multe fotografii ale fundului de ochi, iar imaginile sunt analizate pentru identificarea leziunilor specifice. În funcție de severitate, pacientul poate fi încadrat într-o categorie de risc și poate primi recomandări pentru monitorizare, investigații suplimentare sau tratament. În programele de screening, scopul principal este identificarea pacienților care necesită evaluare medicală mai rapidă, înainte de apariția pierderii ireversibile de vedere [1], [14].

Procesul clinic are însă mai multe limitări [1], [15]:

- **Disponibilitatea specialiștilor:** Pe măsură ce numărul pacienților cu diabet crește, volumul imaginilor care trebuie evaluate devine tot mai mare.
- **Variabilitatea opiniei între observatori:** Doi specialiști pot evalua diferit aceeași imagine, mai ales în cazurile de graniță dintre clase.
- **Variabilitatea intra-observator:** Același specialist poate avea discrepanțe în decizie în funcție de oboseală, experiență sau calitatea imaginii.
- **Calitatea imaginilor:** Imaginile slab focalizate, întunecate sau parțial acoperite de artefacte sunt greu de interpretat.

În practică, problema calității imaginii rămâne deosebit de importantă. În unele cazuri, imaginea trebuie repetată, ceea ce crește timpul și costul examinării. În alte cazuri, imaginea poate fi evaluată cu incertitudine, ceea ce afectează consistența diagnosticului. Din acest motiv, unele sisteme moderne includ nu doar clasificarea retinopatiei, ci și estimarea calității imaginii, astfel încât imaginile neinterpretabile să fie semnalate separat [15].

Sistemele automate bazate pe inteligență artificială nu înlocuiesc medicul, dar pot funcționa ca instrumente de suport. Într-un scenariu de "screening", un algoritm poate analiza rapid un număr mare de imagini și poate prioritiza cazurile suspecte. De asemenea, poate oferi o a doua opinie sau poate reduce timpul necesar pentru trierea imaginilor fără semne evidente. Studiile moderne au demonstrat că modelele de învățare profundă pot atinge performanțe ridicate în detectarea retinopatiei diabetice pe imagini fundus [1], [14]. Totuși, integrarea clinică necesită validare riguroasă, interpretabilitate, robustețe la imagini din surse diferite și control al erorilor critice.

O dificultate specifică diagnosticului automat este faptul că nu toate erorile au aceeași importanță. În clasificarea multiclasă, o predicție greșită cu o clasă adiacentă poate fi tolerabilă în anumite contexte, dar o predicție care subestimează severitatea bolii poate întârzia tratamentul. De exemplu, clasificarea unei imagini severe ca fiind fără retinopatie este o eroare mult mai periculoasă decât clasificarea unei imagini moderate ca ușoare. Din acest motiv, evaluarea unui model nu trebuie să se bazeze exclusiv pe acuratețe globală, ci pe un set mai larg de metrici [14], [15]:

- **Sensibilitate** (*Sensitivity / Recall*) – exprimă proporția cazurilor patologice detectate corect de model. În context medical, această metrică este importantă deoarece un scor scăzut indică existența unor cazuri de boală ratate (*false negatives*), ceea ce poate afecta diagnosticul precoce.
- **Specificitate** (*Specificity*) – exprimă proporția cazurilor sănătoase identificate corect. O specificitate ridicată indică faptul că modelul produce puține alarme false și nu clasifică eronat pacienții sănătoși ca fiind bolnavi.
- **Scorul AUC** (*Area Under the ROC Curve*) – evaluează capacitatea modelului de a separa clasele pentru diferite praguri de decizie. Un scor AUC apropiat de 1 indică o separare foarte bună, iar un scor apropiat de 0.5 sugerează o performanță slabă, comparabilă cu alegerea aleatoare.
- **Matricea de confuzie** – oferă o reprezentare detaliată a relației dintre clasele reale și cele prezise. Aceasta permite identificarea tipurilor de erori produse de model și evidențierea claselor care sunt confundate frecvent.
- **Performanța pe fiecare clasă** – este necesară mai ales în cazul seturilor de date dezechilibrate. Prin raportarea valorilor de *precision*, *recall* și *F1-score* pentru fiecare clasă, se poate evalua dacă modelul funcționează bine doar pentru clasele majoritare sau și pentru cele rare, dar importante clinic.

2.6 Învățarea automată și învățarea profundă în analiza imaginilor medicale

Învățarea automată este un domeniu al inteligenței artificiale care urmărește dezvoltarea unor algoritmi capabili să învețe modele din date. În loc ca regulile să fie definite explicit de

programator, sistemul ajustează parametri interni pe baza unor exemple. În cazul clasificării imaginilor medicale, exemplele sunt imagini asociate cu etichete clinice, iar obiectivul modelului este să învețe o funcție care primește o imagine și produce o predicție. Pentru retinopatia diabetică, această predicție poate fi binară, de tipul retinopatie prezentă/absentă, sau multclasă, corespunzătoare severității bolii [1], [14].

Metodele clasice de învățare automată necesitau de obicei o etapă separată de extragere a trăsăturilor. În imaginile retiniene, aceste trăsături puteau include culoarea, textura, forma leziunilor, grosimea vaselor sau distribuția spațială a exudatelor. După extragerea trăsăturilor, un clasificator precum SVM, k-NN sau Random Forest putea fi antrenat pentru a lua decizia finală. Avantajul acestor metode este interpretabilitatea relativ mai mare a unor trăsături proiectate manual. Dezavantajul este dependența puternică de calitatea acestor trăsături și dificultatea de a acoperi toată aria și diversitatea imaginilor reale [17], [10], [9].

Într-o abordare clasică, "pipeline"-ul era de obicei împărțit în etape distincte:

- preprocesarea imaginii și corectarea iluminării;
- extragerea manuală a trăsăturilor relevante;
- selectarea sau reducerea dimensionalității trăsăturilor;
- antrenarea unui clasificator clasic;
- validarea rezultatului pe imagini nefolosite la antrenare.

Învățarea profundă schimbă această paradigmă prin folosirea unor rețele neuronale cu multe straturi, capabile să învețe reprezentări ierarhice [18]. Într-o rețea convoluțională, primele straturi învață filtre simple, precum muchii, contraste locale sau texturi. Straturile intermediare combină aceste filtre în forme mai complexe, iar straturile profunde pot reprezenta structuri relevante pentru sarcina finală. Această ierarhie este potrivită pentru imagini medicale, deoarece semnele patologice apar la scări diferite și în contexte anatomice variate [18], [11].

În procesul de antrenare supervizată, modelul primește perechi imagine-etichetă și produce o predicție. Diferența dintre predicție și eticheta reală este măsurată printr-o funcție de pierdere, iar parametrii rețelei sunt actualizați prin algoritmi de optimizare, de obicei pe baza propagării inverse a gradientului. Pentru clasificarea multclasă, se pot utiliza mai multe tipuri de funcții de pierdere, fiecare abordând problema dintr-o perspectivă diferită:

- **Cross-Entropy (CE)** – este funcția standard pentru problemele de clasificare multclasă. Aceasta măsoară divergența dintre distribuția de probabilitate prezisă de model și distribuția reală a etichetelor. Funcționează optim atunci când setul de date este echilibrat, dar poate favoriza clasele majoritare în scenarii dezechilibrate.
- **Weighted Cross-Entropy (WCE)** – reprezintă o extensie a funcției standard, introdusă pentru a combate dezechilibrul claselor. Prin asocierea unei ponderi mai mari erorilor provenite din clasele rare (minoritare), modelul este penalizat mai sever dacă greșește în aceste cazuri, asigurând o învățare echitabilă și performanțe superioare pentru clasele critice.

- **Focal Loss** – este o funcție avansată concepută special pentru a rezolva problema dezechilibrului extrem și a exemplor greu de separat. Aceasta modifică ecuația standard prin introducerea unui factor de modulare care reduce semnificativ contribuția exemplorlor clasificate ușor și forțează modelul să se concentreze prioritar pe exemplele dificile (*hard examples*) [19].

Dezechilibrul claselor este o problemă frecventă în retinopatia diabetică. În multe seturi de date, clasa fără retinopatie este mult mai numeroasă decât clasele severe. Dacă modelul este antrenat fără corecții, acesta poate obține acuratețe bună prin favorizarea clasei dominante, dar poate avea sensibilitate slabă pentru clasele rare. Din acest motiv, evaluarea trebuie să urmărească performanța pe fiecare clasă, nu doar media globală. Tehnicile de reechilibrare includ ponderarea funcției de pierdere, eșantionarea echilibrată, augmentare mai puternică a claselor minoritare sau folosirea unor metrici robuste la dezechilibru [19], [15].

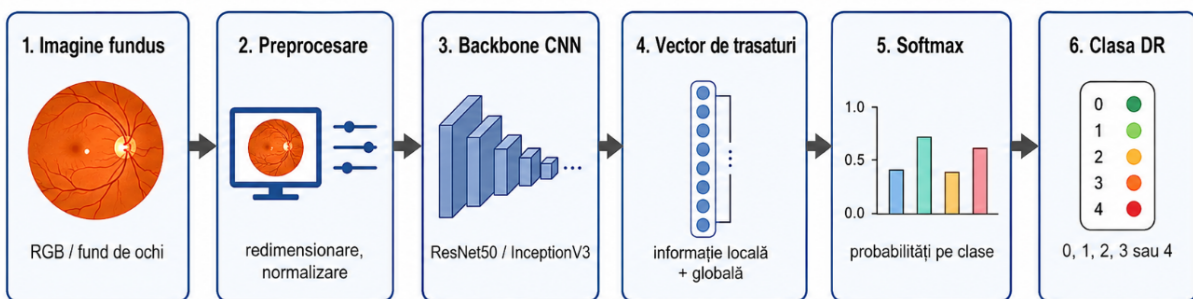


Figura 2.3: Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională

În analiza imaginilor medicale, o altă provocare este dimensiunea limitată a datelor adnotate. Obținerea etichetelor clinice necesită timp și expertiză, iar datele pot fi afectate de restricții etice și de confidențialitate. Una dintre soluțiile folosite frecvent pentru această problemă este transfer learning-ul, prezentat separat la finalul capitolului [2].

2.7 Rețele neuronale convoluționale

Rețelele neuronale convoluționale, cunoscute sub abrevierea CNN, reprezintă una dintre cele mai importante arhitecturi pentru analiza imaginilor. Ele sunt concepute pentru a exploata structura spațială a datelor vizuale. Spre deosebire de o rețea complet conectată, care ar trata fiecare pixel independent de poziția sa, o rețea convoluțională aplică filtre locale pe imagine și păstrează relațiile spațiale dintre pixeli. Această proprietate este esențială pentru imagini retiniene, deoarece semnele patologice sunt definite de configurații locale de culoare, formă și textură [11], [7].

Operația de convoluție constă în deplasarea unui filtru peste imagine și calcularea unui răspuns local. Fiecare filtru învață să detecteze un anumit tipar vizual. În primele straturi, filtrele pot detecta margini, pete sau contraste. În straturile mai profunde, combinațiile de filtre pot detecta structuri mai complexe, precum vase, exudate sau zone hemoragice. Parametrii filtrelor sunt învățați automat în timpul antrenării, ceea ce elimină necesitatea proiectării manuale a descriptorilor vizuali [18], [11].

Pe lângă straturile convoluționale, CNN-urile includ mai multe componente recurente, așa cum se poate observa și în figura 2.4:

- **funcții de activare**, care introduc neliniaritate și permit modelului să aproximeze relații complexe;
- **operații de pooling**, care reduc dimensiunea spațială a hărților de trăsături;
- **straturi de normalizare**, care pot stabiliza antrenarea și accelera convergența;
- **straturi complet conectate sau globale**, care transformă trăsăturile extrase într-o predicție finală;
- **strat softmax**, folosit frecvent pentru a obține probabilități pe clase în clasificarea mult-clasă.

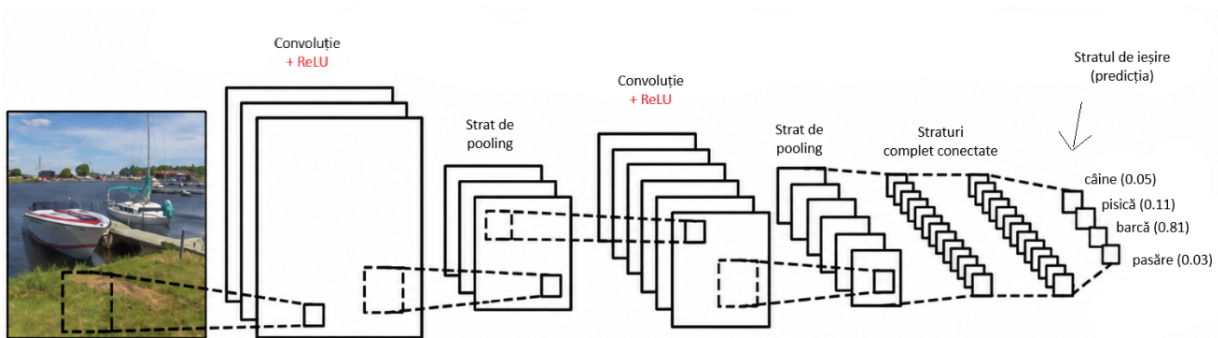


Figura 2.4: Schema generală a unei rețele neuronale convoluționale

Arhitecturile CNN moderne diferă prin adâncime, tipul blocurilor folosite și modul în care gestionează propagarea informației. Dintre acestea, câteva arhitecturi s-au remarcat în mod special în analiza imaginilor medicale:

- **ResNet (Residual Networks)** – introduce conexiuni reziduale (“skip connections”), care permit transmiterea directă a informației peste unul sau mai multe straturi [20]. Această inovație atenuează problema dispariției gradientului și previne degradarea performanței în rețelele foarte adânci, facilitând învățarea unor reprezentări vizuale complexe.

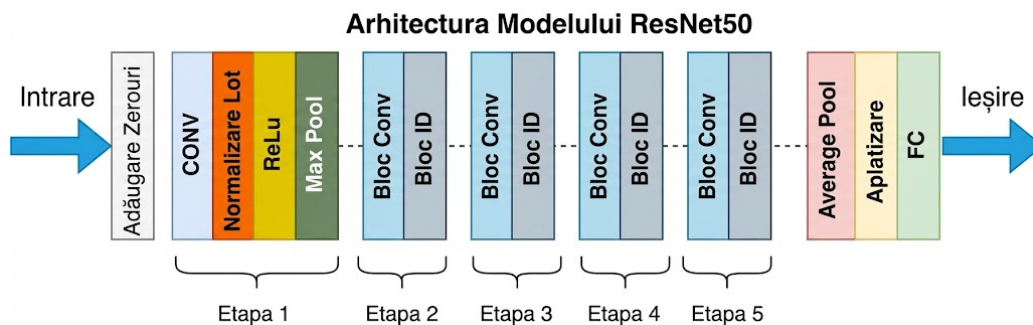


Figura 2.5: Schema unei arhitecturi ResNet50

- **Inception** – utilizează module care aplică simultan filtre convoluționale de dimensiuni diferite (ex. 1×1 , 3×3 , 5×5) pe aceeași intrare, capturând astfel informații la scări spațiale multiple [21]. Acest mecanism este crucial în retinopatia diabetică, unde patologia variază de la leziuni minuscule (microanevrisme punctiforme) la zone masive (exudate extinse sau hemoragii).

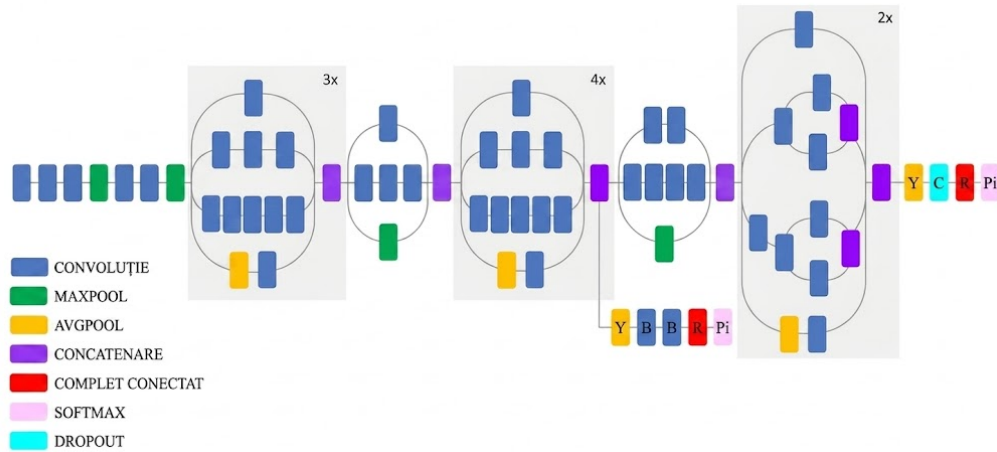


Figura 2.6: Schema unei arhitecturi Inception-v3

- **EfficientNet-B3** – face parte din familia EfficientNet, arhitecturi care folosesc o strategie de scalare compusă (*compound scaling*) pentru a crește simultan adâncimea, lățimea și rezoluția rețelei într-un mod echilibrat. Varianta B3 oferă un compromis între acuratețe și cost computațional, fiind potrivită pentru imagini medicale unde detaliile fine trebuie păstrate fără a crește excesiv complexitatea modelului. În contextul retinopatiei diabetice, această arhitectură poate ajuta la identificarea leziunilor subtile, precum microanevrismele sau exudatele mici, menținând în același timp capacitatea de a surprinde contextul global al imaginii fundus [22].

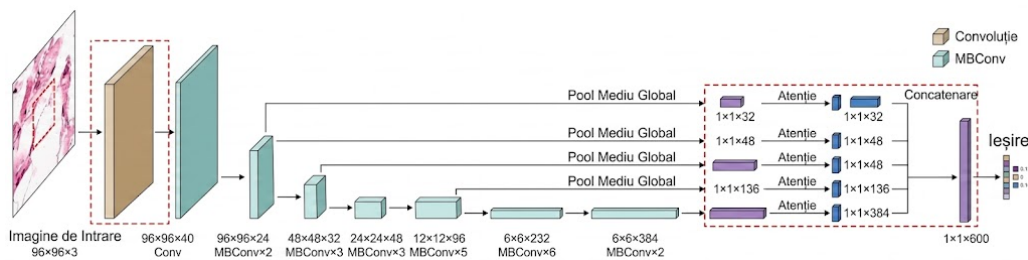


Figura 2.7: Schema unei arhitecturi EfficientNet-B3

- **Inception-ResNet** – reprezintă o arhitectură hibridă care combină eficiența de procesare multi-scală a modulelor Inception cu propagarea robustă a gradientului oferită de conexiunile reziduale. Această fuziune crește viteza de convergență și oferă o capacitate superioară de generalizare, aspect esențial pentru imaginile cu patologii retiniene complexe [2], [6].

Arhitectura Rețelei Inception Resnet V2

Vizualizare Comprimată

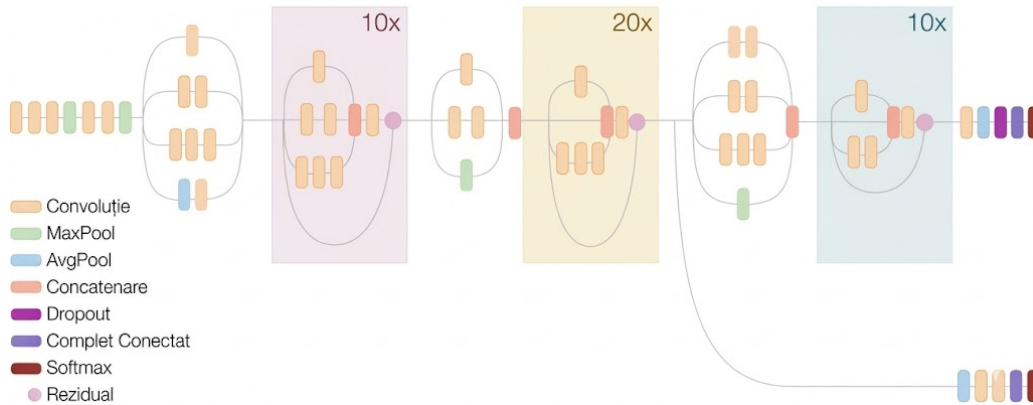


Figura 2.8: Schema unei arhitecturi Inception-ResNet-v2

În retinopatia diabetică, CNN-urile pot fi folosite în mai multe moduri:

- **clasificare globală**, în care modelul primește fotografia fundus și produce gradul de severitate;
- **detectie de leziuni**, în care modelul indică prezența unor semne patologice;
- **segmentare semantică**, în care modelul identifică regiunile afectate la nivel de pixel;
- **modele hibride sau multi-task**, care învață simultan clasificarea, localizarea și interpretarea leziunilor.

Segmentarea poate oferi informație mai detaliată și poate susține interpretabilitatea, dar necesită etichete "pixel-level", mai greu de obținut. Metodele hibride combină ramuri globale și locale pentru a integra contextul cu detaliile patologice [3], [5], [6].

Un aspect important al CNN-urilor este interpretabilitatea. Deși performanța lor poate fi ridicată, modelele profunde sunt adesea percepute ca sisteme de tip "cutie neagră". În medicină, această problemă este critică, deoarece deciziile trebuie să fie verificabile și explicabile. Tehnici precum hărțile de activare a claselor pot evidenția regiunile care au influențat predicția, oferind medicului o indicație vizuală asupra motivului pentru care modelul a ales o anumită clasă. Astfel de metode nu garantează corectitudinea deciziei, dar pot crește transparența și pot ajuta la detectarea cazurilor în care modelul se bazează pe artefacte sau regiuni irelevante [7].

2.8 Transfer learning și modele pre-antrenate

Transfer learning-ul este o tehnică prin care cunoștințele învățate într-o sarcină sunt reutilizate pentru o altă sarcină. În contextul imaginilor medicale, această tehnică este foarte importantă deoarece seturile de date adnotate sunt mai greu de obținut decât în domeniile generale ale vederii computerizate. Un model pre-antrenat pe milioane de imagini poate învăța

filtre generale pentru muchii, texturi, forme și compoziții vizuale. Chiar dacă imaginile naturale diferă de imaginile retiniene, primele straturi ale rețelei pot rămâne utile, iar straturile finale pot fi adaptate la clasificarea medicală [2], [18].

Există mai multe moduri de utilizare a transfer learning-ului:

- **Extractor de trăsături fix.** Straturile convoluționale sunt păstrate fixe, iar doar clasificatorul final este antrenat pe datele medicale.
- **Fine-tuning parțial.** Se ajustează doar ultimele blocuri ale rețelei, pentru a adapta reprezentările la imaginile retiniene.
- **Fine-tuning complet.** Toți parametrii modelului sunt actualizați, variantă utilă când există suficiente date și regularizare adecvată.

Fine-tuning-ul poate oferi performanțe mai bune, dar necesită atenție la rata de învățare, dimensiunea setului de date, regularizare și optimizatorul folosit, Adam fiind una dintre alegerile frecvente în antrenarea rețelelor profunde [2], [23].

În aplicații de retinopatie diabetică, arhitecturile pre-antrenate pot fi adaptate prin înlocuirea ultimului strat cu un clasificator pentru cele cinci clase de severitate și prin continuarea antrenării pe imaginile fundus disponibile [11], [2]. Detaliile arhitecturilor utilizate experimental sunt prezentate ulterior în metodologia propusă.

Transfer learning-ul nu elimină însă toate problemele. Dacă datele de antrenare sunt dezechilibrate sau conțin artefacte sistematice, modelul poate învăța corelații nedorite. De exemplu, dacă imaginile severe provin predominant de la o anumită cameră sau au o anumită rezoluție, modelul poate asocia caracteristicile tehnice cu boala. Această problemă este cunoscută ca bias de domeniu și afectează capacitatea de generalizare. Din acest motiv, este importantă evaluarea pe seturi de validare și testare reprezentative, precum și analiza erorilor prin matrice de confuzie și curbe ROC [8], [15].

În această lucrare, fundamentele prezentate în capitolul curent susțin partea experimentală ulterioară. Retinopatia diabetică este o problemă medicală cu impact ridicat, imaginile de fund de ochi oferă suportul vizual pentru screening automat, iar rețelele convoluționale permit învățarea directă a trăsăturilor relevante. Transfer learning-ul facilitează adaptarea arhitecturilor moderne la date medicale, iar metricile de evaluare permit compararea configurațiilor de antrenare. Împreună, aceste elemente definesc cadrul teoretic necesar pentru metodologia propusă și pentru interpretarea rezultatelor experimentale.

Capitolul 3

Stadiul actual al cercetării

3.1 Analiza seturilor de date disponibile

Performanța și capacitatea de generalizare a modelelor de învățare profundă sunt intrinsec dependente de calitatea, volumul și diversitatea datelor utilizate în procesul de antrenare. În contextul oftalmologiei computaționale, disponibilitatea bazelor de date publice a permis tranziția de la metodele clasice de procesare a imaginii la arhitecturi neuronale complexe, oferind un teren comun pentru evaluarea obiectivă a algoritmilor.

- **Seturi de referință pentru clasificarea globală (“Grading”)**

- **EyePACS** (Kaggle Diabetic Retinopathy Detection) [1], [7] rămâne cea mai vastă resursă publică disponibilă, conținând aproximativ 88.702 de imagini. Importanța sa constă în caracterul eterogen al imaginilor (variații de expunere, focalizare, artefacte), fiind critic pentru antrenarea unor rețele backbone robuste capabile să gestioneze fenomenul de “domain shift” [8] și să generalizeze pe imagini de calitate slabă
- **APTOS 2019 Blindness Detection** [7] reprezintă standardul modern de calitate. Colectat de Aravind Eye Hospital, acest set de 3.662 de imagini este utilizat frecvent în literatura recentă pentru “fine-tuning”, oferind o vizibilitate superioară a detaliilor retiniene comparativ cu seturile mai vechi
- **MESSIDOR-2** [7] este considerat în prezent „Benchmark-ul de Aur” pentru testarea algoritmilor. Extinzând setul original Messidor la 1.748 de imagini, acesta se distinge prin consistența diagnostică și este utilizat ca domeniu țintă (Target Domain) în studiile de adaptare [8] și pentru validarea sistemelor explicabile (XAI) [7], oferind o metrică fidelă a performanței reale pe date curate.
- **DeepDRiD** [15] aduce o inovație necesară prin structurarea celor 2.000 de imagini în perechi binoculare și includerea notelor de calitate, simulând fluxul de triaj clinic.
- **UoA-DR** [7] este o resursă istorică (200 imagini), utilizată complementar în studiile comparative pentru a asigura diversitatea demografică a validării.

- **Seturi specializate pentru segmentarea leziunilor și analiza detaliată**

- **IDRiD** [7] este setul critic pentru sarcinile de segmentare semantică fină [5]. Pentru 81 dintre imagini, experții au trasat manual măști binare la nivel de pixel, făcându-l indispensabil pentru antrenarea arhitecturilor complexe de tip Transformer și pentru validarea localizării leziunilor în sistemele end-to-end [7].
- **DDR** [14] a revoluționat segmentarea pe scară largă, oferind 13.673 de imagini. Este esențial pentru antrenarea rețelelor adânci de segmentare (precum RTNet) fără a risca “overfitting-ul”, oferind adnotări “pixel-wise” pentru leziuni multiple [5].

- **E-Ophtha** [7] este o bază de date vitală pentru reducerea alarmelor false în metodele de detecție a exudatelor [10]. Valoarea sa unică constă în includerea imaginilor cu patologii non-diabetice care pot fi confundate cu DR
- **DiaRetDB1** [7] este standardul istoric pentru calibrarea sensibilității la leziuni roșii (microanevrisme) în lucrările de detecție automată [17]. Complementar, HEI-MED [16] oferă 169 de imagini focalizate exclusiv pe segmentarea zonelor de Edem Macular Diabetic.

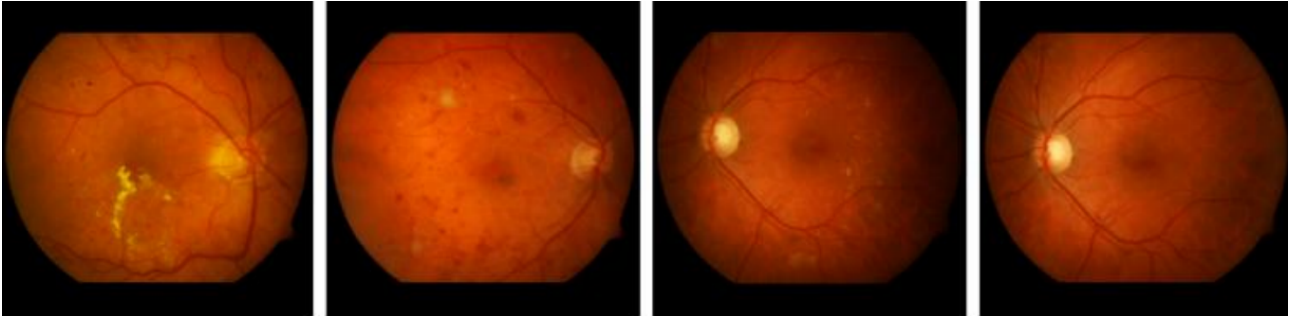


Figura 3.1: Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]

- **Seturi standard pentru preprocesarea vasculară**

- **DRIVE** [9] și **STARE** [7] sunt pilonii etapei de preprocesare. Deși conțin un număr redus de imagini, ele sunt utilizate obligatoriu pentru validarea algoritmilor de segmentare a vaselor de sânge, cum ar fi arhitecturile U-Net modificate [4] sau metodele clasice de extragere a trăsăturilor [17].

Tabelul de mai jos sintetizează caracteristicile tehnice esențiale ale seturilor de date analizate, evidențiind diferențele de volum, rezoluție și tipul de adnotare disponibil, elemente care dictează utilizarea lor în diversele etape ale proiectului.

Tabela 3.1: Tabel comparativ între seturile de date disponibile

Set de date	Număr imagini (total)	Rezoluție tipică (px)	Tip adnotare	Utilizare principală
EyePACS	88.702	Variabilă, până la 4000+	Globală, clase 0–4	Antrenare backbone și generalizare
DDR	13.673	Variabilă	Pixel-wise, măști și bounding box	Segmentare la scară largă
APTOS 2019	3.662	Înaltă, standardizată	Globală, clase 0–4	Fine-tuning și validare
DeepDRiD	2.000	512×512, reconstruit	Globală, calitate și binocular	Analiza calității imaginii
MESSIDOR-2	1.748	1440×960 – 2240×1488	Globală și edem	Benchmark și testare
IDRiD	516	4288×2848	Pixel-wise pentru 81 imagini și globală	Segmentare semantică fină
E-Ophtha	381 EX / 381 MA	Variabilă, medie/mare	Contur leziuni EX/MA	Reducerea alarmelor false pozitive
UoA-DR	~200	Variabilă	Globală, clase 0–4	Validare încrucișată
HEI-MED	169	2196×1958	Măști edem	Detecție edem macular
DiaRetDB1	89	1500×1152	Coordonate punctuale	Calibrare microanevrisme
DRIVE	40	565×584	Măști vase binare	Preprocesare și segmentare vase de sânge
STARE	20	700×605	Măști vase binare	Preprocesare și segmentare vase de sânge

3.2 Metoda bazată pe arhitectura Inception-V3 [1]

Studiul realizat de Gulshan et al. [1] este considerat un reper major în literatura de specialitate, marcând una dintre primele validări clinice ale utilizării rețelelor neuronale convoluționale pentru screening-ul automat al retinopatiei diabetice. Lucrarea demonstrează că un sistem de deep learning poate atinge și chiar depăși performanța medie a medicilor oftalmologi în detectarea retinopatiei diabetice, contribuind decisiv la acceptarea inteligenței artificiale în aplicații medicale reale [1].

Arhitectura utilizată este Inception-v3, o rețea convoluțională profundă caracterizată prin utilizarea modulelor Inception, care combină în paralel convoluții de dimensiuni diferite (1×1, 3×3, 5×5) și operații de pooling. Această strategie permite captarea informațiilor la multiple scări spațiale și reduce costul computațional prin utilizarea convoluțiilor 1×1 pentru diminuarea dimensionalității. Datorită acestei structuri, modelul este capabil să învețe reprezentări vizuale complexe, esențiale pentru identificarea leziunilor subtile specifice retinopatiei diabetice.

Modelul a fost antrenat pe un set de date extins, alcătuit din peste 128.000 de imagini fundus color, etichetate independent de mai mulți medici oftalmologi certificați în 2015. Utilizarea consensului între experți a contribuit la reducerea subiectivității etichetelor și la creșterea fiabilității procesului de antrenare. Sarcina principală a rețelei a fost clasificarea imaginilor în funcție de prezența sau absența retinopatiei diabetice referabile, conform standardelor clinice acceptate internațional [1].

Preprocesarea imaginilor a inclus decuparea automată a regiunii retiniene, redimensionarea la o rezoluție compatibilă cu Inception-v3 și normalizarea intensității pixelilor. În plus, au

fost aplicate tehnici extensive de augmentare a datelor, precum rotații, translații și ajustări de luminozitate și contrast, pentru a îmbunătăți generalizarea și a reduce supraînvățarea [1] [2].

Evaluarea performanței a fost realizată pe seturile de date EyePACS-1 și Messidor-2 [7], unde modelul a obținut o arie sub curba ROC (AUC) de aproximativ 0.99 pentru detectarea retinopatiei diabetice referabile, cu valori ridicate de sensibilitate, comparabile cu performanța medicilor oftalmologi experimentați [1] [11].

3.3 “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]

Lucrarea propusă de Jabbar et al. [2] abordează problema diagnosticului automat al retinopatiei diabetice dintr-o perspectivă practică, concentrându-se pe utilizarea tehnicilor de “transfer learning” pentru a compensa limitările impuse de dimensiunea redusă a seturilor de date medicale etichetate. Studiul evidențiază faptul că modelele CNN pre-antrenate pe seturi de date generale de mari dimensiuni, precum ImageNet, pot fi adaptate eficient pentru analiza imaginilor fundus, obținând performanțe competitive cu un cost de antrenare semnificativ redus [2].

Autorii investighează mai multe arhitecturi consacrate de rețele neuronale convoluționale, printre care VGG16, VGG19, ResNet50, Inception-v3 și DenseNet, utilizate ca extractoare de caracteristici. Strategia de transfer learning constă în inițializarea rețelelor cu ponderi pre-antrenate, urmată de ajustarea straturilor superioare pentru sarcina specifică de clasificare a retinopatiei diabetice. În unele experimente, straturile convoluționale inferioare sunt înghețate pentru a preveni supraînvățarea, în timp ce în altele este aplicat “fine-tuning” progresiv pentru a îmbunătăți adaptarea la domeniul imagisticii medicale. Una din abordările propuse poate fi observată în figura următoare.

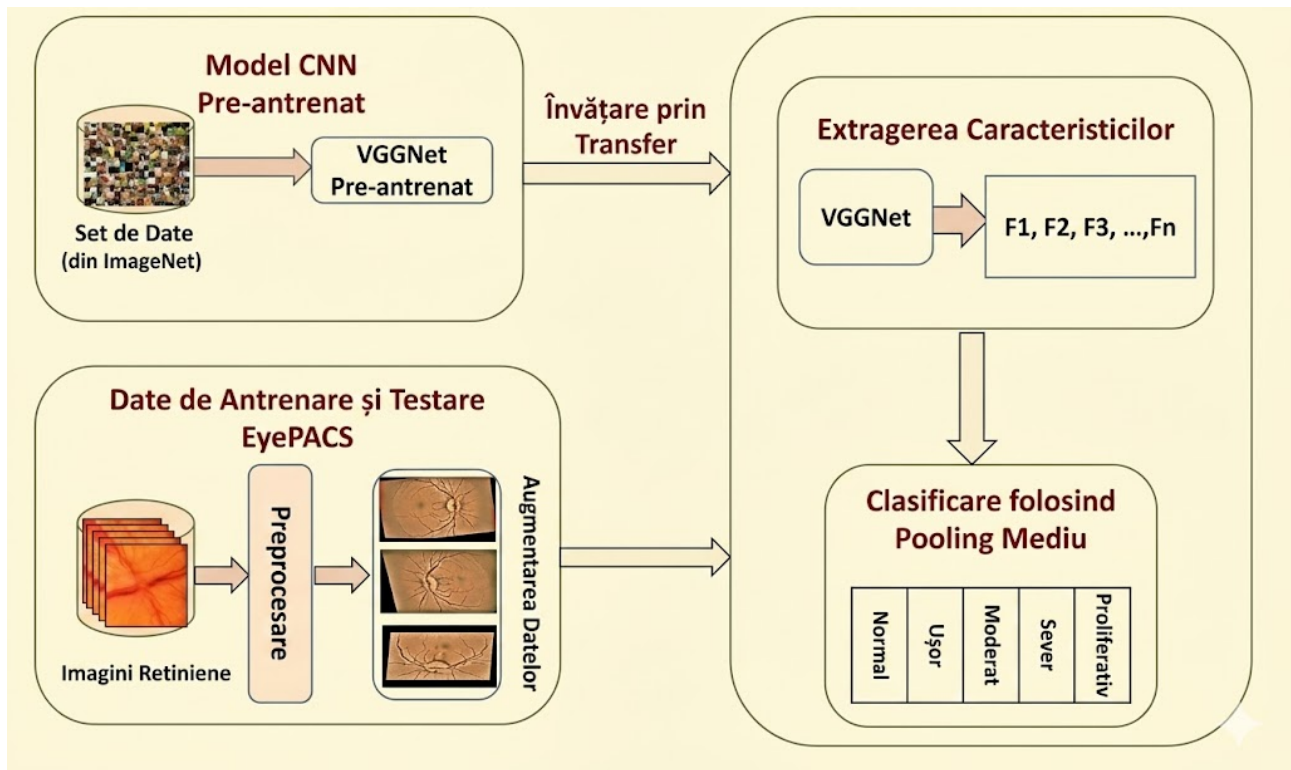


Figura 3.2: Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2]

Seturile de date utilizate includ imagini fundus publice din Kaggle EyePACS [7], preprocesate prin redimensionare, normalizare și augmentare extensivă a datelor. Tehnicile de augmentare, precum rotațiile, flip-urile orizontale și ajustările de luminosități, joacă un rol central în creșterea diversității artificiale a datelor și în îmbunătățirea capacității de generalizare a modelelor. Această etapă este esențială în contextul “transfer learning-ului”, unde diferențele de distribuție dintre datele naturale și imaginile medicale pot afecta performanța rețelelor [2].

Evaluarea performanței este realizată folosind metrici standard, precum acuratețea, sensibilitatea, specificitatea și scorul F1, permițând o comparație riguroasă între diferitele arhitecturi analizate. Rezultatele experimentale indică faptul că modelele bazate pe DenseNet și ResNet obțin performanțe superioare față de arhitecturile mai vechi, confirmând avantajele reutilizării caracteristicilor vizuale generale în sarcini medicale specializate. Studiul subliniază că transfer learning-ul reprezintă o soluție viabilă și eficientă pentru implementarea rapidă a sistemelor automate de screening în contexte cu resurse limitate [2].

3.4 “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]

Lucrarea propusă de Wang et al. [3] introduce o abordare de tip “deep multi-task learning” pentru gradarea automată a retinopatiei diabetice, având ca obiectiv principal exploatarea relației clinice dintre severitatea bolii și prezența leziunilor retiniene specifice. Spre deosebire de metodele clasice de clasificare “single-task”, autorii demonstrează că învățarea simultană a mai multor sarcini corelate clinic conduce la reprezentări vizuale mai discriminative și la performanțe superioare în sarcina principală de gradare [3].

Arhitectura propusă este alcătuită dintr-un backbone convoluțional comun, bazat pe o rețea CNN profundă (ResNet18), utilizat pentru extracția caracteristicilor globale din imaginile fundus. Peste acest extractor comun sunt construite mai multe ramuri specializate (task-specific branches), fiecare corespunzătoare unei sarcini distincte. Sarcina principală constă în clasificarea severității retinopatiei diabetice în mai multe stadii, în timp ce sarcinile auxiliare vizează detectarea prezenței diferitelor tipuri de leziuni, precum microanevrismele, exudatele dure, exudatele moi și hemoragiile, considerate “markeri” clinici relevanți ai progresiei bolii [3].

Un element central al arhitecturii îl reprezintă mecanismul de partajare controlată a caracteristicilor între sarcini. În locul unei separări rigide între ramuri, modelul permite propagarea informației relevante între sarcina de gradare și cele de detecție a leziunilor, încurajând învățarea unor reprezentări comune sensibile la semnalele patologice fine. Funcția de pierdere globală este definită ca o combinație ponderată a pierderilor individuale asociate fiecărei sarcini:

$$L_{isr} = \lambda_{img} \cdot L_{img_isr} + \lambda_{tv} \cdot L_{tv_isr} + \lambda_{sa} \cdot L_{seg-aware} + \lambda_{ca} \cdot L_{cls-aware}$$

,unde λ_{img} , λ_{tv} , λ_{sa} și λ_{ca} sunt hiperparametri care controlează contribuția pierderilor “intra-task”, “segmentation-aware” și “classification-aware”. Această formulare permite echilibrarea influenței fiecărei sarcini și stabilizează procesul de optimizare [3].

Modelul este antrenat și evaluat pe seturi de date publice de imagini fundus etichetate (DDR [14] și EyePACS [7]) atât cu stadiul retinopatiei diabetice, cât și cu adnotări la nivel de imagine privind prezența leziunilor. Imaginile sunt supuse unor etape standard de preprocesare, incluzând decuparea regiunii retiniene, redimensionarea și augmentarea datelor, pentru a reduce variațiile de iluminare și pentru a îmbunătăți capacitatea de generalizare. Prin includerea sarcinilor auxiliare, modelul beneficiază de un efect de regularizare implicită, reducând riscul

de supraînvăţare, aspect observat şi în lucrările anterioare bazate pe CNN-uri “single-task” [1], [11].

Rezultatele experimentale raportate în articol indică îmbunătăţiri consistente ale performanţei faţă de modelele “single-task”, atât în termeni de acurateţe, cât şi de scor F1 pentru sarcina de gradare a retinopatiei diabetice. În plus, autorii evidenţiază o creştere a stabilităţii predicţiilor şi o sensibilitate mai bună la stadiile intermediare ale bolii, considerate dificil de clasificat. Aceste rezultate confirmă faptul că integrarea informaţiilor despre leziuni în procesul de antrenare contribuie direct la o evaluare mai precisă a severităţii retinopatiei diabetice şi validează eficienţa paradigmei “multi-task learning” în context clinic [3].

3.5 Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]

Segmentarea semantică a structurilor retiniene, în special a reţelei vasculare şi a leziunilor asociate retinopatiei diabetice, reprezintă o etapă critică în diagnosticarea automată, deoarece modificările morfologice ale vaselor de sânge sunt adesea primii indicatori ai bolii. Deşi arhitectura UNet, introdusă iniţial de Ronneberger et al. [24] pentru segmentarea biomedicală, a devenit standardul de aur în domeniu datorită structurii sale simetrice de tip “encoder-decoder”, aceasta prezintă limitări în detectarea vaselor capilare foarte fine, pierzând informaţii spaţiale esenţiale în procesul de subeşantionare.

În studiul “Modified U-Net architecture for semantic segmentation of diabetic retinopathy images”, Sambyal et al [4]. propun o optimizare arhitecturală semnificativă pentru a contracara aceste deficienţe. Modelul propus, denumit Modified U-Net, păstrează structura în formă de „U” a reţelei originale, dar inovează fundamental la nivelul blocurilor de procesare.

Autorii au înlocuit blocurile convoluţionale standard cu blocuri reziduale (residual blocks), inspirate din arhitecturile de tip ResNet. Această modificare este crucială din punct de vedere tehnic: în reţelele adânci, apare frecvent problema dispariţiei gradientului (vanishing gradient), ceea ce face antrenarea dificilă şi duce la pierderea detaliilor fine. Prin introducerea conexiunilor reziduale (“skip connections” la nivel de bloc), informaţia originală este propagată mai eficient prin straturi, permiţând reţelei să înveţe caracteristici complexe fără a degrada performanţa pe detaliile de fineţe, precum microanevrismele sau vasele subţiri. Aceste detalii se pot observa şi în figura următoare.

Importanţa acestei abordări constă în capacitatea de generalizare comparativ cu metodele clasice de procesare a imaginii, precum cele bazate pe “clustering fuzzy” propuse anterior de Sopharak et al. [10]. În timp ce metodele de tip “clustering” sunt sensibile la zgomot şi la variaţiile de iluminare (frecvente în imaginile de fund de ochi), arhitectura Modified U-Net demonstrează o robusteţe sporită prin învăţarea contextuală a trăsăturilor

Pentru validarea performanţei, metoda a fost testată pe seturile de date publice standardizate DRIVE [9] şi STARE [7], recunoscute pentru dificultatea lor în segmentarea vaselor.

Rezultatele experimentale raportate de Sambyal et al. [4] au indicat o îmbunătăţire clară faţă de U-Net-ul standard. Pe setul de date DRIVE, autorii au obţinut o acurateţe maximă de 96.94 sub curba ROC (AUC) de 0.98, valori care plasează metoda în topul soluţiilor de segmentare. Mai mult, analiza vizuală a rezultatelor arată o continuitate mult mai bună a vaselor de sânge segmentate şi o reducere a alarmelor false în zonele cu patologii, demonstrând că integrarea învăţării reziduale în arhitectura de segmentare este o direcţie esenţială pentru analiza detaliată a retinopatiei diabetice.

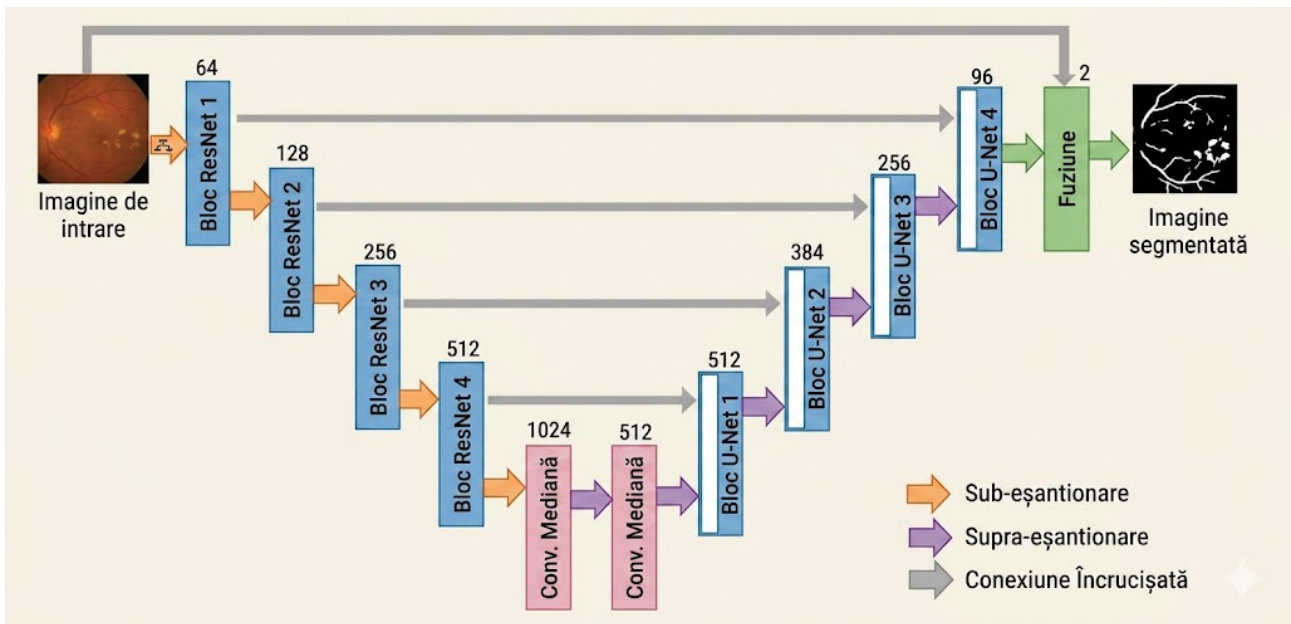


Figura 3.3: Arhitectura U-net modificată [4]

3.6 Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]

Dacă modelele bazate pe U-Net, precum cel discutat anterior [4], excelează în capturarea detaliilor locale, ele suferă totuși de o limitare inerentă a rețelelor convoluționale: incapacitatea de a modela dependențe la distanță mare (long-range dependencies). În patologia retiniană, leziunile nu apar izolat; există adesea o corelație spațială între poziția vaselor de sânge și apariția microanevrismelor sau a exudatelor. Pentru a adresa această lipsă de „context global”, Huang et al. [5] au propus în 2022 o schimbare de paradigmă prin introducerea RTNet (Relation Transformer Network), o arhitectură care marchează trecerea către mecanisme de atenție.

Inovația centrală a RTNet constă în înlocuirea parțială a operațiilor convoluționale standard cu blocuri de tip “Transformer”, denumite Relation Transformer Blocks (RTB). Spre deosebire de CNN-uri care „văd” doar o fereastră mică de pixeli la un moment dat, RTB-ul utilizează două mecanisme de atenție complementare:

1. “Self-Attention” (Atenție proprie): Permite modelului să înțeleagă relațiile dintre leziunile dispersate pe toată suprafața retinei (de exemplu, grupuri de exudate dure care semnalează o zonă de edem).
2. “Cross-Attention” (Atenție încrucișată): Autorii au observat că leziunile vasculare sunt strâns legate de structura vaselor de sânge. Astfel, mecanismul de “cross-attention” folosește harta vaselor de sânge ca un „ghid” pentru a localiza mai precis leziunile fine, transferând informație structurală dinspre ramura vasculară către ramura de detectare a leziunilor.

Pentru validare, metoda a fost testată pe seturile de date IDRiD [7] și DDR [14], care conțin adnotări la nivel de pixel pentru multiple tipuri de leziuni. Rezultatele cantitative au demonstrat că RTNet depășește metodele existente anterioare (inclusiv diverse variante de U-Net și DeepLab), obținând cele mai bune scoruri de segmentare pentru Exudate (EX), Microanevrisme (MA) și Exudate Moi (SE).

Deși inovația teoretică a arhitecturii RTNet (Relation Transformer Network) propusă de Huang et al. [5] este evidentă prin integrarea mecanismelor de atenție, validarea practică a acesteia necesită o demonstrație riguroasă a faptului că fiecare modul propus contribuie activ la îmbunătățirea segmentării. În acest sens, autorii au realizat un studiu extensiv de ablație (ablation study), analizând impactul incremental al componentelor arhitecturale asupra capacității de învățare și generalizare a rețelei.

Această analiză comparativă este sintetizată în graficele de performanță care monitorizează evoluția funcției de pierdere (Loss) și curbele Precision-Recall (PR) pentru cele patru tipuri principale de leziuni asociate retinopatiei diabetice: microanevrisme (MA), exudate (EX), hemoragii (HE) și exudate moi (SE).

Comparatia pornește de la un model “Baseline” (o rețea convoluțională standard de tip U-Net fără mecanisme de atenție) și adaugă progresiv componentele specifice RTNet, regasite în figura de mai jos:

1. GTB (Global Transformer Block): Adăugarea capacității de procesare globală.
2. SAH (Self-Attention Head): Introducerea atenției proprii pentru corelarea leziunilor similare.
3. RAH (Relation Attention Head / Cross-Attention): Modulul critic care leagă leziunile de structura vaselor de sânge.
4. RTB (Relation Transformer Block): Arhitectura completă propusă.

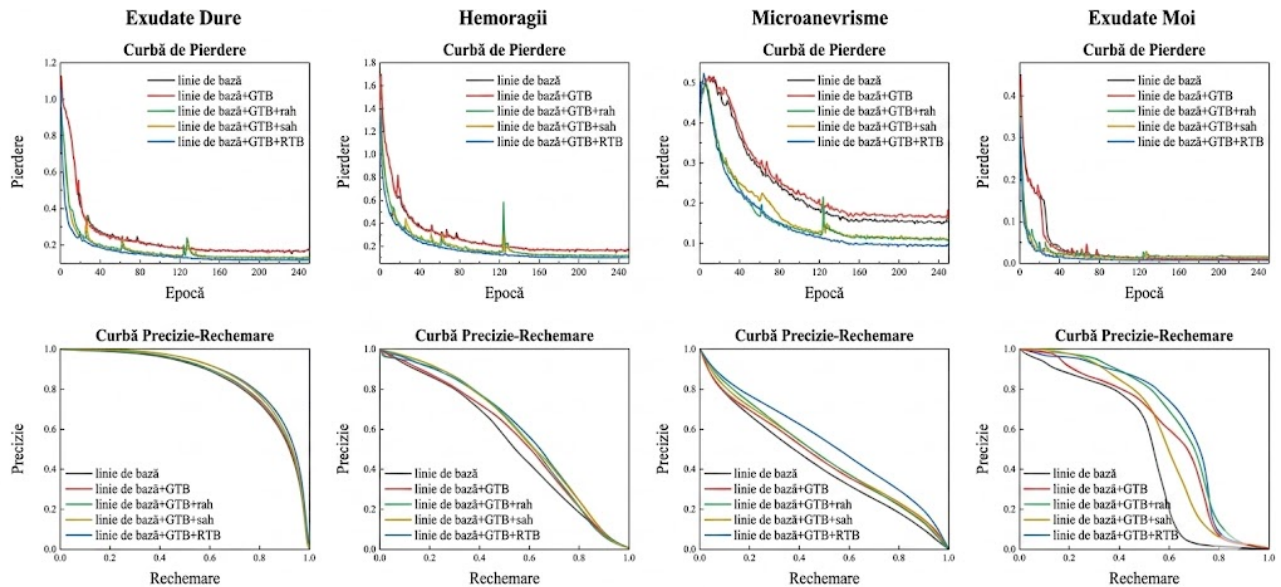


Figura 3.4: Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]

Lucrarea lui Huang et al. [5] validează ipoteza că integrarea informației globale prin Transformere este pasul necesar pentru a depăși plafonul de performanță atins de rețelele convoluționale pure în segmentarea medicală complexă.

3.7 Arhitectura “Dual Branch” [6]

Deși progresele în segmentarea semantică, ilustrate prin metodele bazate pe U-Net [4] sau pe Transformers [5], au permis o delimitare precisă a leziunilor, translatarea acestor detalii într-un diagnostic global (grading) rămâne o provocare majoră din cauza naturii subtile a diferențelor dintre stadiile retinopatiei diabetice. Rețelele convoluționale standard utilizate în studiile fundamentale, precum cel al lui Gulshan et al. [1], tind să piardă detaliile fine, cum ar fi microanevrismele incipiente, în procesul de redimensionare a imaginii necesar pentru analiza globală. Pentru a depăși acest compromis inerent între contextul vizual și rezoluția detaliilor, Shakibania et al. [6] au propus o arhitectură inovatoare de tip “Dual Branch Deep Learning Network”, care restructurează procesul de învățare prin decuplarea extragerii trăsăturilor în două fluxuri paralele distincte.

Conceptul central al acestei abordări constă în imitarea procesului cognitiv uman de diagnosticare, procesând imaginea simultan pe o „ramură globală” și o „ramură locală”. Ramura globală analizează imaginea completă a fundului de ochi pentru a captura structura generală a vaselor și geometria retinei, o strategie similară cu metodele clasice de “Transfer Learning” [2]. În paralel, ramura locală operează pe decupaje de înaltă rezoluție, fiind specializată în identificarea leziunilor microscopice care ar fi invizibile pentru rețeaua globală. Elementul de noutate adus de această lucrare față de abordările “multi-task” anterioare [3], este mecanismul de fuziune a celor două fluxuri, care utilizează tehnici de atenție pentru a pondera importanța trăsăturilor locale în decizia finală, asigurând că un detaliu critic nu este ignorat în favoarea unei imagini de ansamblu aparent sănătoase.

Eficiența acestei arhitecturi hibride este validată prin rezultatele experimentale obținute pe seturile de date standard, unde modelul “Dual Branch” a demonstrat o superioritate clară în clasificarea stadiilor intermediare ale bolii (ușor și moderat). Analiza performanței indică faptul că integrarea informației locale crește semnificativ sensibilitatea modelului și reduce confuzia între clasele adiacente, oferind o robustețe superioară la variațiile de calitate a imaginii comparativ cu arhitecturile “singlestream”. Astfel, lucrarea confirmă faptul că performanța actuală în 2024 nu derivă doar din adâncimea rețelei, ci din capacitatea arhitecturală de a reuni viziunea macroscopică cu cea microscopică într-un diagnostic coerent.

3.8 Abordarea “Explainable AI” [7]

În timp ce arhitecturile precum “Dual Branch Network” [6] sau Transformer-ele relaționale [5] au reușit să împingă limitele acurateței și ale segmentării fine, integrarea acestor sisteme în fluxul clinic real se lovește de o barieră fundamentală: lipsa de încredere a medicilor în algoritmi de tip „black-box”. Validarea clinică inițiată de Gulshan et al. [1] a demonstrat potențialul diagnostic, însă pentru ca un sistem de inteligență artificială să fie acceptat ca asistent de încredere, acesta trebuie să își poată justifica deciziile. În acest context, lucrarea publicată de Chetoui și Akhloufi [7], reprezintă un pilon esențial al stadiului actual al cunoașterii, adresând simultan două probleme critice: interpretabilitatea (Explainable AI - XAI) și robustețea pe seturi de date eterogene.

Metodologia propusă de autori se bazează pe o arhitectură de tip CNN modificată (utilizând backbone-uri precum DenseNet121 sau ResNet50), în care straturile clasice complet conectate sunt înlocuite cu Global Average Pooling (GAP). Această modificare tehnică permite generarea Hărților de Activare a Claselor (CAM), oferind medicului o confirmare vizuală a regiunilor patologice care au determinat diagnosticul.

Relevanța clinică a acestei abordări extensive este evidențiată în rezultatele vizuale, unde

hărțile de saliență generate de algoritm se suprapun cu precizie peste semnele patologice reale, indiferent de setul de date din care provine imaginea. Zonele marcate cu roșu intens pe hărțile CAM corespund anatomic cu microanevrismele și exudatele, confirmând că rețeaua neuronală a învățat trăsăturile intrinseci ale bolii și nu caracteristicile specifice ale camerei foto (bias-ul echipamentului). Prin urmare, modelul propus [7] validează faptul că un sistem trebuie să fie nu doar precis, ci și independent de sursa datelor și transparent în decizii.

3.9 Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]

Ultima frontieră în dezvoltarea sistemelor de diagnostic asistat de calculator (CAD) pentru retinopatia diabetică nu mai este legată strict de acuratețea obținută pe un singur set de date de antrenament, ci de capacitatea algoritmului de a funcționa corect într-un mediu clinic eterogen. În realitate, imaginile de fund de ochi variază drastic în funcție de modelul camerei, setările de iluminare, etnia pacientului sau protocolul de achiziție, fenomen cunoscut sub numele de “domain shift”. Modelele antrenate pe un „domeniu sursă” (de exemplu, imagini de înaltă calitate de la EyePACS [7]) tind să aibă performanțe scăzute atunci când sunt aplicate pe un „domeniu țintă” (imagini clinice noi, cu caracteristici vizuale diferite), ceea ce limitează sever aplicabilitatea lor universală.

Pentru a soluționa această problemă critică de implementare, Zhang et al. [8] au propus în 2022 o metodă inovatoare denumită “Multi-Model Domain Adaptation” (MMDA). Spre deosebire de abordările anterioare care încearcă să alinieze domeniile folosind un singur model, autorii introduc o arhitectură complexă bazată pe o rețea siameză, utilizând două modele distincte pentru a extrage trăsături complementare și pentru a crește robustețea deciziei.

Inovația tehnică centrală a lucrării constă în integrarea simultană a trei mecanisme de optimizare care lucrează concertat pentru a forța modelul să ignore diferențele stilistice dintre seturile de date și să se concentreze exclusiv pe patologie. Astfel, arhitectura combină o pierdere de clasificare pentru a asigura acuratețea diagnostică pe domeniul sursă etichetat, cu o pierdere de discrepanță care maximizează diferența dintre predicțiile celor două clasificatoare pe domeniul țintă pentru a detecta exemplele aflate la granița de decizie. În plus, se utilizează o pierdere de adaptare adversarială care, prin intermediul unui discriminator, minimizează distanța dintre distribuția trăsăturilor din sursă și cea din țintă, făcând ca imaginile provenite din spitale diferite să fie reprezentate similar de către rețeaua neuronală.

Validarea experimentală a metodei a fost realizată folosind setul de date EyePACS [7] ca domeniu sursă și Messidor-2 [7] ca domeniu țintă, simulând un scenariu realist de transfer tehnologic de la imagini de screening la imagini clinice de referință. Rezultatele au demonstrat că abordarea MMDA depășește semnificativ metodele standard de antrenare și alte tehnici de adaptare cunoscute, obținând o îmbunătățire substanțială a metricilor de performanță pe domeniul țintă fără a necesita etichete pentru acesta. Această lucrare încheie logic analiza stadiului actual al cunoașterii, subliniind faptul că un sistem complet nu trebuie doar să segmenteze leziuni fine sau să fie explicabil, ci trebuie să fie capabil să iasă din mediul controlat al laboratorului și să ofere o performanță robustă și generalizabilă în diversele condiții întâlnite în practica medicală globală.

3.10 Concluzii

Analiza literaturii de specialitate relevă faptul că detectarea automată a retinopatiei diabetice a depășit stadiul de experiment teoretic, devenind o direcție de cercetare matură, cu rezultate clinice valide. Totuși, problema rămâne una de o complexitate ridicată, care depășește simpla clasificare a imaginilor. Dificultatea majoră nu constă doar în volumul masiv de date necesar, ci în subtilitatea semnelor patologice incipiente, precum microanevrismele, care pot ocupa doar câțiva pixeli dintr-o imagine de înaltă rezoluție, fiind ușor de confundat cu artefactele sau zgomotul de fond. Această granularitate fină impune algoritmilor o capacitate de discriminare vizuală care, în anumite scenarii limită, trebuie să depășească acuitatea ochiului uman.

În ciuda acestor obstacole, soluțiile algoritmice existente demonstrează o evoluție remarcabilă. Trecerea de la rețelele convoluționale standard la arhitecturi sofisticate, capabile să integreze informația globală cu cea locală prin mecanisme de tip “dual-branch” [6], a permis o creștere semnificativă a sensibilității în detectarea stadiilor intermediare ale bolii. Mai mult, rezultatele obținute prin tehnici de adaptare la domeniu [8] sugerează că bariera variabilității echipamentelor medicale începe să fie depășită, deschizând calea pentru sisteme robuste, capabile să funcționeze corect indiferent de spitalul în care sunt implementate.

Un alt aspect promițător este schimbarea de paradigmă de la modelele de tip „cutie neagră” la sistemele explicabile (Explainable AI). Așa cum evidențiază studiile recente [7], capacitatea modelelor de a genera hărți de activare precise transformă inteligența artificială dintr-un simplu clasificator statistic într-un partener transparent pentru medic, facilitând validarea rapidă a deciziei. Deși provocările legate de dezechilibrul claselor și de generalizarea pe populații diverse persistă, performanțele atinse de metodele actuale, validate pe seturi de date riguroase precum MESSIDOR-2 [7], confirmă potențialul acestor tehnologii de a eficientiza screening-ul în masă și de a preveni orbirea la scară globală.

Capitolul 4

Setul de date și preprocesarea imaginilor

4.1 Sursele de date utilizate

Experimentele din această lucrare nu folosesc un singur set de imagini, ci trei surse principale, pregătite în același format de tip `ImageFolder`. Această alegere a fost necesară deoarece clasificarea retinopatiei diabetice este sensibilă la diferențele de domeniu: imagini provenite din camere diferite, cu iluminare diferită sau cu grade diferite de preprocesare pot produce performanțe semnificativ diferite. Prin urmare, pe lângă setul *Diabetic_Balanced_Data*, au fost introduse și variantele *balanced_aug* și *APTOS_matched*, pentru a testa mai bine robustețea modelelor.

Prima sursă este `datasets/Diabetic_Balanced_Data`, disponibilă pe Kaggle sub denumirea *Diabetic Retinopathy Balanced* [25]. Aceasta conține imagini color ale fundului de ochi, organizate pentru clasificare multclasă în cinci grade de severitate. Setul este aproape echilibrat numeric și reprezintă baza experimentelor inițiale, deoarece permite compararea arhitecturilor și a funcțiilor de pierdere într-un cadru controlat.

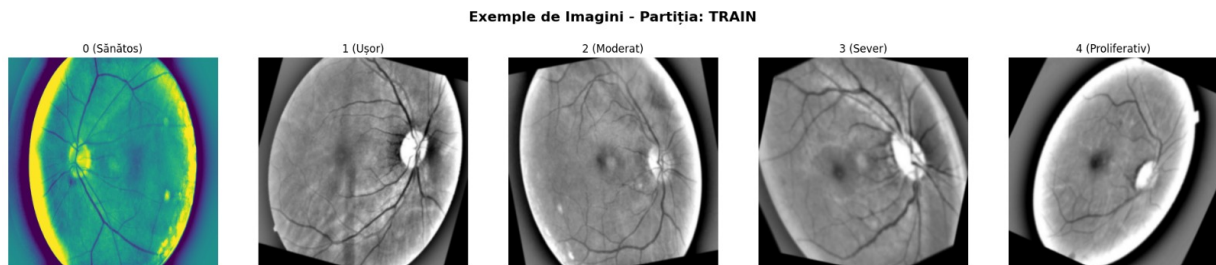


Figura 4.1: Exemple de imagini din setul *Diabetic_Balanced_Data* pentru fiecare clasă de severitate

A doua sursă este `datasets/Diabetic_Balanced_Aug_Ben_Graham_matched`, denumită în cod `balanced_aug`. Aceasta este construită pornind de la setul *Diabetic_Balanced_Aug*, care conține o colecție mai mare de imagini retiniene, dar cu o distribuție natural dezechilibrată. Pentru a o face compatibilă cu restul fluxului experimental, imaginile au fost împărțite stratificat în subseturi de antrenare, validare și testare, apoi au fost preprocesate folosind metoda Ben Graham. Această sursă este importantă deoarece oferă un volum mai mare de imagini normale și moderate, dar păstrează o provocare realistă: clasele severe rămân mult mai puțin reprezentate.

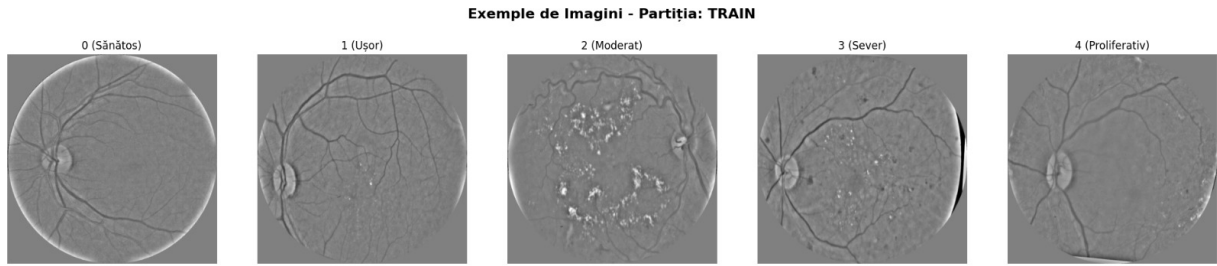


Figura 4.2: Exemple de imagini din setul *Diabetic_Balanced_Aug_Ben_Graham_matched* pentru fiecare clasă de severitate

A treia sursă este `datasets/aptos/aptos_ben_graham_matched`, derivată din APTOS 2019 Blindness Detection [7]. Setul APTOS este folosit pentru a verifica dacă modelele antrenate pe un domeniu pot generaliza pe imagini provenite dintr-o altă distribuție. În proiect, APTOS a fost reorganizat pe clase, balansat prin augmentări blânde și preprocesat cu aceeași metodă Ben Graham, astfel încât diferențele observate în rezultate să fie cauzate mai ales de conținutul vizual și de domeniul imaginilor, nu de un format diferit al fișierelor.

Toate sursele sunt organizate în directoarele `train`, `val` și `test`. În interiorul fiecărui subset există cinci directoare numerotate de la 0 la 4, corespunzătoare nivelurilor de severitate:

- **0** – fără retinopatie diabetică;
- **1** – retinopatie diabetică ușoară;
- **2** – retinopatie diabetică moderată;
- **3** – retinopatie diabetică severă;
- **4** – retinopatie diabetică proliferativă.

Această structură permite extragerea directă a etichetei din numele directorului clasei. În implementare, clasele `BalancedDRDataset` și `AptosDRDataset` parcurg aceste directoare, salvează căile imaginilor și atașează automat eticheta numerică. Astfel, aceeași logică de încărcare poate fi folosită pentru `balanced`, `balanced_aug`, `aptos` și pentru combinațiile acestora.

4.2 Distribuția claselor

Distribuțiile au fost calculate direct din structura locală a directoarelor. Tabelul 4.1 prezintă distribuția pentru `Diabetic_Balanced_Data`. Setul este aproape uniform pe cele cinci clase, cu o diferență minoră pentru clasa 1.

Tabela 4.1: Distribuția imaginilor în setul de date *Diabetic_Balanced_Data*

Clasa	Train	Validare	Test	Total
0	7000	2000	1000	10000
1	6792	1940	971	9703
2	7000	2000	1000	10000
3	7000	2000	1000	10000
4	7000	2000	1000	10000
Total	34792	9940	4971	49703

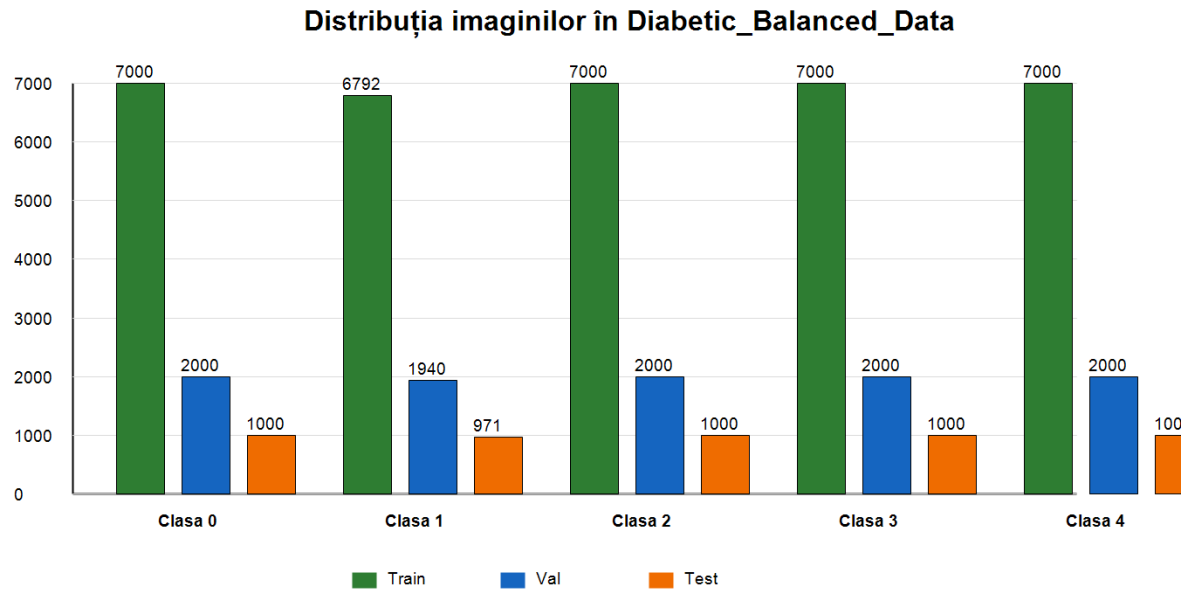


Figura 4.3: Distribuția imaginilor pe clase și subseturi în *Diabetic_Balanced_Data*

Setul *balanced_aug* are o distribuție diferită, prezentată în Tabelul 4.2. Deși numărul total de imagini este mai mare, clasele nu sunt echilibrate. Clasa 0 domină numeric, iar clasele 3 și 4 au considerabil mai puține exemple. Acest lucru îl face util pentru testarea comportamentului modelelor într-un scenariu mai apropiat de distribuțiile naturale întâlnite în practică.

Tabela 4.2: Distribuția imaginilor în setul *Diabetic_Balanced_Aug_Ben_Graham_matched*

Clasa	Train	Validare	Test	Total
0	18061	5160	2581	25802
1	1706	487	245	2438
2	3701	1057	530	5288
3	610	174	88	872
4	495	141	72	708
Total	24573	7019	3516	35108

Pentru APTOS, distribuția finală a fost controlată prin scriptul de reconstruire a setului. Fiecare clasă conține 1500 de imagini pentru antrenare, 200 pentru validare și 250 pentru testare, după aplicarea augmentărilor blânde și a preprocesării Ben Graham. Distribuția este prezentată în Tabelul 4.3.

Tabela 4.3: Distribuția imaginilor în setul *APTOS Ben Graham matched*

Clasa	Train	Validare	Test	Total
0	1500	200	250	1950
1	1500	200	250	1950
2	1500	200	250	1950
3	1500	200	250	1950
4	1500	200	250	1950
Total	7500	1000	1250	9750

Prin comparație, `Diabetic_Balanced_Data` și `APTOS_matched` sunt balansate, în timp ce `balanced_aug` păstrează un dezechilibru puternic. Această diferență este importantă pentru interpretarea rezultatelor: performanța ridicată pe un set echilibrat nu garantează automat robustețe pe un set dezechilibrat sau pe un set provenit din alt domeniu.

4.3 Construirea seturilor augmentate și preprocesate

Pentru a obține variante comparabile între ele, seturile suplimentare au fost generate prin scripturi dedicate. Aceste scripturi separă două etape: augmentarea offline, care produce fișiere noi pe disc, și preprocesarea imaginilor, care standardizează aspectul vizual înainte de antrenare.

Setul `balanced_aug` este construit prin scriptul dedicat conversiei `Diabetic_Balanced_Aug`. În implementare, fișierul folosit este:

```
build_diabetic_balanced_aug_ben_graham.py
```

Acesta citește imaginile din `Diabetic_Balanced_Aug/resized_train_cropped` și etichetele din fișierul `trainLabels_cropped.csv`. Imaginile sunt indexate după nume, apoi sunt împărțite stratificat în proporție de 80% pentru antrenare, 15% pentru testare și 5% pentru validare, folosind seed-ul 42. Fiecare imagine este apoi preprocesată Ben Graham și salvată într-o structură `train/val/test/0--4`.

Setul APTOS este refăcut prin scriptul `rebuild_aptos_dataset.py`. Mai întâi, scriptul copiază imaginile curate din `aptos_reorganized`, ignorând fișierele care conțin deja secvența `_aug` în nume. Apoi, pentru fiecare clasă și subset, generează augmentări până la atingerea țintelor stabilite: 1500 de imagini pe clasă în `train`, 200 în `val` și 250 în `test`. După această echilibrare, imaginile sunt preprocesate Ben Graham și salvate în `aptos_ben_graham_matched`.

Augmentările offline pentru APTOS sunt intenționat moderate. Scriptul aplică flip orizontal cu probabilitatea 0.5, flip vertical cu probabilitatea 0.15, rotații între -8 și 8 grade, scalări între 0.97 și 1.03 și translații de maximum 2.5% din dimensiunea imaginii. Zonele introduse prin transformări geometrice sunt completate cu gri neutru, pentru a nu crea margini negre artificiale. Aceste valori au fost alese pentru a simula variații realiste de achiziție, fără a modifica structura clinică a imaginii.

4.4 Preprocesarea Ben Graham

Preprocesarea Ben Graham are rolul de a reduce variațiile de iluminare, marginile negre și diferențele de scară între imagini. În proiect, această etapă este implementată în `ben_graham_preprocess_dataset.py` și este folosită atât pentru APTOS, cât și pentru varianta `balanced_aug`.

Fluxul aplicat fiecărei imagini este următorul:

1. se elimină marginile negre prin detectarea regiunilor cu intensitate peste pragul 7;
2. retina este scalată astfel încât raza vizibilă să fie apropiată de valoarea 300;
3. imaginea este centrată, decupată sau completată până la dimensiunea de lucru;

4. se aplică blur Gaussian și o combinație ponderată pentru accentuarea contrastului local;
5. se aplică o mască circulară, astfel încât zona retiniană să rămână centrală și fundalul să fie uniformizat;
6. imaginea este redimensionată la 512×512 pixeli;
7. rezultatul este convertit într-o reprezentare grayscale RGB cu contrast controlat.

Această preprocesare nu înlocuiește augmentarea, ci are un scop diferit. Augmentarea crește variația exemplilor, în timp ce Ben Graham reduce variația nedorită produsă de condițiile de captură. Folosirea aceleiași metode pentru APTOS și `balanced_aug` ajută la alinierea aspectului vizual al imaginilor înainte de antrenarea rețelelor.

4.5 Încărcarea datelor în experimente

Încărcarea imaginilor este realizată în fișierul `my_dataset.py`. Clasa `BalancedDRDataset` citește directoarele `train`, `val` sau `test`, parcurge subfolderele 0--4, deschide fiecare imagine cu biblioteca PIL, o convertește în format RGB și returnează imaginea împreună cu eticheta numerică. Clasa `AptosDRDataset` extinde aceeași logică, deoarece APTOS este deja organizat în format compatibil.

Funcția `make_dataset` permite selectarea sursei printr-un nume simbolic:

- `balanced` – folosește `Diabetic_Balanced_Data`;
- `balanced_aug` – folosește `Diabetic_Balanced_Aug_Ben_Graham_matched`;
- `aptos` – folosește `aptos_ben_graham_matched`;
- `combined` sau `balanced_aptos` – concatenează `balanced` și `aptos`;
- `balanced_aug_aptos` – concatenează `balanced_aug` și `aptos`.

Pentru seturile combinate se folosește `ConcatDataset`, astfel încât modelul să vadă imagini din ambele surse în aceeași etapă de antrenare. În scriptul `train.py`, aceste surse pot fi selectate fie prin parametrul `--experiment`, fie prin parametrii `--train_source` și `--test_source`. Opțiunea `--test_sources` permite evaluarea aceluiași model pe mai multe seturi de test, de exemplu separat pe APTOS și pe Balanced.

4.6 Augmentarea datelor

În această lucrare apar două tipuri de augmentare. Primul tip este augmentarea offline, aplicată înainte de antrenare și salvată pe disc. Aceasta apare în `Diabetic_Balanced_Data`, unde o parte dintre fișiere au secvența `_aug_` în nume, și în setul APTOS reconstruit, unde augmentările blânde sunt generate explicit de `rebuild_aptos_dataset.py`. Al doilea tip este augmentarea online, aplicată la încărcarea imaginilor în timpul antrenării.

Augmentarea online este controlată prin parametrul `train_augment`. Pentru valoarea implicită `basic`, transformările aplicate pe subsetul `train` sunt:

- redimensionare la dimensiunea configurată prin `image_size`;

- **RandomHorizontalFlip** cu probabilitatea 0.5;
- **RandomVerticalFlip** cu probabilitatea 0.5;
- **RandomRotation** cu un unghi maxim de 15 grade;
- **ColorJitter** cu variații moderate de luminozitate și contrast;
- conversie în tensor și normalizare.

Pentru **val** și **test**, transformările sunt deterministe: redimensionare, conversie în tensor și normalizare. Această separare este importantă deoarece validarea și testarea trebuie să măsoare comportamentul modelului pe imagini stabile, fără variații aleatorii introduse la fiecare rulare.

În experimentele realizate pe seturi deja augmentate offline, parametrul **train_augment** poate fi setat și la **none**. Această opțiune este utilă pentru a evita o dublă augmentare prea agresivă, mai ales atunci când imaginile au fost deja generate prin rotații, translații sau flip-uri. Astfel, pipeline-ul permite compararea între scenarii cu augmentare online activă și scenarii în care diversitatea provine doar din imaginile salvate în dataset.

4.7 Împărțirea datelor și controlul evaluării

Împărțirea în subseturi de antrenare, validare și testare este păstrată explicit în structura directoarelor. Pentru **Diabetic_Balanced_Data**, raportul este aproximativ 70% antrenare, 20% validare și 10% testare. Pentru **balanced_aug**, împărțirea este generată stratificat cu raportul 80%/5%/15%, iar pentru APTOS sunt folosite ținte fixe pe clasă.

Rolul subseturilor este următorul:

- **train** este folosit pentru actualizarea ponderilor modelului;
- **val** este folosit pentru monitorizarea antrenării, alegerea celui mai bun model și mecanisme precum early stopping;
- **test** este folosit doar la final, pentru evaluarea modelului salvat.

În **DataLoader**, subsetul de antrenare este încărcat cu **shuffle=True**, iar subseturile de validare și testare cu **shuffle=False**. În plus, experimentele cross-dataset, precum **balanced_to_aptos**, **balanced_aptos_to_aptos** sau **balanced_aug_aptos_to_balanced_aug**, sunt folosite pentru a separa două întrebări: cât de bine învață modelul distribuția pe care a fost antrenat și cât de bine se transferă pe imagini provenite dintr-o altă sursă.

Această organizare este esențială pentru interpretarea rezultatelor. Un model poate obține scoruri foarte bune pe un set echilibrat, dar să aibă performanțe mai slabe pe APTOS sau pe **balanced_aug**, unde distribuția claselor și aspectul imaginilor sunt diferite. De aceea, în capitolul experimental rezultatele sunt analizate nu doar prin acuratețe, ci și prin macro F1, AUC, quadratic weighted kappa și matrice de confuzie.

Capitolul 5

Metodologia propusă

5.1 Fluxul general al sistemului

Metodologia propusă urmărește construirea unui flux complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Sistemul pornește de la setul de date *Diabetic_Balanced_Data*, organizat în directoare de antrenare, validare și testare, și ajunge la salvarea unor modele antrenate care pot fi folosite ulterior într-o aplicație de inferență. Implementarea este realizată în Python, folosind PyTorch pentru antrenarea rețelelor, Torchvision și `timm` pentru arhitecturi pre-antrenate, iar scikit-learn, Matplotlib și Seaborn pentru evaluare și vizualizare.

Fluxul de lucru este împărțit în trei componente principale. Fișierul `my_dataset.py` se ocupă de încărcarea imaginilor și de aplicarea transformărilor. Fișierul `builder.py` definește modelele, funcțiile de pierdere, modul de calcul al predicțiilor și optimizatorii. Scriptul `train.py` integrează aceste componente, rulează antrenarea, validează modelul la fiecare epocă, salvează cea mai bună variantă și generează graficele de evaluare.

Problema este formulată ca o clasificare multiclasă în cinci clase, corespunzătoare stadiilor retinopatiei diabetice. Pentru fiecare imagine, modelul produce un vector de cinci scoruri, iar clasa finală este obținută prin alegerea valorii maxime în cazul funcțiilor de pierdere de tip Cross-Entropy și Focal Loss. Pentru varianta ordinală, ieșirea este interpretată prin praguri binare, deoarece clasele au o ordine clinică naturală.

5.2 Modelele analizate

În experimente au fost analizate arhitecturi convoluționale moderne, alese deoarece permit transfer learning și au fost utilizate frecvent în clasificarea imaginilor medicale [11], [2]. Implementarea permite construirea modelelor `resnet50`, `efficientnet_b3`, `inception_v3` și `incres_v2`. Accentul lucrării este pus pe compararea acestor familii arhitecturale, deoarece ele oferă perspective complementare asupra problemei: învățare reziduală, scalare eficientă, procesare multi-scală și combinația dintre module Inception și conexiuni reziduale.

Toate modelele sunt adaptate pentru cinci clase. Straturile finale, proiectate inițial pentru clasificarea ImageNet, sunt înlocuite sau configurate astfel încât ieșirea să corespundă celor cinci stadii ale retinopatiei diabetice. Folosirea ponderilor pre-antrenate este importantă deoarece permite reutilizarea unor trăsături vizuale generale, precum muchii, texturi, forme și tranziții de culoare, care sunt apoi ajustate pentru domeniul medical prin fine-tuning. Această strategie este utilă în special pentru seturile de date medicale, unde numărul de imagini etichetate este mai mic decât în colecțiile generale precum ImageNet, iar variațiile de iluminare, contrast și calitate a achiziției pot influența semnificativ predicția.

5.3 Arhitectura ResNet50

ResNet50 este o arhitectură convoluțională bazată pe conexiuni reziduale [20]. Ideea principală este ca un bloc al rețelei să nu învețe direct o transformare completă, ci o corecție față de intrarea primită. Această abordare facilitează antrenarea rețelelor adânci și reduce riscul degradării performanței odată cu creșterea numărului de straturi. Din acest motiv, ResNet50 este folosită frecvent ca model de referință în probleme de clasificare medicală: are o capacitate suficient de mare pentru a învăța trăsături complexe, dar rămâne mai ușor de antrenat și interpretat decât arhitecturile foarte adânci sau hibride.

În contextul imaginilor de fund de ochi, ResNet50 poate extrage trăsături ierarhice: primele straturi detectează contururi, variații de culoare și texturi simple, iar straturile profunde combină aceste informații în structuri mai complexe, precum vase retiniene, zone de exudate, hemoragii sau regiuni suspecte. În implementarea din `builder.py`, stratul final `fc` este înlocuit cu o secvență formată din `Dropout(0.5)` și un strat liniar cu cinci ieșiri. Dropout-ul are rol de regularizare și reduce tendința de supraînvățare. ResNet50 este astfel folosită atât ca bază de comparație, cât și ca arhitectură robustă pentru experimentele în care se evaluează influența funcțiilor de pierdere, a optimizatorilor și a strategiilor de monitorizare.

5.4 Arhitectura EfficientNet-B3

EfficientNet-B3 aparține familiei EfficientNet, propusă de Tan și Le [22], în care scalarea rețelei nu se face independent doar pe adâncime, lățime sau rezoluție, ci prin *compound scaling*. Această strategie crește simultan cele trei dimensiuni ale modelului folosind un coeficient comun, astfel încât arhitectura să păstreze un echilibru între capacitatea de reprezentare și costul computațional. Varianta B3 este relevantă pentru această lucrare deoarece oferă un compromis între modelele mai mici, care pot pierde detalii fine, și modelele foarte mari, care necesită mai multă memorie și prezintă risc mai ridicat de supraînvățare.

Din punct de vedere structural, EfficientNet utilizează blocuri de tip MBConv, inspirate din convoluțiile separabile pe adâncime și din conexiunile reziduale inversate. Aceste blocuri reduc numărul de parametri în comparație cu o convoluție standard, păstrând totuși capacitatea de a extrage trăsături vizuale relevante. Pentru imaginile de fund de ochi, această eficiență este importantă deoarece leziunile pot fi mici, slab contrastate și distribuite neuniform în imagine. O arhitectură care procesează intrări la rezoluții mai mari poate păstra mai bine informația locală, iar mecanismul de scalare al EfficientNet-B3 susține această nevoie fără a crește excesiv complexitatea modelului.

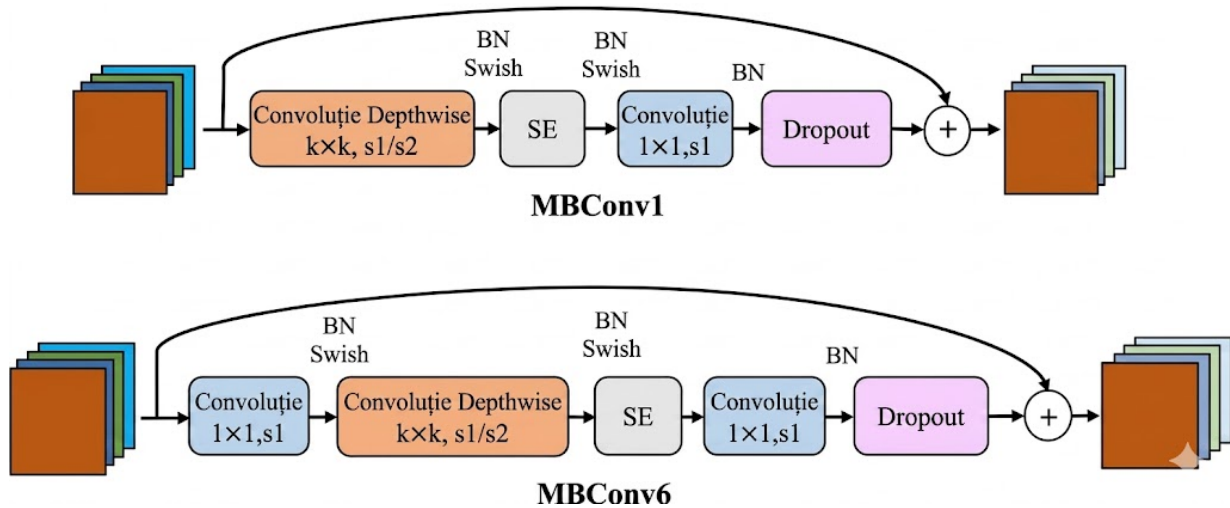


Figura 5.1: Structura blocurilor MBConv în EfficientNet-B3

În implementarea din `builder.py`, modelul este încărcat prin varianta EfficientNet-B3 din Torchvision, cu ponderi pre-antrenate atunci când parametrul `pretrained` este activ. Clasificatorul final este modificat prin înlocuirea stratului `classifier[1]` cu o secvență formată din Dropout (0.5) și un strat liniar cu cinci ieșiri. În cadrul experimentelor, EfficientNet-B3 oferă o alternativă compactă la ResNet50 și Inception-ResNet-V2, fiind utilă pentru a verifica dacă o arhitectură optimizată pentru raportul acuratețe-eficiență poate generaliza bine pe imagini retinene preprocesate și augmentate.

5.5 Arhitectura InceptionV3

InceptionV3 este o arhitectură construită în jurul modulelor Inception, care procesează aceeași intrare prin filtre convoluționale de dimensiuni diferite [21]. Această strategie permite extragerea simultană a trăsăturilor locale și a celor cu context spațial mai larg. Pentru retinopatia diabetică, acest lucru este relevant deoarece semnele patologice pot avea dimensiuni foarte diferite: microanevrismele sunt mici și discrete, în timp ce hemoragiile sau exudatele pot ocupa regiuni mai mari. Arhitectura folosește factorizarea convoluțiilor și module auxiliare pentru a îmbunătăți eficiența și stabilitatea antrenării, fiind una dintre arhitecturile importante folosite în sisteme de detecție automată a retinopatiei diabetice [1].

În cod, InceptionV3 este încapsulat prin clasa `InceptionWrapper`. Această clasă rezolvă particularitatea modelului Torchvision, care în timpul antrenării poate returna și o ieșire auxiliară. Pentru a păstra aceeași interfață cu funcțiile de pierdere, wrapper-ul returnează doar predicția principală. Ultimul strat complet conectat este înlocuit cu un strat liniar cu cinci ieșiri, corespunzătoare claselor 0–4. Prin includerea acestei arhitecturi, metodologia poate evalua dacă procesarea multi-scală aduce beneficii față de o rețea reziduală standard, mai ales în situațiile în care semnele clinice apar la scări foarte diferite în aceeași imagine.

5.6 Arhitectura Inception-ResNet-V2

Inception-ResNet-V2 este o arhitectură hibridă care combină modulele Inception cu conexiuni reziduale [26]. Din familia Inception preia ideea procesării multi-scală, prin ramuri convoluționale paralele, iar din familia ResNet preia conexiunile care facilitează propagarea gradientului. Rezultatul este o rețea profundă, capabilă să învețe reprezentări vizuale bogate

fără a pierde stabilitatea antrenării. În raport cu InceptionV3, arhitectura are o capacitate mai mare și poate beneficia de stabilitatea conexiunilor reziduale atunci când se realizează fine-tuning pe un set de date medical.

Această arhitectură este potrivită pentru imaginile retiniene deoarece decizia de severitate poate depinde atât de leziuni mici, cât și de distribuția globală a modificărilor patologice. Ramurile Inception pot surprinde structuri la scări diferite, iar conexiunile reziduale ajută la păstrarea informației utile de-a lungul rețelei. În implementare, modelul este construit cu biblioteca `timm`, folosind modelul `inception_resnet_v2` și configurând direct numărul de clase la cinci. În comparație cu ResNet50, modelul are o capacitate mai mare, dar în același timp necesită mai multe resurse de calcul. Din acest motiv, experimentele cu Inception-ResNet-V2 sunt utile pentru a observa dacă avantajul reprezentărilor mai bogate justifică un cost computațional mai ridicat.

5.7 Funcții de pierdere utilizate

Funcția de pierdere definește modul în care modelul este penalizat pentru predicții greșite. În această lucrare au fost folosite mai multe variante, pentru a compara atât comportamentul standard în clasificarea multiclasă, cât și strategii mai potrivite pentru clase dezechilibrate sau ordonate clinic.

- **Cross-Entropy** este funcția de pierdere de bază pentru clasificare multiclasă. Aceasta compară distribuția de probabilitate produsă de model cu eticheta reală și oferă un punct de referință stabil pentru celelalte experimente.
- **Weighted Cross-Entropy** extinde Cross-Entropy prin introducerea unor ponderi pentru clase. Această variantă este utilă atunci când anumite clase trebuie penalizate mai puternic, de exemplu pentru a reduce tendința modelului de a favoriza clasele mai ușor de învățat.
- **Focal Loss** este implementată prin clasa `FocalLossMultiClass` și pornește de la Cross-Entropy, dar reduce contribuția exemplelor ușoare și acordă mai multă importanță exemplorilor dificile [19]. Parametrul `gamma` controlează cât de puternic este accentuată această diferență. În experimentele inițiale a fost analizat intervalul 1.5–2.0, iar valoarea 1.6 a fost selectată pentru experimentele extinse deoarece a oferit un compromis bun între acuratețe, AUC și distribuția erorilor.
- **Varianta ordinală bazată pe BCEWithLogitsLoss** exploatează faptul că stadiile retinopatiei diabetice au o ordine naturală. În loc ca eticheta să fie tratată doar ca o clasă independentă, aceasta este transformată într-un vector care descrie nivelurile de severitate depășite.

5.8 Optimizatorul și hiperparametrii de antrenare

Optimizarea parametrilor este realizată prin funcția `get_optimizer`, iar hiperparametrii sunt transmiși prin linia de comandă în `train.py`. Această organizare permite rularea mai multor configurații fără modificarea manuală a codului.

- **Optimizatori disponibili:** implementarea permite selectarea între Adam, AdamW și SGD. Configurațiile principale folosesc rata de învățare 0.0001. Adam este potrivit pentru aceste experimente deoarece adaptează rata de actualizare pentru fiecare parametru și oferă o convergență stabilă în multe probleme de clasificare.

- **Regularizare prin `weight_decay`:** pentru Adam și AdamW este aplicată regularizarea parametrilor, cu rolul de a limita supraînvățarea și de a îmbunătăți generalizarea pe setul de validare și testare.
- **Parametri principali:** argumentele de bază controlează modelul ales, funcția de pierdere, optimizatorul, numărul de epoci, dimensiunea batch-ului, rata de învățare, dimensiunea imaginilor și numărul de procese pentru încărcarea datelor. Alte argumente stabilesc valoarea `gamma` pentru Focal Loss și sursele de date folosite la antrenare și evaluare.
- **Parametri adăugați pentru experimentele recente:** implementarea include argumente pentru metrica de monitorizare, scheduler, mixed precision, gradient clipping, media exponențială a ponderilor, test-time augmentation, dropout configurabil, înghețarea temporară a backbone-ului și controlul augmentărilor de antrenare. Aceștia permit control mai fin asupra criteriului de selecție, stabilității antrenării, evaluării și modului de fine-tuning.
- **Scheduler pentru rata de învățare:** scheduler-ul poate fi selectat între `plateau`, `cosine` și `none`. În varianta `plateau`, rata de învățare este redusă atunci când metrica monitorizată nu se mai îmbunătățește.
- **Metrica de monitorizare:** în experimentele recente, metrica monitorizată poate fi aleasă explicit. Modelul poate fi salvat în funcție de AUC, acuratețe, balanced accuracy, macro F1 sau QWK, în funcție de obiectivul experimentului.

5.9 Strategia de evaluare

Evaluarea se realizează în două etape. În timpul antrenării, modelul este monitorizat pe setul de validare, iar cea mai bună variantă este salvată pe disc. La final, modelul salvat este reîncărcat și evaluat pe setul de testare. Pentru experimentele de generalizare între domenii, implementarea permite evaluarea aceluiași model pe mai multe seturi de test în aceeași rulare, prin argumentul `test_sources`. Astfel, un model antrenat pe date combinate poate fi testat separat pe APTOS și pe setul Balanced, fără reluarea antrenării. Această separare este necesară pentru ca testarea să rămână o evaluare cât mai obiectivă a modelului ales.

Metricile urmărite sunt funcția de pierdere, acuratețea, AUC-ul multiclasă, balanced accuracy, macro F1, quadratic weighted kappa, raportul de clasificare, matricea de confuzie și curbele ROC. AUC-ul este calculat în regim one-vs-rest și mediat macro, astfel încât fiecare clasă să contribuie egal la scorul final.

Pentru retinopatia diabetică, QWK este relevantă deoarece clasele au o ordine naturală. O eroare între două clase vecine este mai puțin gravă decât o eroare între clase îndepărtate, iar această metrică penalizează mai puternic diferențele mari de severitate. Din acest motiv, experimentele recente folosesc frecvent QWK ca metrică principală de monitorizare. În plus, scriptul include generarea de hărți Grad-CAM, utile pentru a observa regiunile imaginii care influențează predicția modelului.

Grad-CAM: Unde se uită modelul pentru a pune diagnosticul?

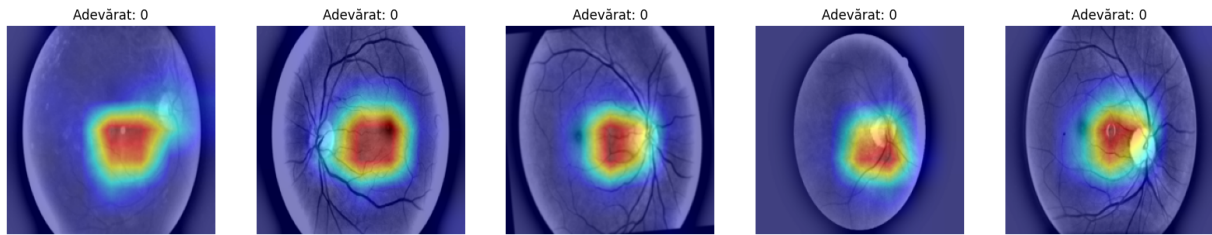


Figura 5.2: Exemplu de hărți Grad-CAM pentru un model ResNet50 antrenat cu Focal Loss și Adam

5.10 Implementarea procesului de antrenare

Scriptul `train.py` coordonează întregul proces experimental. După citirea argumentelor, acesta selectează dispozitivul de calcul disponibil, creează directoarele pentru rezultate și încarcă datele prin clasele de dataset definite în `my_dataset.py`. Implementarea permite folosirea surselor `balanced`, `balanced_aug`, `aptos`, precum și a combinațiilor `combined` și `balanced_aug_aptos`. Fiecare dataset este transmis către un `DataLoader`, cu `shuffle=True` pentru antrenare și `shuffle=False` pentru validare și testare.

Pentru fiecare epocă, modelul trece printr-o fază de antrenare și una de validare. În faza de antrenare se calculează gradientul și se actualizează parametrii, iar în faza de validare calculele sunt realizate fără gradient. Dacă metrica de selecție se îmbunătățește, ponderile sunt salvate într-un fișier `.pth`. La final, scriptul generează graficele de istoric, matricea de confuzie, curbele ROC și, atunci când este posibil, imaginile Grad-CAM. În cazul evaluării pe mai multe surse de test, fiecare set primește fișiere separate, marcate în nume prin sufixe precum `test_aptos`, `test_balanced` sau `test_balanced_aug`.

Pentru seturile care conțin deja imagini augmentate sau preprocesate, scriptul permite dezactivarea augmentărilor online prin `train_augment=None`. Această opțiune evită aplicarea dublă a augmentărilor în experimentele recente, păstrând în același timp comportamentul vechi prin valoarea implicită `train_augment=basic`.

5.11 Optimizări recente ale protocolului experimental

Pentru experimentele finale au fost introduse mai multe optimizări care nu modifică dataset-urile, ci procesul de antrenare și evaluare. Prima optimizare este folosirea "mixed precision" prin `torch.amp`, care reduce consumul de memorie și poate accelera antrenarea pe GPU. A doua este "gradient clipping", folosit pentru a limita actualizările foarte mari ale parametrilor și pentru a crește stabilitatea.

A fost introdusă și media exponențială a ponderilor modelului, prin parametrul `ema_decay`. Modelul EMA păstrează o versiune netezită a parametrilor și poate produce predicții mai stabile la validare și testare. Pentru evaluare a fost adăugat `tta_eval`, care aplică test-time augmentation prin combinarea predicției pe imaginea originală cu predicția pe imaginea oglindită orizontal.

Pentru ResNet50, aceste optimizări sunt folosite împreună cu Focal Loss și `gamma=1.6`. Configurația urmărește îmbunătățirea performanței fără a schimba arhitectura de bază.

Pentru Inception-ResNet-V2, care reprezintă modelul principal vizat, protocolul include și fine-tuning gradual: înghețarea temporară a backbone-ului, rată de învățare mai mică pentru backbone, dropout configurabil și imagini de intrare mai mari. Aceste setări sunt menite să protejeze reprezentările preantrenate și să reducă supraînvățarea.

Capitolul 6

Rezultate experimentale

6.1 Configurația experimentală

Capitolul de rezultate sintetizează experimentele rulate în notebook-ul `experiments.ipynb`, rezultatele inițiale și experimentele extinse salvate în directoarele de rezultate ale notebook-urilor. Prima etapă a urmărit compararea arhitecturilor și a funcțiilor de pierdere pe setul *Diabetic_Balanced_Data*. Etapele următoare au analizat generalizarea între domenii, prin antrenare pe Balanced, APTOS sau combinații ale acestora și prin evaluare separată pe fiecare sursă de test.

Metricile principale urmărite au fost acuratețea, AUC-ul multiclasă, macro F1 și, pentru experimentele recente, QWK. Acuratețea și AUC-ul au fost extrase din output-urile notebook-ului, iar macro F1 și valorile pe clase au fost calculate din rapoartele de clasificare salvate în fișierele `classification_report` și `prf1`.

Tabela 6.1: Rezultatele inițiale pe setul *Diabetic_Balanced_Data*

Model	Funcție pierdere	Optimizer	Val. Acc. max.	Test Acc.	Test AUC	Observație
ResNet50	Focal Loss	Adam	83.2%	82.94%	0.9669	Bază stabilă, dar sub variantele CE.
ResNet50	Focal Loss	AdamW	83.6%	83.44%	0.9684	Ușor peste Adam, dar diferență redusă.
ResNet50	Cross-Entropy	Adam	85.0%	84.41%	0.9734	Rezultat bun pentru configurația standard.
ResNet50	Weighted CE	Adam	84.6%	84.73%	0.9730	Cea mai bună variantă ResNet50 inițială după acuratețe.
EfficientNet-B3	Focal Loss	Adam	85.5%	85.25%	0.9733	Performanță competitivă, între ResNet50 și arhitecturile Inception.
EfficientNet-B3	Weighted CE	Adam	80.4%	80.10%	0.9560	Variantă mai slabă, afectată de ponderarea agresivă a claselor.
InceptionV3	Focal Loss	Adam	86.1%	86.04%	0.9770	Beneficiază de procesarea multi-scală.
Inception-ResNet-V2	Focal Loss	Adam	86.7%	86.52%	0.9761	Cea mai bună acuratețe inițială, cu cost computațional ridicat.

EfficientNet-B3 cu Focal Loss a obținut o acuratețe de 85.25%, AUC de 0.9733 și macro F1 de 0.8500, ceea ce îl plasează între ResNet50 și arhitecturile Inception. Varianta EfficientNet-B3 cu Weighted Cross-Entropy a fost mai slabă, cu 80.10% acuratețe, AUC 0.9560 și macro F1 0.7989.

Tabela 6.2: Analiza parametrului γ pentru ResNet50 cu Focal Loss pe Balanced

Gamma	Test Acc.	Test AUC	Macro F1	Interpretare
1.5	88.61%	0.9811	0.8853	Cel mai bun rezultat pe Balanced, dar puțin mai slab la matricea de confuzie față de 1.6.
1.6	81.37%	0.9620	0.8111	Configurație folosită ulterior pentru experimentele de generalizare.
1.7	79.62%	0.9582	0.7926	Scădere vizibilă a performanței.
1.8	81.09%	0.9597	0.8102	Performanță apropiată de $\gamma=1.6$.
1.9	79.84%	0.9588	0.7941	Nu aduce îmbunătățiri.
2.0	80.23%	0.9590	0.8012	Penalizarea exemplelor ușoare devine prea puternică.

Experimentele cross-dataset au arătat o diferență clară între performanța pe același domeniu și performanța pe imagini provenite din altă distribuție.

Tabela 6.3: Rezultate cross-dataset și evaluare separată pe domenii

Model	Train/Val	Test	Acc.	AUC	Macro F1	QWK
ResNet50	Balanced	APTOS	40.40%	0.7434	0.3704	–
ResNet50	APTOS	Balanced	30.34%	0.6352	0.2122	–
ResNet50	Balanced + APTOS	APTOS	62.32%	0.8595	0.6165	0.7573
ResNet50	Balanced + APTOS	Balanced	90.40%	0.9855	0.9036	0.9349
ResNet50	Balanced Aug + APTOS	APTOS	62.08%	0.8564	0.6150	–
ResNet50	Balanced Aug + APTOS	Balanced Aug	79.32%	0.8391	0.4659	–
Inception-ResNet-V2	Balanced + APTOS	Combined	78.51%	–	0.7893	–
Inception-ResNet-V2	Balanced Aug + APTOS	APTOS	66.00%	–	0.6572	–
Inception-ResNet-V2	Balanced Aug + APTOS	Balanced Aug	82.00%	–	0.5414	–

Valorile din tabel indică faptul că adăugarea APTOS în antrenare îmbunătățește transferul către APTOS față de modelul antrenat doar pe Balanced, dar nu elimină complet decalajul de domeniu. Pentru Balanced Aug, acuratețea este bună, însă macro F1 scade, ceea ce arată că performanța globală este influențată de distribuția claselor și că unele clase sunt recunoscute mai slab.

6.2 Evoluția funcției de pierdere

Evoluția funcției de pierdere arată că modelele învață rapid în primele epoci, după care diferența dintre pierderea de antrenare și cea de validare devine mai vizibilă. În experimentele

inițiale, ResNet50, InceptionV3 și Inception-ResNet-V2 au prezentat o scădere constantă a pierderii de antrenare, semn că rețelele se adaptează la imaginile retiniene.

Pentru experimentele cross-dataset, curbele confirmă că antrenarea pe o distribuție mixtă este mai dificilă decât antrenarea pe Balanced. În cazul Balanced Aug + APTOS, validarea se stabilizează devreme, iar mecanismul de early stopping se activează după 14 epoci pentru ResNet50, cu cea mai bună valoare AUC de validare în jurul epocii 4.

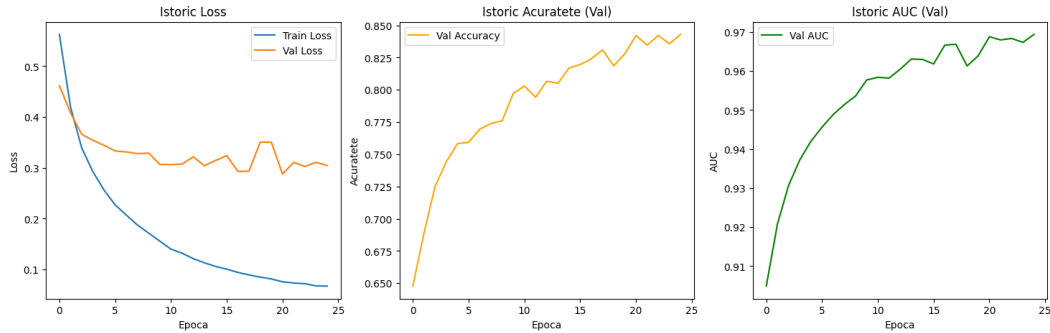


Figura 6.1: Evoluția antrenării pentru Inception-ResNet-V2 cu Focal Loss în experimentele inițiale

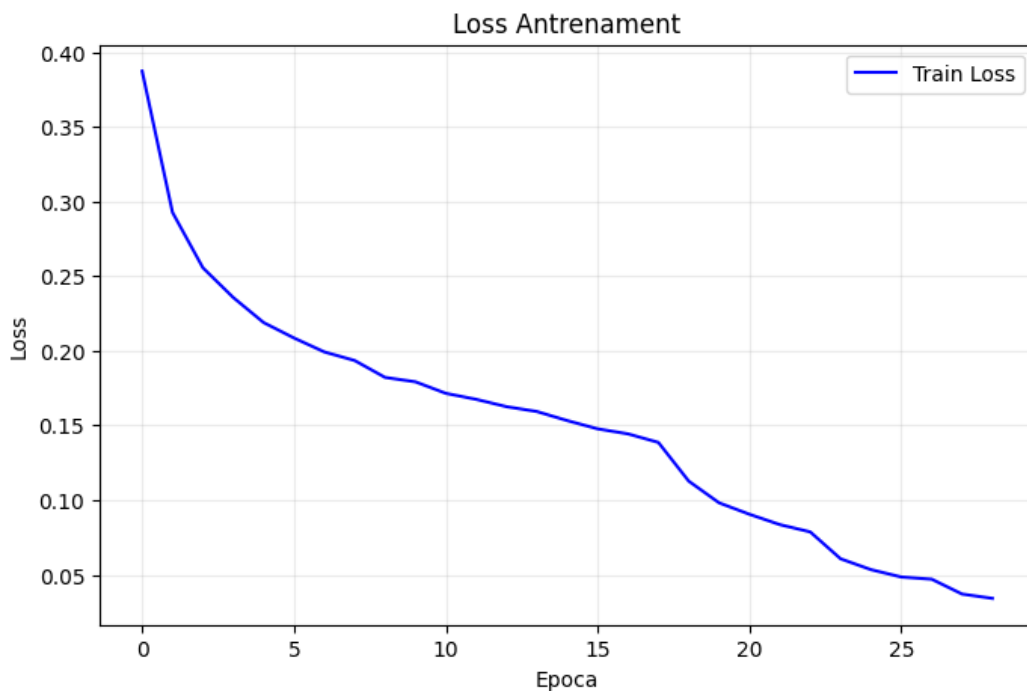


Figura 6.2: Evoluția funcției de pierdere pentru ResNet50 pe Balanced Aug + APTOS

6.3 Evoluția acurateții pe setul de validare

În experimentele inițiale, acuratețea pe validare crește treptat pentru toate modelele. ResNet50 ajunge la aproximativ 83–85% în funcție de funcția de pierdere, InceptionV3 ajunge la 86.1%, iar Inception-ResNet-V2 atinge 86.7%. Aceste valori se reflectă și în testare: Inception-ResNet-V2 obține 86.52%, iar InceptionV3 86.04%, depășind variantele ResNet50.

În experimentele extinse, acuratețea de validare nu mai este suficientă pentru a descrie generalizarea. De exemplu, modelul ResNet50 antrenat pe Balanced + APTOS ajunge la 62.32% acuratețe pe APTOS, dar la 90.40% pe Balanced. Această diferență arată că validarea combinată poate ascunde diferențe importante între domenii.

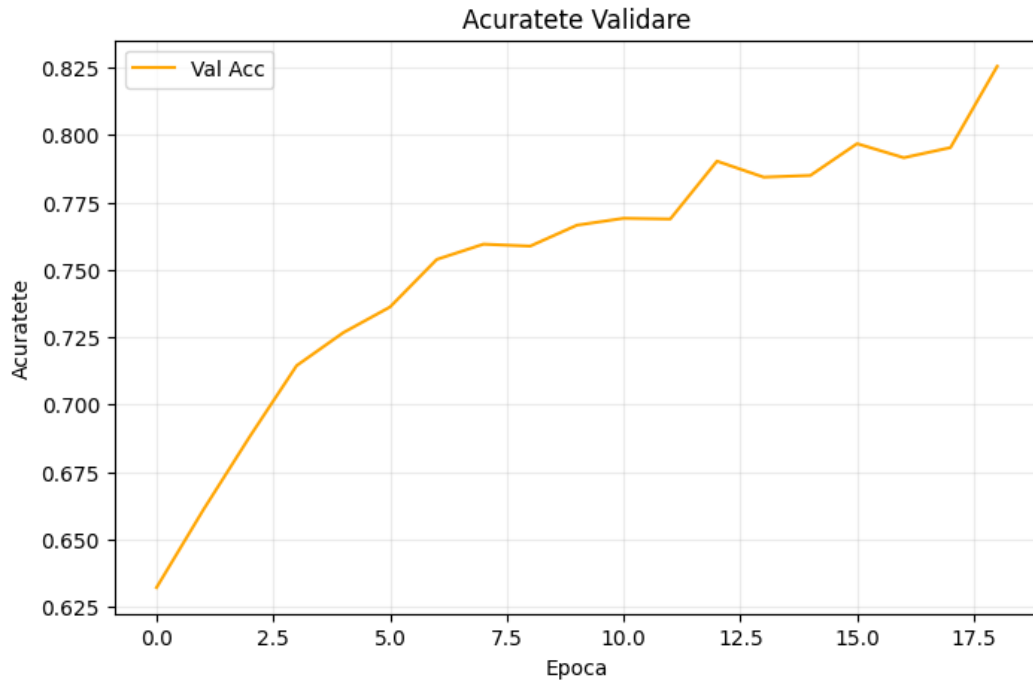


Figura 6.3: Evoluția acurateții pentru ResNet50 antrenat pe Balanced + APTOS

6.4 Analiza scorului AUC

AUC-ul a fost una dintre cele mai stabile metrice în experimentele inițiale. ResNet50 cu Cross-Entropy a obținut AUC 0.9734, ResNet50 cu Weighted Cross-Entropy 0.9730, InceptionV3 0.9770, iar Inception-ResNet-V2 0.9761. EfficientNet-B3 cu Focal Loss a obținut AUC 0.9733, confirmând că arhitectura separă bine clasele.

În schimb, AUC-ul scade puternic în transferul între domenii. Balanced $\rightarrow$ APTOS are AUC 0.7434, iar APTOS $\rightarrow$ Balanced are AUC 0.6352. Prin adăugarea APTOS în antrenare, AUC-ul pe APTOS crește la aproximativ 0.86–0.88, însă rămâne sub valorile obținute pe Balanced.

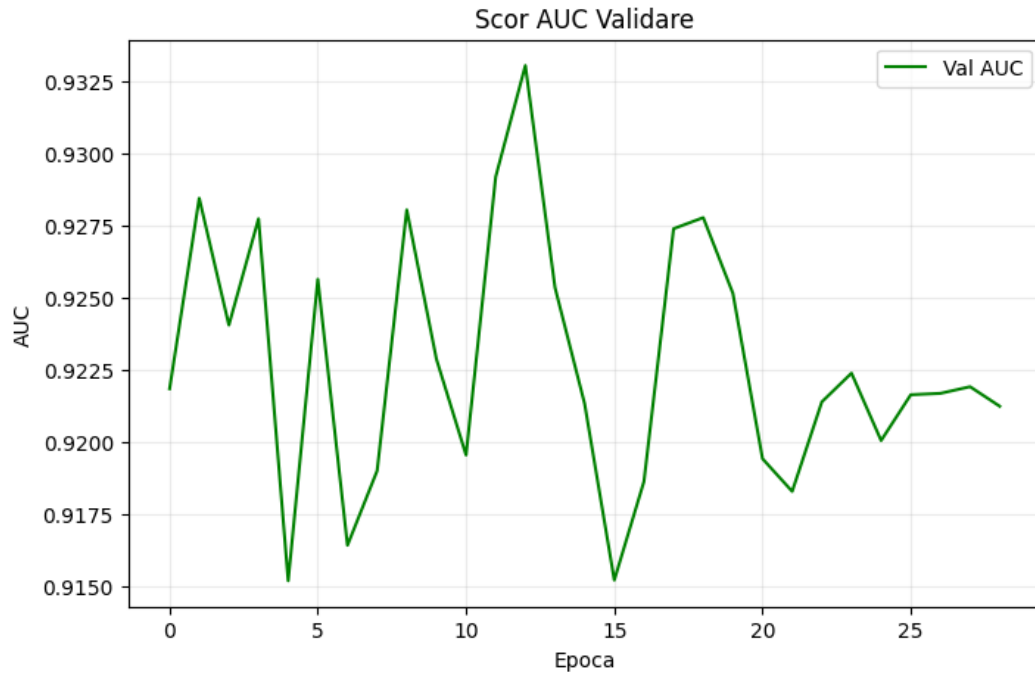


Figura 6.4: Evoluția AUC pentru ResNet50 pe Balanced Aug + APTOS

6.5 Analiza curbelor ROC multclasă

Curbele ROC confirmă tendința observată prin AUC: clasele extreme sunt, în general, mai ușor de separat, iar clasele intermediare sunt mai dificile. În experimentele pe Balanced, curbele sunt ridicate pentru majoritatea claselor, ceea ce explică AUC-urile apropiate de 0.97–0.98.

Compararea curbelor ROC pentru același model evaluat pe APTOS și Balanced arată că performanța agregată nu este suficientă. Modelul poate păstra o separare foarte bună pe Balanced, dar poate avea praguri mult mai nesigure pe APTOS.

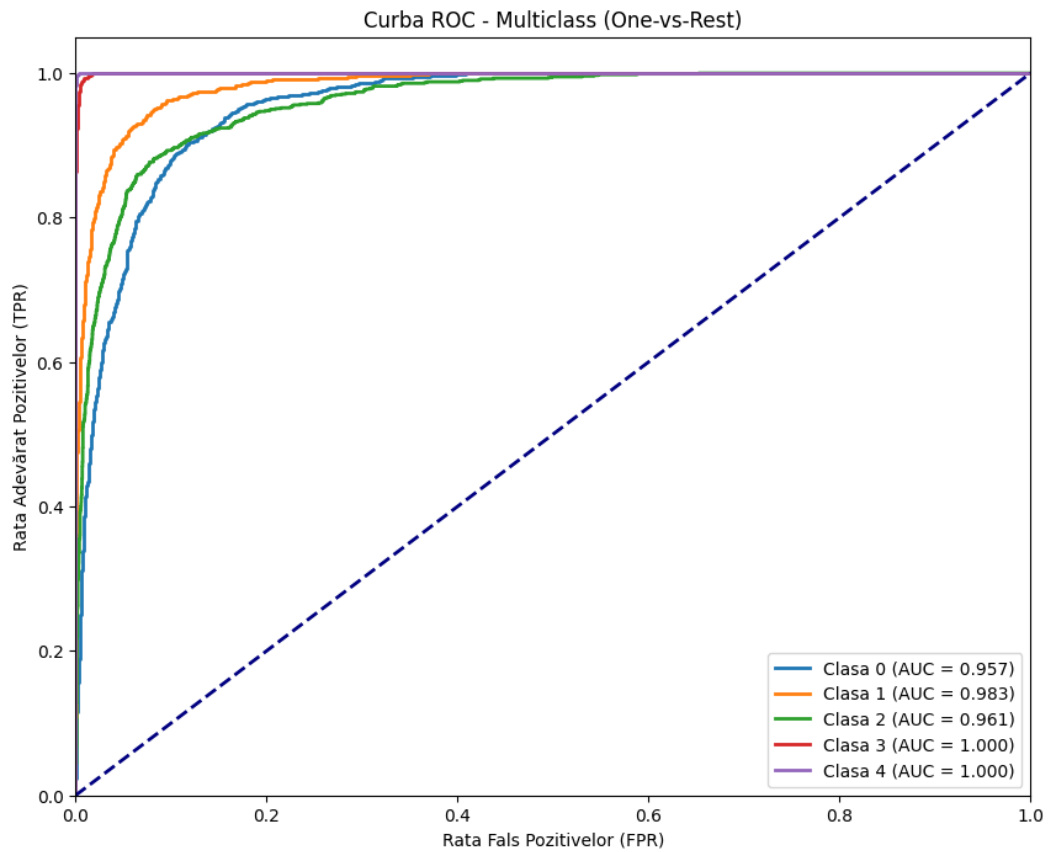


Figura 6.5: Curbe ROC pentru InceptionV3 în experimentele inițiale

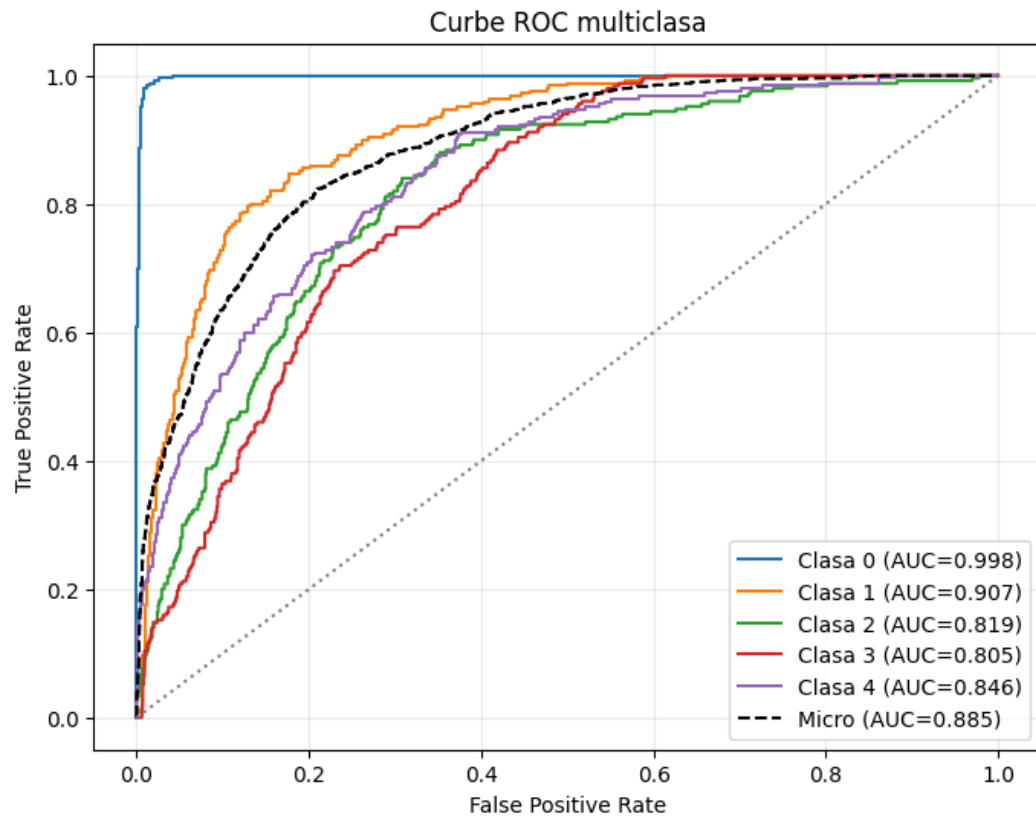


Figura 6.6: Curbe ROC pentru ResNet50 antrenat pe Balanced + APTOS și testat pe APTOS

6.6 Analiza matricelor de confuzie

Matricile de confuzie arată că modelele obțin cele mai bune rezultate pentru clasele avansate, în special clasele 3 și 4, unde semnele patologice sunt mai vizibile. Pentru ResNet50 cu $\gamma=1.6$ pe Balanced, raportul de clasificare indică F1 de 0.9553 pentru clasa 3 și 0.9807 pentru clasa 4. În schimb, clasele 0, 1 și 2 sunt mai apropiate vizual și produc confuzii mai frecvente.

Această distribuție a erorilor este importantă clinic. O confuzie între clase vecine poate reflecta tranziția graduală a bolii, însă o confuzie între o imagine fără retinopatie și o formă proliferativă ar avea risc mult mai mare. În experimentele cross-dataset, matricile de confuzie pe APTOS arată o degradare mai puternică a claselor intermediare.

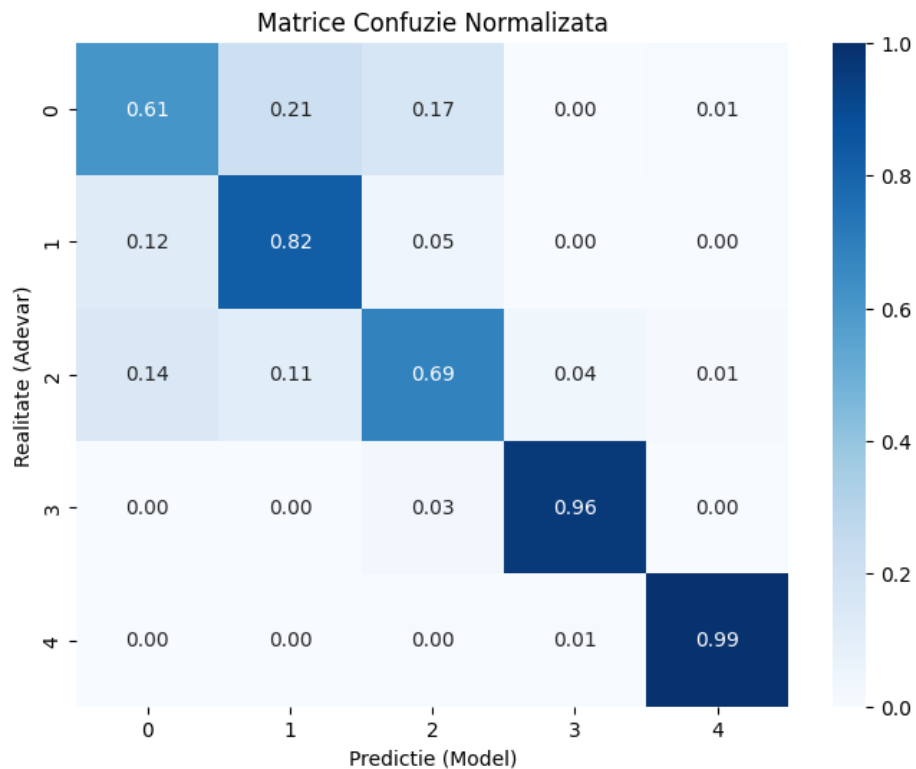


Figura 6.7: Matrice de confuzie normalizată pentru ResNet50 pe Balanced, $\gamma=1.6$

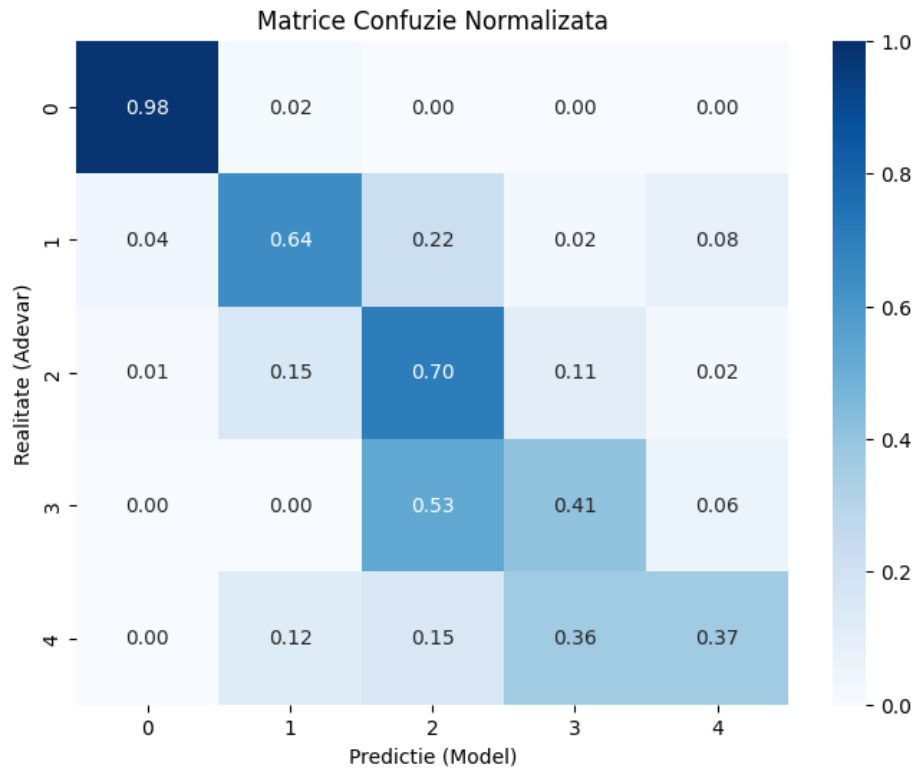


Figura 6.8: Matrice de confuzie normalizată pentru ResNet50 antrenat pe Balanced + APTOS și testat pe APTOS

6.7 Comparație între modelele testate

Compararea arhitecturilor arată că modelele mai complexe aduc un avantaj în experimentele inițiale pe Balanced. InceptionV3 obține 86.04% acuratețe și AUC 0.9770, iar Inception-ResNet-V2 obține 86.52% acuratețe și AUC 0.9761. ResNet50 rămâne competitiv, dar variantele sale inițiale se opresc între 82.94% și 84.73% acuratețe. EfficientNet-B3 cu Focal Loss ajunge la 85.25%, confirmând că scalarea eficientă poate oferi un compromis bun între performanță și complexitate.

În scenariile cross-dataset, diferența dintre arhitecturi este mai puțin importantă decât diferența dintre domenii. ResNet50 și Inception-ResNet-V2 pot obține scoruri bune pe Balanced, dar performanța pe APTOS rămâne limitată. Inception-ResNet-V2 pe Balanced Aug + APTOS obține 66.00% acuratețe și macro F1 0.6572 pe APTOS, ușor peste ResNet50 în același scenariu, dar pe Balanced_Aug scorul macro F1 rămâne redus, 0.5414, deși acuratețea este 82.00%.

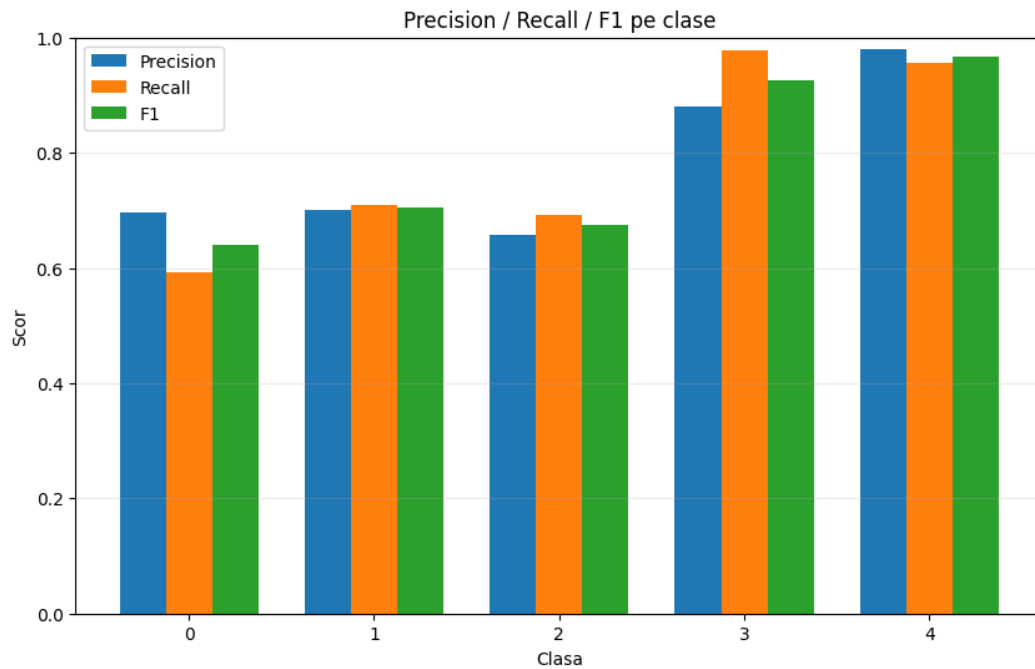


Figura 6.9: Precision, recall și F1 pe clase pentru ResNet50 testat pe Balanced după antrenare Balanced + APTOS

6.8 Interpretarea erorilor de clasificare

Erorile de clasificare apar în principal în clasele intermediare, unde diferențele clinice și vizuale sunt mai subtile. În rapoartele de clasificare, clasele 3 și 4 au de obicei scoruri F1 ridicate, în timp ce clasele 0–2 au scoruri mai mici și sunt mai sensibile la domeniul de testare.

Pentru interpretarea vizuală a deciziilor, scriptul generează hărți Grad-CAM. Acestea evidențiază zonele imaginii care au contribuit cel mai mult la predicție. Grad-CAM este util ca instrument de inspecție, însă nu înlocuiește analiza cantitativă prin metrice și matrici de confuzie.

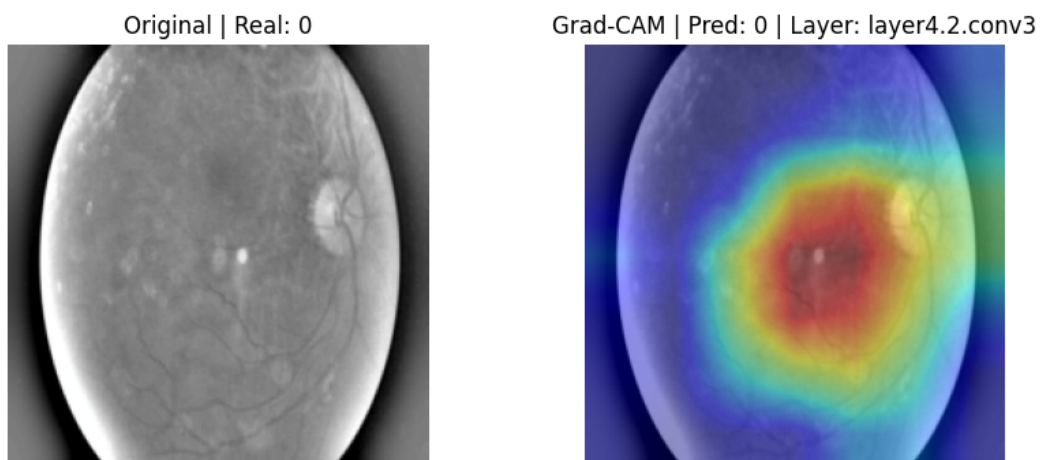


Figura 6.10: Exemplu Grad-CAM pentru ResNet50 evaluat pe Balanced după antrenare Balanced + APTOS

Concluzia generală a capitolului este că modelele pot atinge performanțe ridicate pe testul

provenit din aceeași distribuție, în special pe Balanced, dar generalizarea între seturi de date rămâne dificilă. Cele mai bune rezultate inițiale sunt obținute de Inception-ResNet-V2 și InceptionV3, iar EfficientNet-B3 se comportă competitiv. Pentru utilizare practică, rezultatele pe APTOS arată că este necesară o strategie mai puternică de adaptare la domeniu.

Capitolul 7

Aplicația de inferență

7.1 Scopul aplicației dezvoltate

Pentru a demonstra utilizarea practică a modelelor antrenate, a fost dezvoltată o aplicație web de inferență. Scopul aplicației este de a permite încărcarea unei imagini de fund de ochi, selectarea unui model salvat și obținerea unei predicții privind stadiul retinopatiei diabetice. Aplicația are rol experimental și nu înlocuiește diagnosticul medical, deoarece sistemele automate pentru retinopatia diabetică necesită validare clinică riguroasă înainte de utilizare în screening sau decizie medicală [1], [14].

Rezultatul afișat include clasa prezisă, denumirea stadiului, nivelul de încredere, probabilitățile pentru fiecare clasă, timpul de inferență și recomandări orientative pentru medic și pacient. În versiunea recentă, interfața include și informații tehnice suplimentare despre model, pentru a susține analiza din perspectiva inginerului. Aceste informații arată cum poate fi conectat un model antrenat la o interfață utilizabilă, nu doar la un script de evaluare.

7.2 Arhitectura aplicației

Aplicația este organizată în două componente principale: un backend Flask și un frontend React, două tehnologii potrivite pentru separarea logicii de inferență de interfața utilizatorului [27], [28]. Backend-ul este responsabil pentru încărcarea modelului, verificarea și preprocesarea imaginii, precum și calcularea predicției. Frontend-ul este responsabil pentru interfața de utilizare, selecția modelului, încărcarea imaginii, vizualizarea 3D și afișarea rezultatului.

Cele două componente comunică prin cereri HTTP. Backend-ul expune endpoint-uri sub forma `/api/models` și `/api/predict`, iar frontend-ul trimite cereri către acestea folosind biblioteca Axios [29]. Pentru comunicarea între porturi diferite, backend-ul folosește `flask_cors`, care permite configurarea politicilor CORS pentru aplicații Flask [30].

7.3 Componenta backend

Componenta backend este implementată în fișierul `Inference_site/app.py`. Aceasta definește aplicația Flask, stabilește directorul în care se află modelele salvate și selectează dispozitivul de calcul: CUDA, dacă este disponibil, sau CPU în caz contrar. Pentru reconstruirea arhitecturii modelului, backend-ul importă funcția `get_model` din `builder.py`.

Endpoint-ul `/api/models` returnează lista fișierelor `.pth` disponibile. Endpoint-ul `/api/predict` primește imaginea și numele modelului, verifică existența fișierului, aplică transformările necesare și rulează inferența. Dacă imaginea pare să fie o imagine brută, de exemplu din EyePACS, backend-ul aplică preprocesarea necesară înainte de inferență. Imaginea este convertită în RGB, redimensionată la dimensiunea așteptată de model, transformată în tensor și normalizată cu valorile ImageNet. Predicția este calculată cu `torch.no_grad()`, pentru a dezactiva calculul gradientului în timpul inferenței, iar probabilitățile sunt obținute prin softmax [31].

7.4 Componenta frontend

Componenta frontend este implementată în `Inference_site/App.jsx`. La pornire, aceasta solicită lista de modele disponibile și selectează automat primul model găsit. Utilizatorul poate alege un model din listă, poate încărca o imagine și poate vizualiza o previzualizare înainte de trimiterea către backend.

După apăsarea butonului de analiză, frontend-ul construiește un obiect `FormData` care conține imaginea și numele modelului selectat. Răspunsul primit de la backend este afișat în zone distincte pentru rezultat, recomandări și metadata tehnice. Aceste zone au fost echilibrate vizual pentru a avea dimensiuni comparabile și pentru a rămâne ușor de citit pe rezoluții diferite.

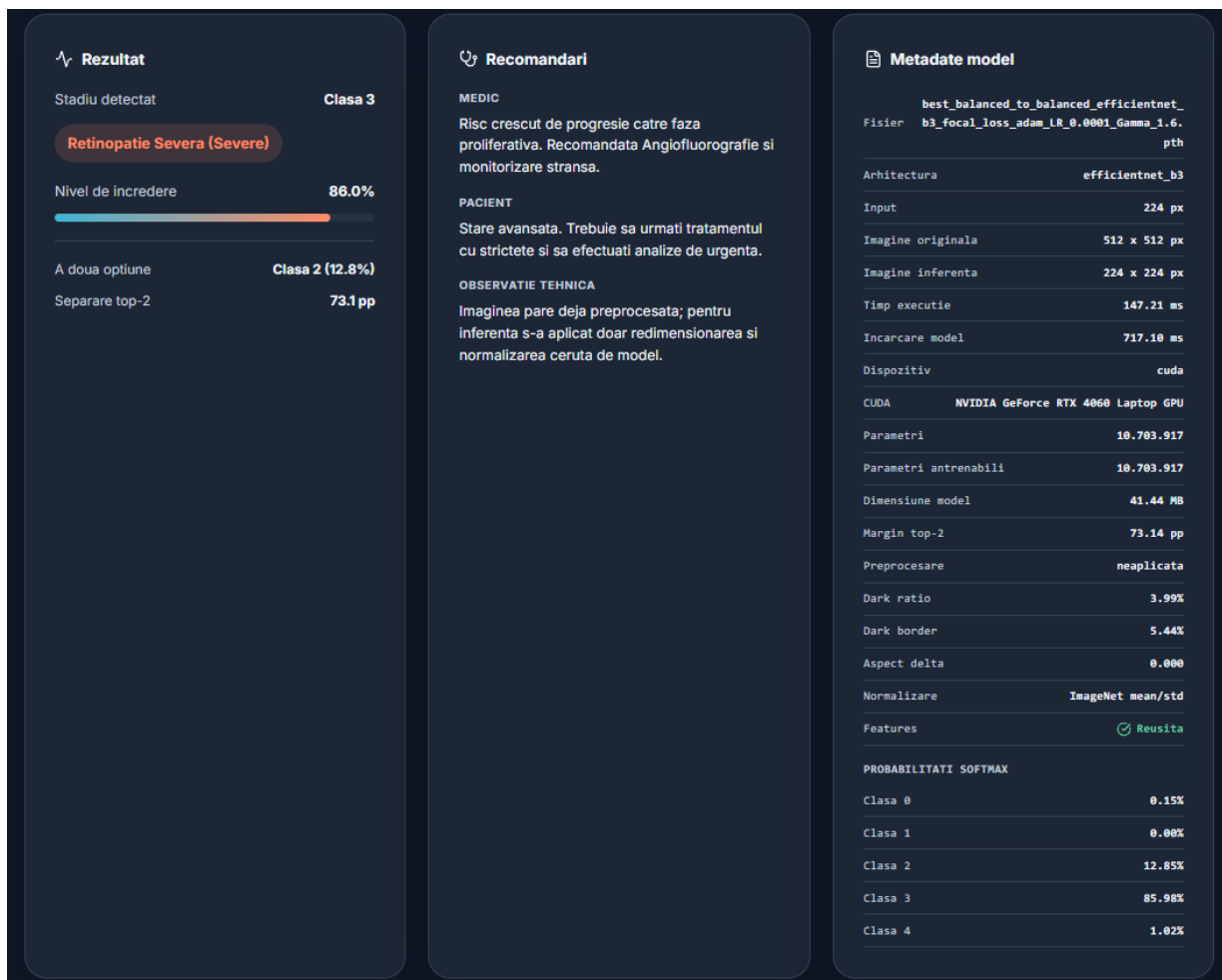


Figura 7.1: Zona de recomandări afișată în aplicația de inferență după obținerea predicției

Interfața a fost adaptată pentru utilizare responsive și include un comutator pentru modul light/dark. De asemenea, aplicația conține o vizualizare 3D a imaginii originale încărcate, nu a imaginii preprocesate. Utilizatorul poate roti obiectul cu mouse-ul, ceea ce oferă o reprezentare interactivă a fundului de ochi în interiorul aplicației. Pentru această componentă a fost utilizată biblioteca `Three.js`, care simplifică randarea graficii 3D în browser prin WebGL [32].

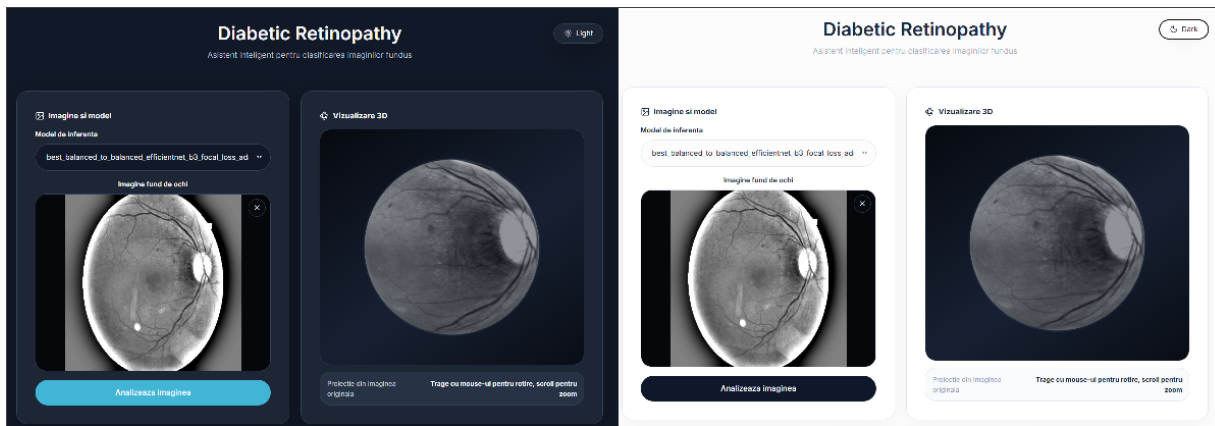


Figura 7.2: Interfața aplicației de inferență dezvoltate pentru clasificarea retinopatiei diabetice

7.5 Fluxul de utilizare

Fluxul de utilizare este simplu. Utilizatorul pornește backend-ul, deschide frontend-ul, selectează modelul dorit, încarcă o imagine de fund de ochi și apasă butonul de analiză. Imaginea este trimisă către server, unde este preprocesată și analizată de modelul ales. Rezultatul este apoi returnat în format JSON și afișat în interfața web.

Această structură permite testarea rapidă a mai multor modele pe aceeași imagine. De exemplu, o imagine poate fi analizată cu ResNet50, InceptionV3 și Inception-ResNet-V2, iar diferențele dintre probabilități pot oferi indicii despre stabilitatea predicției.

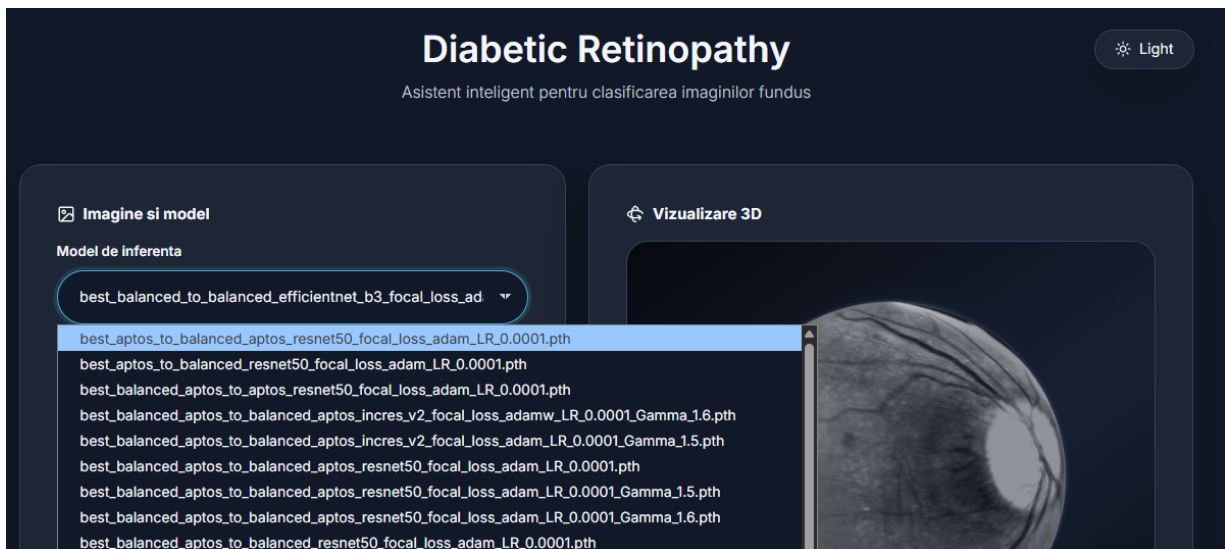


Figura 7.3: Meniul dropdown pentru selecția modelului în aplicația de inferență

7.6 Integrarea modelului antrenat

Integrarea modelului antrenat se bazează pe reutilizarea aceleiași funcții `get_model` folosite în etapa experimentală. Fișierul `.pth` salvează ponderile, nu întreaga logică a arhitecturii, deci modelul trebuie reconstruit înainte de încărcarea parametrilor. Backend-ul identifică arhitectura prin funcția `get_base_model_name`, analizând numele fișierului selectat.

Dacă numele conține `resnet50`, se construiește un ResNet50; dacă numele conține `inception_v3`, se construiește InceptionV3; iar pentru `incres_v2`, se construiește Inception-ResNet-V2. Ponderile sunt încărcate cu `torch.load`, folosind `map_location` pentru compatibilitate cu dispozitivul disponibil. Această abordare păstrează coerența între antrenare și inferență.

7.7 Limitări practice ale aplicației

Aplicația are rol demonstrativ și prezintă mai multe limitări. În primul rând, verificarea preprocesării nu este echivalentă cu validarea medicală a imaginii și nu garantează că imaginea are o calitate suficientă pentru diagnostic. În al doilea rând, modelul este încărcat la fiecare cerere, ceea ce este simplu pentru testare, dar inefficient pentru producție. Controlul calității imaginilor este o etapă importantă în evaluarea sistemelor pentru retinopatie diabetică, deoarece imaginile neclare, slab iluminate sau incorect centrate pot afecta predicția modelului [15].

Recomandările afișate sunt statice și orientative. Ele nu țin cont de istoricul pacientului, de acuitatea vizuală, de edemul macular, de tratamente existente sau de alte informații clinice. Într-o versiune reală, aplicația ar trebui să includă validare medicală, control al calității imaginilor, autentificare, audit și un mecanism de marcare a predicțiilor incerte. În plus, integrarea într-un flux clinic ar trebui însoțită de explicabilitate și analiză a erorilor, astfel încât deciziile modelului să poată fi interpretate de utilizatori umani [7].

Capitolul 8

Concluzii și direcții viitoare

8.1 Concluzii generale

Lucrarea a prezentat un sistem complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Au fost analizate fundamentele medicale ale bolii, stadiile de severitate, particularitățile imaginilor retiniene și direcțiile relevante din literatura de specialitate. Pe această bază a fost construit un pipeline experimental în PyTorch, folosind transfer learning și arhitecturi convoluționale moderne.

Metodologia include încărcarea datelor, preprocesarea imaginilor, augmentarea pentru antrenare, construirea modelelor, alegerea funcțiilor de pierdere, antrenarea, validarea, testarea și generarea graficelor de analiză. În plus, a fost realizată o aplicație web de inferență, care demonstrează modul în care modelele salvate pot fi utilizate într-o interfață practică.

8.2 Rezultatele obținute față de obiectivele inițiale

Obiectivele inițiale au fost îndeplinite în mai multe direcții. A fost realizată o analiză a metodelor existente pentru detecția și clasificarea retinopatiei diabetice, au fost studiate seturi de date publice și a fost selectat un dataset echilibrat pentru experimente. De asemenea, a fost implementată o metodă bazată pe rețele convoluționale și transfer learning.

Codul permite testarea mai multor arhitecturi, funcții de pierdere și optimizatori. Au fost generate modele salvate, grafice de evaluare și rapoarte de clasificare. În plus, aplicația de inferență arată că rezultatele experimentale pot fi integrate într-un flux software accesibil, în care utilizatorul încarcă o imagine și primește o predicție.

8.3 Limitările lucrării

Principala limitare este faptul că modelele au fost evaluate pe un set de date public și echilibrat, care nu reflectă complet distribuția clinică reală. În practică, clasele nu sunt uniform distribuite, iar performanța pe un dataset echilibrat nu garantează automat aceeași performanță într-un program real de screening.

O altă limitare este prezența imaginilor augmentate direct în dataset. Acestea ajută la echilibrarea claselor, dar pot introduce exemple foarte asemănătoare. Pentru o validare mai strictă, ar fi utilă o împărțire la nivel de pacient, astfel încât imagini apropiate provenite de la același pacient să nu apară simultan în antrenare și testare.

Lucrarea nu include validare clinică realizată de medici oftalmologi și nu testează sistemul pe imagini provenite din mai multe centre medicale. De asemenea, aplicația de inferență este un prototip tehnic, fără autentificare, audit, stocare securizată a datelor sau certificare medicală.

8.4 Direcții viitoare

O primă direcție viitoare este validarea sistemului pe seturi de date externe, provenite din surse diferite. Această etapă ar permite evaluarea capacității de generalizare la camere, populații și protocoale de achiziție diferite. De asemenea, ar fi utilă testarea pe distribuții de date mai apropiate de practica clinică.

O altă direcție este introducerea unui modul de control al calității imaginii. Înainte de clasificare, sistemul ar putea verifica dacă fotografia este clară, bine iluminată și centrată. Cazurile neconforme ar putea fi respinse sau marcate pentru recapturare.

Pe partea de modelare, clasificarea globală poate fi completată cu detecția sau segmentarea leziunilor. Un sistem care indică nu doar stadiul bolii, ci și regiunile suspecte, ar fi mai ușor de interpretat. Aplicația de inferență poate fi extinsă prin păstrarea modelelor în memorie, calibrarea probabilităților, definirea unor praguri de incertitudine și generarea unor rapoarte exportabile.

Bibliografie

- [1] Varun Gulshan, Lily Peng, Marc Coram, Martin C Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, Kasumi Widner, Tom Madams, Jorge Cuadros, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22):2402–2410, 2016.
- [2] Muhammad Kashif Jabbar, Jianzhuo Yan, Hongxia Xu, Zaka Ur Rehman, and Ayesha Jabbar. Transfer learning-based model for diabetic retinopathy diagnosis using retinal images. *Brain Sciences*, 12(5):535, 2022.
- [3] Xiaofei Wang, Mai Xu, Jicong Zhang, Lai Jiang, and Liu Li. Deep multi-task learning for diabetic retinopathy grading in fundus images. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 2826–2834, 2021.
- [4] Nitigya Sambyal, Poonam Saini, Rupali Syal, and Varun Gupta. Modified u-net architecture for semantic segmentation of diabetic retinopathy images. *Biocybernetics and Biomedical Engineering*, 40(3):1094–1109, 2020.
- [5] Shiqi Huang, Jianan Li, Yuze Xiao, Ning Shen, and Tingfa Xu. Rtnet: relation transformer network for diabetic retinopathy multi-lesion segmentation. *IEEE Transactions on Medical Imaging*, 41(6):1596–1607, 2022.
- [6] Hossein Shakibania, Sina Raoufi, Behnam Pourafkham, Hassan Khotanlou, and Muharram Mansoorizadeh. Dual branch deep learning network for detection and stage grading of diabetic retinopathy. *Biomedical Signal Processing and Control*, 93:106168, 2024.
- [7] Mohamed Chetoui and Moulay A Akhloufi. Explainable end-to-end deep learning for diabetic retinopathy detection across multiple datasets. *Journal of Medical Imaging*, 7(4):044503–044503, 2020.
- [8] Guanghua Zhang, Bin Sun, Zhaoxia Zhang, Jing Pan, Weihua Yang, and Yunfang Liu. Multi-model domain adaptation for diabetic retinopathy classification. *Frontiers in Physiology*, 13:918929, 2022.
- [9] Joes Staal, Michael D Abràmoff, Meindert Niemeijer, Max A Viergever, and Bram Van Ginneken. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4):501–509, 2004.
- [10] Akara Sopharak, Bunyarit Uyyanonvara, and Sarah Barman. Automatic exudate detection from non-dilated diabetic retinopathy retinal images using fuzzy c-means clustering. *sensors*, 9(3):2148–2161, 2009.
- [11] Harry Pratt, Frans Coenen, Deborah M Broadbent, Simon P Harding, and Yalin Zheng. Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90:200–205, 2016.
- [12] American Diabetes Association Professional Practice Committee. 2. diagnosis and classification of diabetes: Standards of care in diabetes–2024. *Diabetes Care*, 47(Supplement_1):S20–S42, 2024.

- [13] Tien Yin Wong, Chui Ming Gemmy Cheung, Michael Larsen, Sobha Sharma, and Rafael Simó. Diabetic retinopathy. *Nature Reviews Disease Primers*, 2:16012, 2016.
- [14] Tao Li, Yingqi Gao, Kai Wang, Song Guo, Hanruo Liu, and Hong Kang. Diagnostic assessment of deep learning algorithms for diabetic retinopathy screening. *Information Sciences*, 501:511–522, 2019.
- [15] Ruhan Liu, Xiangning Wang, Qiang Wu, Ling Dai, Xi Fang, Tao Yan, Jaemin Son, Shiqi Tang, Jiang Li, Zijian Gao, et al. Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge. *Patterns*, 3(6), 2022.
- [16] Luca Giancardo, Fabrice Meriaudeau, Thomas P Karnowski, Yaqin Li, Seema Garg, Kenneth W Tobin Jr, and Edward Chaum. Exudate-based diabetic macular edema detection in fundus images using publicly available datasets. *Medical image analysis*, 16(1):216–226, 2012.
- [17] Meindert Niemeijer, Bram Van Ginneken, Joes Staal, Maria SA Suttorp-Schulten, and Michael D Abràmoff. Automatic detection of red lesions in digital color fundus photographs. *IEEE Transactions on medical imaging*, 24(5):584–592, 2005.
- [18] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [19] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [21] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- [22] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114, 2019.
- [23] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [24] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [25] Kushagra Tandon. Diabetic retinopathy balanced. Kaggle dataset, 2024. Disponibil la: <https://www.kaggle.com/datasets/kushagrandon12/diabetic-retinopathy-balanced>.
- [26] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

- [27] Pallets Projects. Flask documentation, 2026. Disponibil la: <https://flask.palletsprojects.com/>.
- [28] Meta Open Source. React documentation, 2026. Disponibil la: <https://react.dev/>.
- [29] Axios Project. Axios documentation, 2026. Disponibil la: <https://axios-http.com/docs/intro>.
- [30] Flask-CORS Contributors. Flask-cors documentation, 2026. Disponibil la: <https://flask-cors.readthedocs.io/en/latest/>.
- [31] PyTorch Foundation. Pytorch documentation, 2026. Disponibil la: <https://docs.pytorch.org/docs/stable/>.
- [32] three.js Authors. three.js documentation, 2026. Disponibil la: <https://threejs.org/docs/>.

Anexa A

Cod

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torchvision.models as models
5 import torch.optim as optim
6
7 # Importam timm pentru Inception-ResNet-v2
8 try:
9     import timm
10 except ImportError:
11     timm = None
12
13 class InceptionWrapper(nn.Module):
14     """
15     Carcasa de protectie pentru Inception-v3.
16     Lasa modelul sa se incarce nativ, dar returneaza o singura iesire la antrenare,
17     exact cum asteapta functiile noastre de Loss.
18     """
19     def __init__(self, num_classes, pretrained=True):
20         super().__init__()
21         # Lasam Inception sa se initializeze 100% standard (fara erori de aux_logits)
22         weights = models.Inception_V3_Weights.DEFAULT if pretrained else None
23         self.model = models.inception_v3(weights=weights)
24
25         # Modificam ultimul strat
26         in_features = self.model.fc.in_features
27         self.model.fc = nn.Linear(in_features, num_classes)
28
29     def forward(self, x):
30         outputs = self.model(x)
31         # La antrenare, Inception returneaza un obiect cu 2 iesiri. Noi o dam mai departe doar
32         # pe prima.
33         if self.training:
34             return outputs[0] # outputs[0] reprezinta predictia principala (logits)
35         # La validare, returneaza oricum o singura iesire
36         return outputs
37
38 # =====
39 # 1. Implementare focal loss pentru clasificare multi-clasa
40 # =====
41 class FocalLossMultiClass(nn.Module):
42     """
43     Focal Loss adaptat pentru clasificare multi-clasa.
44     Accepta predictii de forma [Batch, Clase] si etichete standard (indici) de forma [Batch].
45     """
46     def __init__(self, alpha=None, gamma=1.5, reduction='mean'):
47         super(FocalLossMultiClass, self).__init__()
48         self.gamma = gamma
49         self.alpha = alpha
50         self.reduction = reduction
51
52     def forward(self, inputs, targets):
53         # Calculam direct Cross Entropy (care stie sa citeasca indicii claselor)
54         ce_loss = F.cross_entropy(inputs, targets, weight=self.alpha, reduction='none')
55         pt = torch.exp(-ce_loss)
56         focal_loss = ((1 - pt) ** self.gamma) * ce_loss
57
58         if self.reduction == 'mean':
59             return focal_loss.mean()
60         elif self.reduction == 'sum':
61             return focal_loss.sum()
62         else:

```



```

63         return focal_loss
64
65 # =====
66 # 2. FABRICA DE MODELE
67 # =====
68 def get_model(model_name, num_classes=5, pretrained=True, drop_rate=0.0):
69     """
70     Fabrica de modele. Toate au strat final modificat pentru num_classes.
71     """
72     model_name = model_name.lower()
73
74     if model_name == 'resnet50':
75         weights = models.ResNet50_Weights.DEFAULT if pretrained else None
76         model = models.resnet50(weights=weights)
77         num_fts = model.fc.in_features
78         model.fc = nn.Sequential(
79             nn.Dropout(0.5),
80             nn.Linear(num_fts, num_classes)
81         )
82
83     elif model_name == 'efficientnet_b3': # TODO adauga experiment cu asta, daca timpul
permite
84         weights = models.EfficientNet_B3_Weights.DEFAULT if pretrained else None
85         model = models.efficientnet_b3(weights=weights)
86         in_features = model.classifier[1].in_features
87         model.classifier[1] = nn.Sequential(
88             nn.Dropout(0.5),
89             nn.Linear(in_features, num_classes)
90         )
91
92     elif model_name == 'inception_v3':
93         model = InceptionWrapper(num_classes=num_classes, pretrained=pretrained)
94
95     elif model_name == 'incres_v2':
96         if timm is None:
97             raise ImportError(
98                 "Modelul incres_v2 necesita pachetul 'timm'. Instaleaza-l cu: pip install timm
99             )
100         model = timm.create_model(
101             'inception_resnet_v2',
102             pretrained=pretrained,
103             num_classes=num_classes,
104             drop_rate=drop_rate,
105         )
106
107     else:
108         raise ValueError(f"Modelul {model_name} nu este suportat!")
109
110     return model
111
112 # =====
113 # 3. FABRICA DE LOSS-URI SI PREDICTII (Totul se intampla aici)
114 # =====
115 def get_loss_function(loss_name, class_weights=None, device='cuda', gamma=1.5):
116     loss_name = loss_name.lower()
117
118     # -----
119     # REPARAT: Transformam lista venita din consola intr-un Tensor
120     # -----
121     if class_weights is not None and isinstance(class_weights, list):
122         class_weights = torch.tensor(class_weights, dtype=torch.float)
123
124     if loss_name in ['bce_ordinal', 'ordinal']:
125         return nn.BCEWithLogitsLoss()
126
127     elif loss_name == 'focal_loss':
128         # Trimitem ponderile pe GPU (daca exista) pentru Focal Loss
129         alpha_tensor = class_weights.to(device) if class_weights is not None else None
130         return FocalLossMultiClass(alpha=alpha_tensor, gamma=gamma)
131
132     elif loss_name == 'ce':
133         return nn.CrossEntropyLoss()
134
135     elif loss_name == 'weighted_ce':
136         if class_weights is None:

```

```

137         raise ValueError("Pentru Weighted CE, trebuie sa trimiti '--class_weights'!")
138         # Acum class_weights este oficial un Tensor, deci .to(device) va functiona perfect!
139         return nn.CrossEntropyLoss(weight=class_weights.to(device))
140
141     else:
142         raise ValueError(f"Loss-ul '{loss_name}' nu este suportat.")
143
144
145 def compute_loss(criterion, outputs, labels, loss_name, device):
146     """
147     Calculeaza valoarea erorii. Gestioneaza automat transformările necesare
148     pentru etichete (ex. transformarea in vectori ordinali).
149     """
150     loss_name = loss_name.lower()
151
152     if loss_name in ['bce_ordinal', 'ordinal']:
153         # Ordinal Regression necesita transformarea [2] -> [1, 1, 1, 0, 0]
154         levels = torch.arange(5).to(device)
155         labels_ordinal = (labels.unsqueeze(1) >= levels).float()
156         return criterion(outputs, labels_ordinal)
157
158     else:
159         # Pentru CE, Weighted CE si noul Focal Loss, putem da etichetele exact asa cum vin (
160         # indici 0-4)
161         return criterion(outputs, labels)
162
163 def get_predictions(outputs, loss_name):
164     """
165     Transforma iesirea modelului (tensor de probabilitati/logits) intr-o eticheta finala (0,
166     1, 2, 3, 4).
167     """
168     loss_name = loss_name.lower()
169
170     if loss_name in ['bce_ordinal', 'ordinal']:
171         # Adunam valorile care trec pragul de 0.0 (pragul implicit dupa BCEWithLogits)
172         preds = (outputs > 0.0).sum(dim=1) - 1
173         return torch.clamp(preds, min=0, max=4)
174
175     else:
176         # Pentru CE, Weighted CE si Focal Loss, extragem direct valoarea maxima
177         return torch.argmax(outputs, dim=1)
178
179 HEAD_PARAM_KEYWORDS = (
180     'fc',
181     'classifier',
182     'classif',
183     'head',
184     'last_linear',
185     'logits',
186 )
187
188 def is_head_parameter(name):
189     lowered = name.lower()
190     return any(keyword in lowered for keyword in HEAD_PARAM_KEYWORDS)
191
192
193 def set_backbone_trainable(model, trainable):
194     for name, param in model.named_parameters():
195         if not is_head_parameter(name):
196             param.requires_grad = trainable
197
198
199 def count_trainable_parameters(model):
200     return sum(param.numel() for param in model.parameters() if param.requires_grad)
201
202
203 def get_optimizer_params(model, lr, backbone_lr_mult=1.0):
204     if backbone_lr_mult == 1.0:
205         return model.parameters()
206
207     head_params = []
208     backbone_params = []
209     for name, param in model.named_parameters():
210         if is_head_parameter(name):

```

```

211         head_params.append(param)
212     else:
213         backbone_params.append(param)
214
215     return [
216         {'params': backbone_params, 'lr': lr * backbone_lr_mult},
217         {'params': head_params, 'lr': lr},
218     ]
219
220
221 def format_lrs(optimizer):
222     return ', '.join([f"{group['lr']:.8f}" for group in optimizer.param_groups])
223
224
225 # =====
226 # 4. OPTIMIZATORII
227 # =====
228 def get_optimizer(model, optimizer_name='adam', lr=1e-4, backbone_lr_mult=1.0):
229     optimizer_name = optimizer_name.lower()
230     params = get_optimizer_params(model, lr, backbone_lr_mult=backbone_lr_mult)
231
232     if optimizer_name == 'adam':
233         return optim.Adam(params, lr=lr, weight_decay=1e-4)
234     elif optimizer_name == 'adamw':
235         return optim.AdamW(params, lr=lr, weight_decay=1e-4)
236     elif optimizer_name == 'sgd':
237         return optim.SGD(params, lr=lr, momentum=0.9, weight_decay=1e-5)
238     else:
239         raise ValueError(f"Optimizatorul '{optimizer_name}' nu este suportat.")

```

Listing A.1: builder.py – construirea modelelor, funcțiilor de pierdere și optimizatorilor

```

1 import os
2
3 import torch
4 from PIL import Image
5 from torch.utils.data import ConcatDataset, Dataset
6 import torchvision.transforms as T
7
8
9 IMAGE_EXTENSIONS = ('.png', '.jpg', '.jpeg')
10
11 EXPERIMENTS = {
12     'balanced_to_balanced': ('balanced', 'balanced'),
13     'balanced_apos_to_balanced_apos': ('combined', 'combined'),
14     'balanced_apos_to_apos': ('combined', 'apost'),
15     'balanced_apos_to_balanced': ('combined', 'balanced'),
16     'apost_to_balanced': ('apost', 'balanced'),
17     'apost_to_balanced_apos': ('apost', 'combined'),
18     'balanced_to_apost': ('balanced', 'apost'),
19     'balanced_aug_to_balanced_aug': ('balanced_aug', 'balanced_aug'),
20     'balanced_aug_apos_to_balanced_aug_apos': ('balanced_aug_apos', 'balanced_aug_apos'),
21     'balanced_aug_apos_to_apos': ('balanced_aug_apos', 'apost'),
22     'balanced_aug_apos_to_balanced_aug': ('balanced_aug_apos', 'balanced_aug'),
23     'my_dataset_to_my_dataset': ('my_dataset', 'my_dataset'),
24     'balanced_my_dataset_to_balanced_my_dataset': ('balanced_my_dataset', 'balanced_my_dataset'),
25     'balanced_my_dataset_to_my_dataset': ('balanced_my_dataset', 'my_dataset'),
26     'balanced_my_dataset_to_balanced': ('balanced_my_dataset', 'balanced'),
27     'my_dataset_to_balanced': ('my_dataset', 'balanced'),
28     'balanced_to_my_dataset': ('balanced', 'my_dataset'),
29     'apost_new_to_apost_new': ('apost_new', 'apost_new'),
30     'apost_new_to_balanced': ('apost_new', 'balanced'),
31     'balanced_to_apost_new': ('balanced', 'apost_new'),
32     'balanced_apost_new_to_balanced_apost_new': ('balanced+apost_new', 'balanced+apost_new'),
33     'balanced_apost_new_to_apost_new': ('balanced+apost_new', 'apost_new'),
34     'balanced_aug_final_to_balanced_aug_final': ('balanced_aug_final', 'balanced_aug_final'),
35     'balanced_aug_final_to_balanced': ('balanced_aug_final', 'balanced'),
36     'balanced_to_balanced_aug_final': ('balanced', 'balanced_aug_final'),
37     'balanced_data_original_to_balanced_data_original': ('balanced_data_original', 'balanced_data_original'),
38     'balanced_data_augmented_to_balanced_data_augmented': ('balanced_data_augmented', 'balanced_data_augmented'),

```

```

39     'balanced_data_original_augmented_to_balanced': ('balanced_data_original+
balanced_data_augmented', 'balanced'),
40     'balanced_aug_resized_to_balanced_aug_resized': ('balanced_aug_resized', '
balanced_aug_resized'),
41     'balanced_aug_resized_cropped_to_balanced_aug_resized_cropped': ('
balanced_aug_resized_cropped', 'balanced_aug_resized_cropped'),
42     'balanced_aug_clean_split_to_balanced_aug_clean_split': ('balanced_aug_clean_split', '
balanced_aug_clean_split'),
43     'my_dataset_balanced_aug_final_to_my_dataset': ('my_dataset+balanced_aug_final', '
my_dataset'),
44     'my_dataset_balanced_aug_final_to_balanced': ('my_dataset+balanced_aug_final', 'balanced')
45 }
46
47 DATA_SOURCES = [
48     'balanced',
49     'balanced_aug',
50     'balanced_aug_final',
51     'balanced_aug_resized',
52     'balanced_aug_resized_cropped',
53     'balanced_aug_clean_split',
54     'balanced_data_original',
55     'balanced_data_augmented',
56     'aptos',
57     'aptos_new',
58     'aptos_part_2',
59     'aptos_part_4',
60     'my_dataset',
61     'combined',
62     'balanced_aptos',
63     'balanced_aug_aptos',
64     'balanced_my_dataset',
65     'diabetic_balanced_data+aptos',
66     'diabetic_balanced_aug+aptos',
67     'diabetic_balanced_data+my_dataset',
68     'balanced+aptos_new',
69     'balanced_data_original+balanced_data_augmented',
70     'my_dataset+balanced_aug_final',
71 ]
72
73 SOURCE_ALIASES = {
74     'combined': 'balanced+aptos',
75     'balanced_aptos': 'balanced+aptos',
76     'balanced_aug_aptos': 'balanced_aug+aptos',
77     'balanced_my_dataset': 'balanced+my_dataset',
78     'diabetic_balanced_data+aptos': 'balanced+aptos',
79     'diabetic_balanced_aug+aptos': 'balanced_aug+aptos',
80     'diabetic_balanced_data+my_dataset': 'balanced+my_dataset',
81     'aptos_part_2': 'aptos',
82 }
83
84
85 def default_transform(split='train', image_size=224, train_augment='basic'):
86     split = split.lower()
87     train_augment = train_augment.lower()
88
89     if split == 'train' and train_augment == 'basic':
90         return T.Compose([
91             T.Resize((image_size, image_size)),
92             T.RandomHorizontalFlip(p=0.5),
93             T.RandomVerticalFlip(p=0.5),
94             T.RandomRotation(degrees=15),
95             T.ColorJitter(brightness=0.1, contrast=0.1),
96             T.ToTensor(),
97             T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
98         ])
99
100     return T.Compose([
101         T.Resize((image_size, image_size)),
102         T.ToTensor(),
103         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
104     ])
105
106
107 class BalancedDRDataset(Dataset):

```

```

108 def __init__(self, root_dir, split='train', image_size=224, transform=None, train_augment=
    'basic'):
109     self.root_dir = root_dir
110     self.split = split.lower()
111     self.image_size = image_size
112     self.split_dir = os.path.join(self.root_dir, self.split)
113
114     if not os.path.exists(self.split_dir):
115         raise FileNotFoundError(f"[Eroare] Folderul nu exista: {self.split_dir}")
116
117     self.image_paths = []
118     self.labels = []
119
120     for class_id in range(5):
121         class_folder = os.path.join(self.split_dir, str(class_id))
122         if not os.path.exists(class_folder):
123             continue
124         for img_name in os.listdir(class_folder):
125             if img_name.lower().endswith(IMAGE_EXTENSIONS):
126                 self.image_paths.append(os.path.join(class_folder, img_name))
127                 self.labels.append(class_id)
128
129     self.transform = transform or default_transform(self.split, self.image_size,
    train_augment=train_augment)
130
131 def __len__(self):
132     return len(self.image_paths)
133
134 def __getitem__(self, idx):
135     img_path = self.image_paths[idx]
136     label = self.labels[idx]
137
138     try:
139         image = Image.open(img_path).convert('RGB')
140     except Exception:
141         image = Image.new('RGB', (self.image_size, self.image_size), (0, 0, 0))
142
143     if self.transform:
144         image = self.transform(image)
145
146     return image, torch.tensor(label, dtype=torch.long)
147
148
149 class AptosDRDataset(BalancedDRDataset):
150     """APTOS Ben Graham este deja preprocesat si organizat pe foldere train/val/test/0..4."""
151     pass
152
153
154 def make_dataset(
155     source,
156     split,
157     image_size,
158     balanced_root,
159     aptos_root,
160     balanced_aug_root=None,
161     my_dataset_root=None,
162     dataset_roots=None,
163     train_augment='basic',
164 ):
165     source = source.lower()
166     split = split.lower()
167     source = SOURCE_ALIASES.get(source, source)
168
169     roots = {
170         'balanced': balanced_root,
171         'balanced_aug': balanced_aug_root,
172         'balanced_aug_final': balanced_aug_root,
173         'aptos': aptos_root,
174         'my_dataset': my_dataset_root,
175     }
176     if dataset_roots:
177         roots.update({key.lower(): value for key, value in dataset_roots.items() if value is
    not None})
178
179     if '+' in source:
180         return ConcatDataset([

```

```

181         make_dataset(
182             part,
183             split=split,
184             image_size=image_size,
185             balanced_root=balanced_root,
186             aptos_root=aptos_root,
187             balanced_aug_root=balanced_aug_root,
188             my_dataset_root=my_dataset_root,
189             dataset_roots=roots,
190             train_augment=train_augment,
191         )
192         for part in source.split('+')
193     ])
194
195     if source == 'balanced':
196         return BalancedDRDataset(balanced_root, split=split, image_size=image_size,
197                                   train_augment=train_augment)
198
199     if source == 'balanced_aug':
200         if balanced_aug_root is None:
201             raise ValueError("balanced_aug_root trebuie setat pentru sursa 'balanced_aug'.")
202         return BalancedDRDataset(balanced_aug_root, split=split, image_size=image_size,
203                                   train_augment=train_augment)
204
205     if source == 'aptos':
206         return AptosDRDataset(aptos_root, split=split, image_size=image_size, train_augment=
207                                   train_augment)
208
209     if source == 'my_dataset':
210         if my_dataset_root is None:
211             raise ValueError("my_dataset_root trebuie setat pentru sursa 'my_dataset'.")
212         return BalancedDRDataset(my_dataset_root, split=split, image_size=image_size,
213                                   train_augment=train_augment)
214
215     if source in roots:
216         if roots[source] is None:
217             raise ValueError(f"Root-ul trebuie setat pentru sursa '{source}'.")
218         dataset_cls = AptosDRDataset if source.startswith('aptos') else BalancedDRDataset
219         return dataset_cls(roots[source], split=split, image_size=image_size, train_augment=
220                                   train_augment)
221
222     raise ValueError(f"Sursa de date necunoscuta: {source}")
223
224 def resolve_experiment(experiment=None, train_source=None, test_source=None):
225     if experiment:
226         experiment = experiment.lower()
227         if experiment not in EXPERIMENTS:
228             valid = ', '.join(EXPERIMENTS.keys())
229             raise ValueError(f"Experiment necunoscut: {experiment}. Variante: {valid}")
230         return EXPERIMENTS[experiment]
231
232     return (train_source or 'balanced').lower(), (test_source or 'balanced').lower()

```

Listing A.2: my_dataset.py – încărcarea setului de date echilibrat

i generarea graficelor,i generarea graficelor,

```

1 import argparse
2 import importlib
3 import os
4 import sys
5 import time
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import seaborn as sns
10 import torch
11 import torch.nn.functional as F
12 from sklearn.metrics import (
13     auc,
14     balanced_accuracy_score,
15     classification_report,
16     cohen_kappa_score,

```

```

17     confusion_matrix,
18     f1_score,
19     precision_recall_fscore_support,
20     roc_auc_score,
21     roc_curve,
22 )
23 from sklearn.preprocessing import label_binarize
24 from torch.utils.data import DataLoader
25 from tqdm import tqdm
26
27
28 try:
29     CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
30 except NameError:
31     CURRENT_DIR = os.getcwd()
32
33 PARENT_DIR = os.path.abspath(os.path.join(CURRENT_DIR, '..'))
34 PROJECT_ROOT = PARENT_DIR
35
36 if PARENT_DIR not in sys.path:
37     sys.path.insert(0, PARENT_DIR)
38
39 from builder import (
40     compute_loss,
41     count_trainable_parameters,
42     format_lrs,
43     get_loss_function,
44     get_model,
45     get_optimizer,
46     get_predictions,
47     set_backbone_trainable,
48 )
49 import my_dataset as my_dataset_module
50
51 my_dataset_module = importlib.reload(my_dataset_module)
52 EXPERIMENTS = my_dataset_module.EXPERIMENTS
53 DATA_SOURCES = my_dataset_module.DATA_SOURCES
54 make_dataset = my_dataset_module.make_dataset
55 resolve_experiment = my_dataset_module.resolve_experiment
56
57
58 CLASS_NAMES = ['0', '1', '2', '3', '4']
59 IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
60 IMAGENET_STD = np.array([0.229, 0.224, 0.225])
61
62 RESULT_SUBDIRS = {
63     'history': 'istoric',
64     'loss': 'loss',
65     'accuracy': 'acuratete',
66     'auc': 'auc',
67     'qwk': 'qwk',
68     'macro_f1': 'macro_f1',
69     'balanced_acc': 'balanced_accuracy',
70     'confusion': 'matrici_confuzie',
71     'classification_reports': 'rapoarte_clasificare',
72     'precision_recall_f1': 'precision_recall_f1',
73     'roc': 'roc',
74     'gradcam': 'gradcam',
75     'errors': 'erori',
76     'other': 'altele',
77 }
78
79
80 def parse_args():
81     parser = argparse.ArgumentParser(description="Antrenament DR - experimente Balanced/APTOS")
82     parser.add_argument('--model', type=str, default='resnet50', help='ex: resnet50, efficientnet_b3, inception_v3')
83     parser.add_argument('--loss_name', type=str, default='ce', help='ex: ce, bce_ordinal, focal_loss, weighted_ce')
84     parser.add_argument('--optimizer', type=str, default='adam', help='Variante: adam, adamw, sgd')
85     parser.add_argument('--epochs', type=int, default=20)
86     parser.add_argument('--batch_size', type=int, default=32)
87     parser.add_argument('--lr', type=float, default=0.0001)
88     parser.add_argument('--img_size', type=int, default=224)

```

```

89 parser.add_argument('--num_workers', type=int, default=0)
90 parser.add_argument('--class_weights', type=float, nargs='+', default=None)
91 parser.add_argument('--gamma', type=float, default=1.5, help='Valoarea gamma pentru Focal
Loss (daca este selectat).')
92 parser.add_argument(
93     '--monitor_metric',
94     type=str,
95     default='qwk',
96     choices=['qwk', 'macro_f1', 'balanced_acc', 'auc', 'acc', 'val_loss'],
97     help='Metrica folosita pentru checkpoint si early stopping.',
98 )
99 parser.add_argument(
100     '--scheduler',
101     type=str,
102     default='plateau',
103     choices=['plateau', 'cosine', 'none'],
104     help='Scheduler LR. plateau pastreaza comportamentul vechi; cosine e util pentru fine-
tuning.',
105 )
106 parser.add_argument('--amp', action='store_true', help='Activeaza mixed precision pe CUDA.
')
107 parser.add_argument('--grad_clip', type=float, default=0.0, help='Clip gradient norm. 0
dezactiveaza.')
108 parser.add_argument('--ema_decay', type=float, default=0.0, help='EMA pentru greutati. Ex:
0.999. 0 dezactiveaza.')
109 parser.add_argument(
110     '--model_dropout',
111     type=float,
112     default=0.0,
113     help='Dropout intern pentru modele timm, util pentru incres_v2. 0 pastreaza
comportamentul implicit.',
114 )
115 parser.add_argument(
116     '--freeze_backbone_epochs',
117     type=int,
118     default=0,
119     help='Antreneaza doar head-ul in primele N epoci, apoi deblocheaza backbone-ul.',
120 )
121 parser.add_argument(
122     '--backbone_lr_mult',
123     type=float,
124     default=1.0,
125     help='Multiplcator LR pentru backbone. Ex: 0.1 inseamna backbone LR = lr * 0.1.',
126 )
127 parser.add_argument(
128     '--tta_eval',
129     action='store_true',
130     help='Activeaza TTA la validare/test pentru multiclass: original + horizontal flip.',
131 )
132 parser.add_argument(
133     '--train_augment',
134     type=str,
135     default='basic',
136     choices=['basic', 'none'],
137     help='Augmentare online pentru split-ul train. basic pastreaza comportamentul vechi;
none aplica doar resize/tensor/normalizare.',
138 )
139 parser.add_argument(
140     '--early_stopping_patience',
141     type=int,
142     default=0,
143     help='Opreste antrenarea dupa N epoci fara imbunatatire. 0 dezactiveaza early stopping
.',
144 )
145 parser.add_argument(
146     '--early_stopping_min_delta',
147     type=float,
148     default=0.0,
149     help='Imbunatatirea minima a metricii monitorizate necesara pentru resetarea patience.
',
150 )
151
152 parser.add_argument(
153     '--experiment',
154     type=str,
155     default=None,

```



```

156     choices=list(EXPERIMENTS.keys()),
157     help='Alege direct combinatia train/test. Ex: balanced_aptos_to_aptos',
158 )
159 parser.add_argument(
160     '--train_source',
161     type=str,
162     default='balanced',
163     choices=DATA_SOURCES,
164     help='Sursa pentru train/val daca nu folosesti --experiment.',
165 )
166 parser.add_argument(
167     '--test_source',
168     type=str,
169     default='balanced',
170     choices=DATA_SOURCES,
171     help='Sursa pentru test daca nu folosesti --experiment.',
172 )
173 parser.add_argument(
174     '--test_sources',
175     type=str,
176     nargs='+',
177     default=None,
178     choices=DATA_SOURCES,
179     help='Optional: ruleaza testarea finala pe mai multe surse, in aceeasi antrenare. Ex:
--test_sources aptos balanced',
180 )
181 parser.add_argument(
182     '--root_dir',
183     type=str,
184     default=None,
185     help='Compatibilitate veche: radacina pentru Diabetic_Balanced_Data.',
186 )
187 parser.add_argument('--balanced_root', type=str, default=None)
188 parser.add_argument(
189     '--balanced_aug_root',
190     type=str,
191     default=None,
192     help='Radacina pentru datasetul balanced_aug final, cu structura train/val/test/0..4.'
193 )
194 parser.add_argument(
195     '--aptos_root',
196     type=str,
197     default=None,
198     help='Radacina APTOS default, cu structura train/val/test/0..4.',
199 )
200 parser.add_argument(
201     '--my_dataset_root',
202     type=str,
203     default=None,
204     help='Radacina pentru datasets/my_dataset, cu structura train/val/test/0..4.',
205 )
206 parser.add_argument('--aptos_new_root', type=str, default=None)
207 parser.add_argument('--aptos_part_4_root', type=str, default=None)
208 parser.add_argument('--balanced_aug_resized_root', type=str, default=None)
209 parser.add_argument('--balanced_aug_resized_cropped_root', type=str, default=None)
210 parser.add_argument('--balanced_aug_clean_split_root', type=str, default=None)
211 parser.add_argument('--balanced_data_original_root', type=str, default=None)
212 parser.add_argument('--balanced_data_augmented_root', type=str, default=None)
213 parser.add_argument(
214     '--train_dataset_root',
215     type=str,
216     default=None,
217     help='Path custom pentru datasetul de antrenare/validare. Trebuie sa contina train
/0..4 si val/0..4.',
218 )
219 parser.add_argument(
220     '--train_dataset_roots',
221     type=str,
222     nargs='+',
223     default=None,
224     help='Doua sau mai multe path-uri custom pentru train/val; dataseturile vor fi
concatenate.',
225 )
226 parser.add_argument(
227     '--train_dataset_names',

```

```

228         type=str,
229         nargs='+',
230         default=None,
231         help='Optional: nume pentru dataseturile din --train_dataset_roots, in aceeasi ordine.
',
232     )
233     parser.add_argument(
234         '--test_dataset_roots',
235         type=str,
236         nargs='+',
237         default=None,
238         help='Unul sau mai multe path-uri custom pentru testare. Fiecare trebuie sa contina
test/0..4.',
239     )
240     parser.add_argument(
241         '--test_dataset_names',
242         type=str,
243         nargs='+',
244         default=None,
245         help='Optional: nume pentru dataseturile din --test_dataset_roots, in aceeasi ordine.'
246     )
247     return parser.parse_args()
248
249
250 def resolve_dataset_path(path):
251     if path is None:
252         return None
253     path = os.path.expanduser(path)
254     if os.path.isabs(path):
255         return os.path.abspath(path)
256     return os.path.abspath(os.path.join(PROJECT_ROOT, path))
257
258
259 def safe_source_name(name):
260     return name.replace(' ', '_').replace('/', '_').replace('\\', '_').lower()
261
262
263 def make_results_dirs(grafice_dir):
264     dirs = {}
265     os.makedirs(grafice_dir, exist_ok=True)
266     for key, folder_name in RESULT_SUBDIRS.items():
267         path = os.path.join(grafice_dir, folder_name)
268         os.makedirs(path, exist_ok=True)
269         dirs[key] = path
270     return dirs
271
272 # grafice si rapoarte pentru analiza post-antrenare, care vor fi salvate in subfolderele din
results_dirs
273 def save_combined_history(history, path, show=False):
274     fig, axes = plt.subplots(2, 3, figsize=(18, 10))
275     axes = axes.ravel()
276
277     axes[0].plot(history['train_loss'], label='Train Loss')
278     axes[0].plot(history['val_loss'], label='Val Loss')
279     axes[0].set_title('Istoric Loss')
280     axes[0].set_xlabel('Epoca')
281     axes[0].set_ylabel('Loss')
282     axes[0].legend()
283
284     axes[1].plot(history['val_acc'], label='Val Accuracy', color='orange')
285     axes[1].set_title('Istoric Acuratete (Val)')
286     axes[1].set_xlabel('Epoca')
287     axes[1].set_ylabel('Acuratete')
288     axes[1].legend()
289
290     axes[2].plot(history['val_auc'], label='Val AUC', color='green')
291     axes[2].set_title('Istoric AUC (Val)')
292     axes[2].set_xlabel('Epoca')
293     axes[2].set_ylabel('AUC')
294     axes[2].legend()
295
296     axes[3].plot(history['val_qwk'], label='Val QWK', color='purple')
297     axes[3].set_title('Istoric QWK (Val)')
298     axes[3].set_xlabel('Epoca')
299     axes[3].set_ylabel('QWK')

```

```

300     axes[3].legend()
301
302     axes[4].plot(history['val_macro_f1'], label='Val Macro F1', color='red')
303     axes[4].set_title('Istoric Macro F1 (Val)')
304     axes[4].set_xlabel('Epoca')
305     axes[4].set_ylabel('Macro F1')
306     axes[4].legend()
307
308     axes[5].plot(history['val_balanced_acc'], label='Val Balanced Acc', color='brown')
309     axes[5].set_title('Istoric Balanced Accuracy (Val)')
310     axes[5].set_xlabel('Epoca')
311     axes[5].set_ylabel('Balanced Accuracy')
312     axes[5].legend()
313
314     plt.savefig(path, bbox_inches='tight')
315     if show:
316         plt.show()
317     plt.close()
318
319
320 def save_line_plot(values, title, ylabel, path, color=None, label=None, show=False):
321     plt.figure(figsize=(8, 5))
322     plt.plot(values, color=color, label=label or ylabel)
323     plt.title(title)
324     plt.xlabel('Epoca')
325     plt.ylabel(ylabel)
326     plt.legend()
327     plt.grid(alpha=0.25)
328     plt.savefig(path, bbox_inches='tight')
329     if show:
330         plt.show()
331     plt.close()
332
333
334 def save_confusion_matrix(labels, preds, path, normalized=False, show=False):
335     if normalized:
336         cm = confusion_matrix(labels, preds, labels=list(range(5)), normalize='true')
337         fmt = '.2f'
338         title = 'Matrice Confuzie Normalizata'
339     else:
340         cm = confusion_matrix(labels, preds, labels=list(range(5)))
341         fmt = 'd'
342         title = 'Matrice Confuzie'
343
344     plt.figure(figsize=(8, 6))
345     sns.heatmap(
346         cm,
347         annot=True,
348         fmt=fmt,
349         cmap='Blues',
350         xticklabels=CLASS_NAMES,
351         yticklabels=CLASS_NAMES,
352         vmin=0 if normalized else None,
353         vmax=1 if normalized else None,
354     )
355     plt.xlabel('Predictie (Model)')
356     plt.ylabel('Realitate (Adevar)')
357     plt.title(title)
358     plt.savefig(path, bbox_inches='tight')
359     if show:
360         plt.show()
361     plt.close()
362
363
364 def save_precision_recall_f1(labels, preds, base_name, results_dirs, show=False):
365     precision, recall, f1, support = precision_recall_fscore_support(
366         labels,
367         preds,
368         labels=list(range(5)),
369         zero_division=0,
370     )
371
372     x = np.arange(len(CLASS_NAMES))
373     width = 0.25
374
375     plt.figure(figsize=(10, 6))

```

```

376 plt.bar(x - width, precision, width, label='Precision')
377 plt.bar(x, recall, width, label='Recall')
378 plt.bar(x + width, f1, width, label='F1')
379 plt.xticks(x, CLASS_NAMES)
380 plt.ylim(0, 1)
381 plt.xlabel('Clasa')
382 plt.ylabel('Scor')
383 plt.title('Precision / Recall / F1 pe clase')
384 plt.legend()
385 plt.grid(axis='y', alpha=0.25)
386
387 prf1_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.png")
388 plt.savefig(prf1_path, bbox_inches='tight')
389 if show:
390     plt.show()
391 plt.close()
392
393 report = classification_report(
394     labels,
395     preds,
396     labels=list(range(5)),
397     target_names=CLASS_NAMES,
398     zero_division=0,
399 )
400 report_path = os.path.join(results_dirs['classification_reports'], f"
classification_report_{base_name}.txt")
401 with open(report_path, 'w', encoding='utf-8') as report_file:
402     report_file.write(report)
403
404 metrics_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.csv")
405 with open(metrics_path, 'w', encoding='utf-8') as metrics_file:
406     metrics_file.write('class,precision,recall,f1,support\n')
407     for class_name, p, r, f, s in zip(CLASS_NAMES, precision, recall, f1, support):
408         metrics_file.write(f'{class_name},{p:.6f},{r:.6f},{f:.6f},{int(s)}\n')
409
410 return prf1_path, report_path, metrics_path, report
411
412
413 def save_roc_curve(labels, probs, base_name, results_dirs, show=False):
414     labels = np.asarray(labels)
415     probs = np.asarray(probs)
416     y_true = label_binarize(labels, classes=list(range(5)))
417
418     plt.figure(figsize=(8, 6))
419     plotted_any = False
420
421     for class_idx, class_name in enumerate(CLASS_NAMES):
422         if y_true[:, class_idx].sum() == 0:
423             continue
424         if y_true[:, class_idx].sum() == len(y_true):
425             continue
426
427         fpr, tpr, _ = roc_curve(y_true[:, class_idx], probs[:, class_idx])
428         class_auc = auc(fpr, tpr)
429         plt.plot(fpr, tpr, label=f'Clasa {class_name} (AUC={class_auc:.3f})')
430         plotted_any = True
431
432     if plotted_any:
433         try:
434             fpr_micro, tpr_micro, _ = roc_curve(y_true.ravel(), probs.ravel())
435             micro_auc = auc(fpr_micro, tpr_micro)
436             plt.plot(fpr_micro, tpr_micro, linestyle='--', color='black', label=f'Micro (AUC={
micro_auc:.3f})')
437         except Exception:
438             pass
439
440     plt.plot([0, 1], [0, 1], linestyle=':', color='gray')
441     plt.xlabel('False Positive Rate')
442     plt.ylabel('True Positive Rate')
443     plt.title('Curbe ROC multiclasa')
444     plt.legend(loc='lower right')
445     plt.grid(alpha=0.25)
446
447     roc_path = os.path.join(results_dirs['roc'], f"roc_{base_name}.png")
448     plt.savefig(roc_path, bbox_inches='tight')
449     if show:

```

```

450         plt.show()
451     plt.close()
452     return roc_path
453
454
455 def find_last_conv_layer(model):
456     last_name, last_module = None, None
457     for name, module in model.named_modules():
458         if isinstance(module, torch.nn.Conv2d):
459             last_name, last_module = name, module
460
461     if last_module is None:
462         raise RuntimeError("Nu am gasit niciun strat Conv2d pentru Grad-CAM.")
463
464     return last_name, last_module
465
466
467 def denormalize_image(tensor):
468     image = tensor.detach().cpu().permute(1, 2, 0).numpy()
469     image = (image * IMAGENET_STD) + IMAGENET_MEAN
470     return np.clip(image, 0, 1)
471
472
473 def save_gradcam_overlay(model, data_loader, loss_name, device, base_name, results_dirs, show=
False):
474     try:
475         layer_name, target_layer = find_last_conv_layer(model)
476     except RuntimeError as error:
477         print(f"Grad-CAM sarit: {error}")
478         return None
479
480     activations = []
481     gradients = []
482
483     def forward_hook(_, __, output):
484         activations.append(output.detach())
485
486     def backward_hook(_, grad_input, grad_output):
487         gradients.append(grad_output[0].detach())
488
489     forward_handle = target_layer.register_forward_hook(forward_hook)
490     backward_handle = target_layer.register_full_backward_hook(backward_hook)
491
492     model.eval()
493     try:
494         imgs, labels = next(iter(data_loader))
495         img = imgs[:1].to(device)
496         label = labels[0].item()
497
498         model.zero_grad(set_to_none=True)
499         outputs = model(img)
500         preds = get_predictions(outputs, loss_name)
501         pred = preds[0].item()
502         target_class = pred
503         score = outputs[0, target_class]
504         score.backward()
505
506         activation = activations[-1][0]
507         gradient = gradients[-1][0]
508         weights = gradient.mean(dim=(1, 2), keepdim=True)
509         cam = torch.sum(weights * activation, dim=0)
510         cam = F.relu(cam)
511         cam = cam - cam.min()
512         cam = cam / (cam.max() + 1e-8)
513         cam = F.interpolate(
514             cam.unsqueeze(0).unsqueeze(0),
515             size=img.shape[-2:],
516             mode='bilinear',
517             align_corners=False,
518         ).squeeze().cpu().numpy()
519
520         image = denormalize_image(imgs[0])
521         heatmap = plt.cm.jet(cam)[..., :3]
522         overlay = np.clip((0.55 * image) + (0.45 * heatmap), 0, 1)
523
524         gradcam_path = os.path.join(results_dirs['gradcam'], f"gradcam_{base_name}.png")

```

```

525     plt.figure(figsize=(10, 4))
526     plt.subplot(1, 2, 1)
527     plt.imshow(image)
528     plt.title(f'Original | Real: {label}')
529     plt.axis('off')
530
531     plt.subplot(1, 2, 2)
532     plt.imshow(overlay)
533     plt.title(f'Grad-CAM | Pred: {pred} | Layer: {layer_name}')
534     plt.axis('off')
535
536     plt.savefig(gradcam_path, bbox_inches='tight')
537     if show:
538         plt.show()
539     plt.close()
540     return gradcam_path
541 except Exception as error:
542     print(f"Grad-CAM sarit: {error}")
543     return None
544 finally:
545     forward_handle.remove()
546     backward_handle.remove()
547
548
549 class ModelEMA:
550     def __init__(self, model, decay):
551         self.decay = decay
552         self.shadow = {
553             name: param.detach().clone()
554             for name, param in model.state_dict().items()
555             if torch.is_floating_point(param)
556         }
557         self.backup = {}
558
559     @torch.no_grad()
560     def update(self, model):
561         model_state = model.state_dict()
562         for name, shadow_param in self.shadow.items():
563             shadow_param.mul_(self.decay).add_(model_state[name].detach(), alpha=1.0 - self.
decay)
564
565     def apply_shadow(self, model):
566         self.backup = {}
567         model_state = model.state_dict()
568         for name, shadow_param in self.shadow.items():
569             self.backup[name] = model_state[name].detach().clone()
570             model_state[name].copy_(shadow_param)
571
572     def restore(self, model):
573         model_state = model.state_dict()
574         for name, backup_param in self.backup.items():
575             model_state[name].copy_(backup_param)
576         self.backup = {}
577
578
579 def get_multiclass_outputs(model, imgs, use_tta=False):
580     outputs = model(imgs)
581     if not use_tta:
582         return outputs
583
584     flipped_outputs = model(torch.flip(imgs, dims=[3]))
585     return (outputs + flipped_outputs) / 2.0
586
587
588 def compute_extra_metrics(labels, preds):
589     try:
590         qwk = cohen_kappa_score(labels, preds, weights='quadratic')
591     except Exception:
592         qwk = 0.0
593
594     try:
595         macro_f1 = f1_score(labels, preds, labels=list(range(5)), average='macro',
zero_division=0)
596     except Exception:
597         macro_f1 = 0.0
598
599     try:
600         balanced_acc = balanced_accuracy_score(labels, preds)

```

```

599     except Exception:
600         balanced_acc = 0.0
601     return qwk, macro_f1, balanced_acc
602
603
604 def evaluate(model, data_loader, criterion, loss_name, device, desc, use_tta=False):
605     model.eval()
606     total_loss, correct, total = 0.0, 0, 0
607     all_preds, all_labels, all_probs = [], [], []
608     use_tta = use_tta and loss_name.lower() not in ['bce_ordinal', 'ordinal']
609
610     with torch.no_grad():
611         progress = tqdm(data_loader, desc=desc, leave=False)
612         for imgs, labels in progress:
613             imgs, labels = imgs.to(device), labels.to(device)
614             outputs = model(imgs)
615             loss = compute_loss(criterion, outputs, labels, loss_name, device)
616             metric_outputs = get_multiclass_outputs(model, imgs, use_tta=use_tta)
617
618             total_loss += loss.item()
619             preds = get_predictions(metric_outputs, loss_name)
620             correct += (preds == labels).sum().item()
621             total += labels.size(0)
622
623             all_preds.extend(preds.cpu().numpy())
624             all_labels.extend(labels.cpu().numpy())
625             all_probs.extend(F.softmax(metric_outputs, dim=1).cpu().numpy())
626
627     avg_loss = total_loss / max(len(data_loader), 1)
628     acc = correct / max(total, 1)
629     try:
630         auc = roc_auc_score(all_labels, all_probs, multi_class='ovr', average='macro')
631     except Exception:
632         auc = 0.0
633     qwk, macro_f1, balanced_acc = compute_extra_metrics(all_labels, all_preds)
634
635     return avg_loss, acc, auc, qwk, macro_f1, balanced_acc, all_preds, all_labels, all_probs
636
637
638 def select_metric(metrics, monitor_metric):
639     return {
640         'acc': metrics['acc'],
641         'auc': metrics['auc'],
642         'qwk': metrics['qwk'],
643         'macro_f1': metrics['macro_f1'],
644         'balanced_acc': metrics['balanced_acc'],
645         'val_loss': metrics['val_loss'],
646     }[monitor_metric]
647
648
649 def is_lower_better_metric(monitor_metric):
650     return monitor_metric == 'val_loss'
651
652
653 def metric_improved(metric, best_metric, monitor_metric, min_delta):
654     if is_lower_better_metric(monitor_metric):
655         return metric < best_metric - min_delta
656     return metric > best_metric + min_delta
657
658
659 def make_result_base_name(base_name, test_source, use_test_suffix):
660     if not use_test_suffix:
661         return base_name
662     safe_test_source = test_source.replace('+', '_').replace('/', '_').replace('\\', '_')
663     return f"{base_name}_test_{safe_test_source}"
664
665
666 def make_experiment_base_name(experiment_name, args):
667     parts = [experiment_name, args.model]
668
669     if args.loss_name != 'focal_loss':
670         parts.append(args.loss_name)
671     if args.optimizer != 'adam':
672         parts.append(args.optimizer)
673     if args.loss_name == 'focal_loss':
674         parts.append(f"g{args.gamma:g}")

```

```

675
676     return "_".join(parts)
677
678
679 def run_final_test(
680     model,
681     test_source,
682     test_loader,
683     criterion,
684     args,
685     device,
686     base_name,
687     results_dirs,
688     use_test_suffix,
689 ):
690     result_base_name = make_result_base_name(base_name, test_source, use_test_suffix)
691
692     print("\n" + "=" * 50)
693     print(f"START TEST FINAL: {test_source}")
694     print("=" * 50)
695
696     test_loss, test_acc, test_auc, test_qwk, test_macro_f1, test_balanced_acc, test_preds,
697     test_labels, test_probs = evaluate(
698         model,
699         test_loader,
700         criterion,
701         args.loss_name,
702         device,
703         desc=f"Testare model [{test_source}]",
704         use_tta=args.tta_eval,
705     )
706
707     print("Rezultate testare finala:")
708     print(f"    Dataset test: {test_source}")
709     print(f"    Acuratete: {test_acc * 100:.2f}%")
710     print(f"    Scor AUC:    {test_auc:.4f}")
711     print(f"    QWK:        {test_qwk:.4f}")
712     print(f"    Macro F1:   {test_macro_f1:.4f}")
713     print(f"    Bal Acc:    {test_balanced_acc:.4f}")
714     print(f"    Loss:       {test_loss:.4f}\n")
715
716     print("Classification report:")
717     prf1_path, report_path, metrics_path, report = save_precision_recall_f1(
718         test_labels,
719         test_preds,
720         result_base_name,
721         results_dirs,
722         show=True
723     )
724     print(report)
725
726     cm_path = os.path.join(results_dirs['confusion'], f"cm_{result_base_name}.png")
727     save_confusion_matrix(test_labels, test_preds, cm_path, normalized=False, show=True)
728
729     cm_norm_path = os.path.join(results_dirs['confusion'], f"cm_norm_{result_base_name}.png")
730     save_confusion_matrix(test_labels, test_preds, cm_norm_path, normalized=True, show=True)
731
732     roc_path = save_roc_curve(test_labels, test_probs, result_base_name, results_dirs, show=
733     True)
734     gradcam_path = save_gradcam_overlay(model, test_loader, args.loss_name, device,
735     result_base_name, results_dirs, show=True)
736
737     print(f"Rezultate pentru test={test_source}:")
738     print(f"Matrice confuzie: {cm_path}")
739     print(f"Matrice confuzie normalizata: {cm_norm_path}")
740     print(f"Precision/Recall/F1: {prf1_path}")
741     print(f"Raport clasificare: {report_path}")
742     print(f"Metrici CSV: {metrics_path}")
743     print(f"ROC curve: {roc_path}")
744     if gradcam_path:
745         print(f"Grad-CAM overlay: {gradcam_path}")
746
747     return {
748         'source': test_source,
749         'loss': test_loss,
750         'acc': test_acc,

```



```

748         'auc': test_auc,
749         'qwk': test_qwk,
750         'macro_f1': test_macro_f1,
751         'balanced_acc': test_balanced_acc,
752         'cm_path': cm_path,
753         'cm_norm_path': cm_norm_path,
754         'prf1_path': prf1_path,
755         'report_path': report_path,
756         'metrics_path': metrics_path,
757         'roc_path': roc_path,
758         'gradcam_path': gradcam_path,
759     }
760
761
762 def main():
763     args = parse_args()
764     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
765
766     balanced_root = args.balanced_root or args.root_dir or os.path.join(PROJECT_ROOT, '
datasets', 'originals', 'Diabetic_Balanced_Data')
767     balanced_aug_root = args.balanced_aug_root or os.path.join(PROJECT_ROOT, 'datasets', '
processed_by_me', 'balanced_aug_new', 'final_augmented+ben_graham_dataset')
768     aptos_root = args.aptos_root or os.path.join(PROJECT_ROOT, 'datasets', 'processed_by_me',
'aptos_old', 'aptos_ben_graham_part_2')
769     my_dataset_root = args.my_dataset_root or os.path.join(PROJECT_ROOT, 'datasets', '
processed_by_me', 'My_dataset')
770     dataset_roots = {
771         'balanced': balanced_root,
772         'balanced_aug': balanced_aug_root,
773         'balanced_aug_final': balanced_aug_root,
774         'aptos': aptos_root,
775         'my_dataset': my_dataset_root,
776         'aptos_new': args.aptos_new_root or os.path.join(PROJECT_ROOT, 'datasets', '
processed_by_me', 'aptos_new', 'aptos_sorted_better_augmented'),
777         'aptos_part_4': args.aptos_part_4_root or os.path.join(PROJECT_ROOT, 'datasets', '
processed_by_me', 'aptos_old', 'aptos_ben_graham_part_4'),
778         'balanced_aug_resized': args.balanced_aug_resized_root or os.path.join(PROJECT_ROOT, '
datasets', 'processed_by_me', 'balanced_aug_new', 'Diabetic_Balanced_Aug_resized_train'),
779         'balanced_aug_resized_cropped': args.balanced_aug_resized_cropped_root or os.path.join
(PROJECT_ROOT, 'datasets', 'processed_by_me', 'balanced_aug_new', '
Diabetic_Balanced_Aug_resized_train_cropped'),
780         'balanced_aug_clean_split': args.balanced_aug_clean_split_root or os.path.join(
PROJECT_ROOT, 'datasets', 'processed_by_me', 'balanced_aug_new', 'Final_split_dataset'),
781         'balanced_data_original': args.balanced_data_original_root or os.path.join(
PROJECT_ROOT, 'datasets', 'processed_by_me', 'balanced_data_split', '
Diabetic_Balanced_Data_original'),
782         'balanced_data_augmented': args.balanced_data_augmented_root or os.path.join(
PROJECT_ROOT, 'datasets', 'processed_by_me', 'balanced_data_split', '
Diabetic_Balanced_Data_augmented'),
783     }
784     train_source, test_source = resolve_experiment(args.experiment, args.train_source, args.
test_source)
785     test_sources = args.test_sources or [test_source]
786     val_source = train_source
787     experiment_name = args.experiment or f"{train_source}_to_{test_source}"
788
789     if args.train_dataset_root and args.train_dataset_roots:
790         raise ValueError("Foloseste fie --train_dataset_root, fie --train_dataset_roots, nu
ambele.")
791
792     if args.train_dataset_root:
793         train_source = 'custom_train'
794         val_source = 'custom_train'
795         dataset_roots[train_source] = resolve_dataset_path(args.train_dataset_root)
796         experiment_name = f"{train_source}_to_{test_source}"
797
798     if args.train_dataset_roots:
799         if args.train_dataset_names and len(args.train_dataset_names) != len(args.
train_dataset_roots):
800             raise ValueError("--train_dataset_names trebuie sa aiba acelasi numar de valori ca
--train_dataset_roots.")
801
802     train_sources = []
803     for index, train_root in enumerate(args.train_dataset_roots, start=1):
804         if args.train_dataset_names:
805             source_name = args.train_dataset_names[index - 1]

```

```

806         else:
807             base_name = os.path.basename(os.path.normpath(train_root)) or f"dataset_{index}"
808             source_name = f"custom_train_{index}_{base_name}"
809
810             source_name = safe_source_name(source_name)
811             dataset_roots[source_name] = resolve_dataset_path(train_root)
812             train_sources.append(source_name)
813
814             train_source = '+'.join(train_sources)
815             val_source = train_source
816             experiment_name = f"{train_source}_to_{test_source}"
817
818     if args.test_dataset_roots:
819         if args.test_dataset_names and len(args.test_dataset_names) != len(args.
test_dataset_roots):
820             raise ValueError("--test_dataset_names trebuie sa aiba acelasi numar de valori ca
--test_dataset_roots.")
821
822     test_sources = []
823     for index, test_root in enumerate(args.test_dataset_roots, start=1):
824         if args.test_dataset_names:
825             source_name = args.test_dataset_names[index - 1]
826         else:
827             base_name = os.path.basename(os.path.normpath(test_root)) or f"dataset_{index}"
828             source_name = f"custom_test_{index}_{base_name}"
829
830             source_name = safe_source_name(source_name)
831             dataset_roots[source_name] = resolve_dataset_path(test_root)
832             test_sources.append(source_name)
833
834             experiment_name = f"{train_source}_to_{'.'.join(test_sources)}"
835
836     # Show an image from each dataset used in training or testing
837     print(f"Sanity check: o imagine din fiecare dataset pentru experimentul '{args.experiment
}':")
838     for source in set([train_source] + test_sources):
839         try:
840             sanity_split = 'train' if source == train_source else 'test'
841             ds = make_dataset(
842                 source,
843                 split=sanity_split,
844                 image_size=args.img_size,
845                 balanced_root=balanced_root,
846                 aptos_root=aptos_root,
847                 balanced_aug_root=balanced_aug_root,
848                 my_dataset_root=my_dataset_root,
849                 dataset_roots=dataset_roots,
850                 train_augment=args.train_augment,
851             )
852             img, label = ds[0]
853             img = denormalize_image(img)
854             plt.figure(figsize=(4, 4))
855             plt.imshow(img)
856             plt.title(f"{source}/{sanity_split} | Label: {label}")
857             plt.axis('off')
858             plt.show()
859         except Exception as e:
860             print(f"    Nu am putut incarca o imagine din {source}: {e}")
861
862     rezultate_dir = os.path.join(CURRENT_DIR, "Rezultate", "New")
863     modele_dir = os.path.join(rezultate_dir, "modele")
864     grafice_dir = os.path.join(rezultate_dir, "grafice")
865     os.makedirs(modele_dir, exist_ok=True)
866     results_dirs = make_results_dirs(grafice_dir)
867
868     print(f"Config: {args.model} | Loss: {args.loss_name} | Opt: {args.optimizer} | LR: {args.
lr}")
869     print(f"Experiment: {experiment_name} | TRAIN/VAL={train_source} | TEST={'.'.join(
test_sources)}")
870     print(f"Train augment: {args.train_augment}")
871     print(f"Monitor metric: {args.monitor_metric} | Scheduler: {args.scheduler}")
872     print(f"AMP: {args.amp} | Grad clip: {args.grad_clip} | EMA decay: {args.ema_decay} | TTA
eval: {args.tta_eval}")
873     print(

```

```

874         f"Model dropout: {args.model_dropout} | Freeze backbone epochs: {args.
freeze_backbone_epochs} | "
875         f"Backbone LR mult: {args.backbone_lr_mult}"
876     )
877     if args.model.lower() == 'incres_v2' and args.img_size < 299:
878         print(
879             "Atentie: incres_v2 este de obicei folosit cu input >= 299. "
880             "Pentru rularea principala ia in calcul --img_size 299 sau 384."
881         )
882     print(f"Balanced root: {balanced_root}")
883     print(f"Balanced Aug root: {balanced_aug_root}")
884     print(f"APTOS root: {aptos_root}")
885     print(f"My dataset root: {my_dataset_root}")
886     print("Extra dataset roots:")
887     for source_name in sorted(dataset_roots):
888         if source_name not in {'balanced', 'balanced_aug', 'balanced_aug_final', 'aptos', '
my_dataset'}:
889             print(f" {source_name}: {dataset_roots[source_name]}")
890
891     train_ds = make_dataset(
892         train_source,
893         split='train',
894         image_size=args.img_size,
895         balanced_root=balanced_root,
896         aptos_root=aptos_root,
897         balanced_aug_root=balanced_aug_root,
898         my_dataset_root=my_dataset_root,
899         dataset_roots=dataset_roots,
900         train_augment=args.train_augment,
901     )
902     val_ds = make_dataset(
903         val_source,
904         split='val',
905         image_size=args.img_size,
906         balanced_root=balanced_root,
907         aptos_root=aptos_root,
908         balanced_aug_root=balanced_aug_root,
909         my_dataset_root=my_dataset_root,
910         dataset_roots=dataset_roots,
911         train_augment=args.train_augment,
912     )
913     test_datasets = {}
914     for current_test_source in test_sources:
915         test_datasets[current_test_source] = make_dataset(
916             current_test_source,
917             split='test',
918             image_size=args.img_size,
919             balanced_root=balanced_root,
920             aptos_root=aptos_root,
921             balanced_aug_root=balanced_aug_root,
922             my_dataset_root=my_dataset_root,
923             dataset_roots=dataset_roots,
924             train_augment=args.train_augment,
925         )
926
927     train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True, num_workers=
args.num_workers)
928     val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False, num_workers=
args.num_workers)
929     test_loaders = {
930         current_test_source: DataLoader(test_ds, batch_size=args.batch_size, shuffle=False,
num_workers=args.num_workers)
931         for current_test_source, test_ds in test_datasets.items()
932     }
933
934     # Daca avem focal loss, afisam valoarea gamma
935     if args.loss_name == 'focal_loss':
936         print(f"Focal Loss activat cu gamma={args.gamma}")
937     test_sizes = ' | '.join([f"TESTARE[{source}]= {len(dataset)}" for source, dataset in
test_datasets.items()])
938     print(f"Distributie date: TRAIN={len(train_ds)} | VALIDARE={len(val_ds)} | {test_sizes}")
939
940     model = get_model(args.model, num_classes=5, drop_rate=args.model_dropout).to(device)
941     if args.freeze_backbone_epochs > 0:
942         set_backbone_trainable(model, trainable=False)
943         print(

```

```

944         f"Backbone inghetat pentru primele {args.freeze_backbone_epochs} epoci. "
945         f"Parametri antrenabili initial: {count_trainable_parameters(model):,}"
946     )
947     criterion = get_loss_function(args.loss_name, class_weights=args.class_weights, device=
device, gamma=args.gamma)
948     optimizer = get_optimizer(
949         model,
950         optimizer_name=args.optimizer,
951         lr=args.lr,
952         backbone_lr_mult=args.backbone_lr_mult,
953     )
954     if args.scheduler == 'plateau':
955         scheduler_mode = 'min' if is_lower_better_metric(args.monitor_metric) else 'max'
956         scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode=scheduler_mode,
factor=0.75, patience=5)
957     elif args.scheduler == 'cosine':
958         scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=max(args.
epochs, 1), eta_min=args.lr * 0.01)
959     else:
960         scheduler = None
961     use_amp = args.amp and device.type == 'cuda'
962     scaler = torch.amp.GradScaler(device.type, enabled=use_amp)
963     ema = ModelEMA(model, args.ema_decay) if args.ema_decay > 0 else None
964
965     history = {
966         'train_loss': [],
967         'val_loss': [],
968         'val_acc': [],
969         'val_auc': [],
970         'val_qwk': [],
971         'val_macro_f1': [],
972         'val_balanced_acc': [],
973     }
974     best_metric = float('inf') if is_lower_better_metric(args.monitor_metric) else -float('inf')
975
976     epochs_without_improvement = 0
977     stopped_early = False
978     stopped_epoch = None
979     base_name = make_experiment_base_name(experiment_name, args)
980     model_path = os.path.join(modele_dir, f"best_{base_name}.pth")
981
982     for epoch in range(args.epochs):
983         start_time = time.time()
984         if args.freeze_backbone_epochs > 0 and epoch == args.freeze_backbone_epochs:
985             set_backbone_trainable(model, trainable=True)
986             print(
987                 f"Backbone deblocat la epoca {epoch + 1}. "
988                 f"Parametri antrenabili: {count_trainable_parameters(model):,}"
989             )
990         model.train()
991         train_loss = 0.0
992
993         progress = tqdm(train_loader, desc=f"Ep {epoch + 1:02d} [TRAIN]", leave=False)
994         for imgs, labels in progress:
995             imgs, labels = imgs.to(device), labels.to(device)
996             optimizer.zero_grad(set_to_none=True)
997             with torch.amp.autocast(device.type, enabled=use_amp):
998                 outputs = model(imgs)
999                 loss = compute_loss(criterion, outputs, labels, args.loss_name, device)
1000
1001             scaler.scale(loss).backward()
1002             if args.grad_clip > 0:
1003                 scaler.unscale_(optimizer)
1004                 torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
1005             scaler.step(optimizer)
1006             scaler.update()
1007             if ema is not None:
1008                 ema.update(model)
1009
1010             train_loss += loss.item()
1011             progress.set_postfix({'loss': f"{loss.item():.4f}"})
1012
1013         epoch_train_loss = train_loss / max(len(train_loader), 1)
1014         if ema is not None:
1015             ema.apply_shadow(model)

```

```

1015     val_loss, val_acc, val_auc, val_qwk, val_macro_f1, val_balanced_acc, _, _, _ =
evaluate(
1016     model,
1017     val_loader,
1018     criterion,
1019     args.loss_name,
1020     device,
1021     desc=f"Ep {epoch + 1:02d} [VALID]",
1022     use_tta=args.tta_eval,
1023 )
1024 if ema is not None:
1025     ema.restore(model)
1026
1027 history['train_loss'].append(epoch_train_loss)
1028 history['val_loss'].append(val_loss)
1029 history['val_acc'].append(val_acc)
1030 history['val_auc'].append(val_auc)
1031 history['val_qwk'].append(val_qwk)
1032 history['val_macro_f1'].append(val_macro_f1)
1033 history['val_balanced_acc'].append(val_balanced_acc)
1034
1035 metrics = {
1036     'acc': val_acc,
1037     'auc': val_auc,
1038     'qwk': val_qwk,
1039     'macro_f1': val_macro_f1,
1040     'balanced_acc': val_balanced_acc,
1041     'val_loss': val_loss,
1042 }
1043 metric = select_metric(metrics, args.monitor_metric)
1044 if scheduler is not None:
1045     if args.scheduler == 'plateau':
1046         scheduler.step(metric)
1047     else:
1048         scheduler.step()
1049 marker = ""
1050 if metric_improved(metric, best_metric, args.monitor_metric, args.
early_stopping_min_delta):
1051     best_metric = metric
1052     epochs_without_improvement = 0
1053     if ema is not None:
1054         ema.apply_shadow(model)
1055     torch.save(model.state_dict(), model_path)
1056     if ema is not None:
1057         ema.restore(model)
1058     marker = "NOU BEST"
1059 else:
1060     epochs_without_improvement += 1
1061
1062 print(
1063     f"Ep {epoch + 1:02d}/{args.epochs} | "
1064     f"T_Loss: {epoch_train_loss:.4f} | V_Loss: {val_loss:.4f} | "
1065     f"V_Acc: {val_acc * 100:.1f}% | V_AUC: {val_auc:.4f} | "
1066     f"V_QWK: {val_qwk:.4f} | V_F1: {val_macro_f1:.4f} | "
1067     f"V_BalAcc: {val_balanced_acc:.4f} | Best[{args.monitor_metric}]= {best_metric:.4f}
{marker}"
1068 )
1069 print(f"LR curent: {format_lrs(optimizer)}")
1070 print(f"Durata epoca: {(time.time() - start_time) / 60:.2f} min")
1071 if args.early_stopping_patience > 0:
1072     print(
1073         f"Early stopping: {epochs_without_improvement}/"
1074         f"{args.early_stopping_patience} epoci fara imbunatatire"
1075     )
1076     if epochs_without_improvement >= args.early_stopping_patience:
1077         stopped_early = True
1078         stopped_epoch = epoch + 1
1079         print(
1080             f"Early stopping activat la epoca {stopped_epoch}. "
1081             f"Cea mai buna metrica: {best_metric:.4f}"
1082         )
1083         break
1084
1085 model.load_state_dict(torch.load(model_path, map_location=device))
1086

```

```

1087 # Afisare grafice de acuratete, loss, AUC, matricile de confuzie (normala + normalizata),
1088 Grad-CAM, curba ROC
1089 # Am adaugat paramtetrul show=True ca sa apara pozele dupa testare
1090
1091 history_path = os.path.join(results_dirs['history'], f"istoric_combinat_{base_name}.png")
1092 save_combined_history(history, history_path, show=True)
1093
1094 loss_path = os.path.join(results_dirs['loss'], f"loss_{base_name}.png")
1095 save_line_plot(history['train_loss'], 'Loss Antrenament', 'Loss', loss_path, color='blue',
1096 label='Train Loss', show=True)
1097
1098 acc_path = os.path.join(results_dirs['accuracy'], f"acc_{base_name}.png")
1099 save_line_plot(history['val_acc'], 'Acuratete Validare', 'Acuratete', acc_path, color='
1100 orange', label='Val Acc', show=True)
1101
1102 auc_path = os.path.join(results_dirs['auc'], f"auc_{base_name}.png")
1103 save_line_plot(history['val_auc'], 'Scor AUC Validare', 'AUC', auc_path, color='green',
1104 label='Val AUC', show=True)
1105
1106 qwk_path = os.path.join(results_dirs['qwk'], f"qwk_{base_name}.png")
1107 save_line_plot(history['val_qwk'], 'Quadratic Weighted Kappa Validare', 'QWK', qwk_path,
1108 color='purple', label='Val QWK', show=True)
1109
1110 macro_f1_path = os.path.join(results_dirs['macro_f1'], f"macro_f1_{base_name}.png")
1111 save_line_plot(history['val_macro_f1'], 'Macro F1 Validare', 'Macro F1', macro_f1_path,
1112 color='red', label='Val Macro F1', show=True)
1113
1114 balanced_acc_path = os.path.join(results_dirs['balanced_acc'], f"balanced_acc_{base_name}.
1115 png")
1116 save_line_plot(
1117     history['val_balanced_acc'],
1118     'Balanced Accuracy Validare',
1119     'Balanced Accuracy',
1120     balanced_acc_path,
1121     color='brown',
1122     label='Val Balanced Acc',
1123     show=True,
1124 )
1125
1126 use_test_suffix = len(test_loaders) > 1
1127 test_results = []
1128 for current_test_source, current_test_loader in test_loaders.items():
1129     test_results.append(run_final_test(
1130         model=model,
1131         test_source=current_test_source,
1132         test_loader=current_test_loader,
1133         criterion=criterion,
1134         args=args,
1135         device=device,
1136         base_name=base_name,
1137         results_dirs=results_dirs,
1138         use_test_suffix=use_test_suffix,
1139     ))
1140
1141 print(f"\nGata. Model: {model_path}")
1142 print(f"Grafic evolutie combinat: {history_path}")
1143 print(f"Grafic loss: {loss_path}")
1144 print(f"Grafic acuratete: {acc_path}")
1145 print(f"Grafic AUC: {auc_path}")
1146 print(f"Grafic QWK: {qwk_path}")
1147 print(f"Grafic Macro F1: {macro_f1_path}")
1148 print(f"Grafic Balanced Acc: {balanced_acc_path}")
1149 if stopped_early:
1150     print(f"Antrenare oprita anticipat la epoca {stopped_epoch}/{args.epochs}.")
1151 if len(test_results) > 1:
1152     print("Rezumat testari finale:")
1153     for result in test_results:
1154         print(
1155             f"    {result['source']}: "
1156             f"Acc={result['acc'] * 100:.2f}% | AUC={result['auc']:.4f} | "
1157             f"QWK={result['qwk']:.4f} | F1={result['macro_f1']:.4f} | "
1158             f"BalAcc={result['balanced_acc']:.4f} | Loss={result['loss']:.4f}"
1159         )
1160
1161 if __name__ == '__main__':

```

```
1156     if not hasattr(sys.modules['__main__'], '__spec__'):
1157         sys.modules['__main__'].__spec__ = None
1158     main()
```

Listing A.3: train.py – antrenarea